

臺灣自編大學教科書

大模型導論

Introduction to Large Language Models

第 5 篇

算力、系統與現代架構

Compute, Systems, and Modern Architectures

第 18 章 GPU、混合精度、記憶體與高效核心

第 19 章 分散式訓練與多卡系統

第 20 章 現代架構、長上下文與混合專家

第 21 章 臺灣算力生態、資源盤點與永續成本

李世淵（機器人叫獸） 編著

助教 李天宇、李宇晴

2026 年 8 月 第一版

序言 把「我們沒有算力」變成一個可以計算的問題

「臺灣沒有算力」——這句話你會在很多場合聽到，而且它有一部分是真的。但如果你把它當成一個結論，你就不會去做接下來該做的事。

這一冊要把它拆成四個可以計算的問題：我手上這張卡為什麼跑不滿？加卡值得嗎？架構該怎麼選？臺灣到底有什麼資源、要多少錢、排放多少碳？

四個問題答完之後，「沒有算力」會變成一句更精確的話——而精確的話才能被行動。

一個貫穿全篇的立場

本冊有一個反覆出現的順序：先把手上的做好，再去要更多。

第 19 章開頭就寫著：在一張沒調好的卡上加卡，只是把低效率乘以卡數。如果你的 MFU 只有 0.12，先把它提升到 0.30——那等同於免費得到 2.5 倍的加速，而且不增加任何複雜度、不多花一毛錢、不多排一克碳。

第 21 章的申請審查也是同一個邏輯：審查者最擔心的不是你能力不足，而是「申請了大量資源卻因為不會用而閒置」。所以能證明你已經在小規模上跑通，比宣稱你會做什麼更有說服力。

這個順序不只是策略，它也是誠實的：多數宣稱資源不足的事案，如果真的去量一次 GPU 使用率，會發現數字落在 30% 到 50% 之間。那不是誰的錯，但它意味著在買下一張卡之前，還有別的事可以做。

四章，四個工具箱

第 18 章給你診斷單卡的能力。核心概念是運算強度與屋頂線——它讓「跑得慢」從一句抱怨變成一個可以定位的問題。你會發現所有的逐元素運算都是頻寬受限的，也會理解為什麼 FlashAttention 做了更多次浮點運算卻更快。

第 19 章處理多卡。它教的不只是怎麼做，更是怎麼判斷該不該做——章末的系統案例明確寫著：若你的結論是「不需要多卡」，那同樣是合格的交付，而且常常是正確答案。

第 20 章是全書技術半衰期最短的一章。它介紹長上下文、稀疏注意力、MoE 與狀態空間模型，但重點不在記住每個方法，而在建立「這個技術解決哪個瓶頸、付出什麼代價」的分析框架。技術會過時，框架不會。

第 21 章把一切落回臺灣。國網中心有什麼、怎麼申請、電費多少、碳排多少、以及一個只有兩張卡的實驗室該怎麼規劃三年。這一章的所有數字都標註了查核日，因為它們每年都在變。

幾個值得先知道的結論

逐元素運算占計算不到 2%，卻占了記憶體流量的四成。這解釋了為什麼核心融合能帶來一至三成的加速。

梯度累積時忘記使用 `no_sync`，會讓通訊量變成累積步數倍。累積 8 步時，原本占 10% 的通訊會變成占 47%——這是多卡訓練中最常見、也最容易被忽略的效能問題。

ZeRO-1 只分片最佳化器狀態，就能把 7 B 模型在 8 卡下的每卡記憶體從 112 GB 降到 38.5 GB，而額外通訊幾乎為零。

滑動視窗讓 KV cache 有上界、不隨對話長度增長——這讓長時間的多輪對話在有限記憶體下變得可行。

MoE 省的是計算，不是記憶體。總參數 143 B 的模型即使活躍只有 9 B，你仍需把 143 B 全部載入。

把 MFU 提升一倍，等同於電費減半、碳排減半、時間減半。多數降低環境成本的手段，同時也降低金錢與時間成本——它們不是取捨。

關於本冊的資料時效

第 21 章引用了具體的設施規格、電價與電力排碳係數，查核日為 2026 年 8 月。這些數字會變：電價每年審議約兩次、排碳係數每年公告一次、設施每隔數年更新。

本書的處理方式是：所有版本敏感的數值都標註來源與查核日，並在第 21.6 節提供一份查核清單與腳本。使用前請重新確認，並在你的報告中註明你的查核日——這與第 4 章建立的環境指紋原則是同一件事。

教師若採用本冊授課，建議每學期開課前跑一次那份清單並更新講義。讓學生看到你自己也在做版本管理，比講十遍原則有效。

關於用語

本書一律使用臺灣的專業用語與教育部標準用字。以下是本冊特別容易混淆的幾組：

本書用語	對應英文	說明與區別
平行	parallel	資料平行、張量平行、管線平行。不使用「並行」。
並行	concurrent	指同時服務的請求數，如「並行請求」。與上一列刻意區分。
分散式	distributed	多機多卡的訓練或部署。

本書用語	對應英文	說明與區別
叢集	cluster	不使用「集群」。
管線	pipeline	管線平行、資料管線。不使用「流水線」。
執行緒	thread	不使用「線程」。行程為 process。
最佳化	optimization	不使用「優化」。最佳化器即 optimizer。
正規化	normalization	層正規化。不使用「歸一化」。
激勵函數	activation function	不使用「激活函數」。函數為數學意義。
函式	function (程式)	程式中的可呼叫單元。與上一列刻意區分。
遮罩	mask	不使用「掩碼」。
快取	cache	KV cache 依慣例保留英文。不使用「緩存」。
頻寬	bandwidth	不使用「帶寬」。
顯示記憶體	VRAM	亦可直接用 VRAM。不使用「顯存」。
運算強度	arithmetic intensity	FLOPs 除以搬移位元組數。
脊點	ridge point	屋頂線模型中計算受限與頻寬受限的分界。
稀疏注意力	sparse attention	不使用「疏鬆注意力」。
視窗	window	滑動視窗。不使用「窗口」。
專家	expert	MoE 的組成單元。專家平行即 expert parallelism。
路由	routing	MoE 中決定使用哪些專家的機制。
注意力沉沒	attention sink	注意力集中於序列開頭的現象。
外推	extrapolation	位置外推。不使用「外插」。
餘裕	headroom	預算中保留的緩衝。不使用「餘量」。
排碳係數	emission factor	每度電的碳排放量，由能源署公告。

其中最需要注意的是「平行」與「並行」的區分：前者指 parallel (資料平行、張量平行)，後者指 concurrent (同時服務的請求數)。兩者在本冊都會出現，混用會造成誤解。

英文專有名詞在首次出現時並列原文，其後視情況直接使用英文（例如 FlashAttention、ZeRO、FSDP、MoE、PUE）——這反映臺灣工程現場的真實習慣。

李世淵（機器人叫獸）

2026 年 8 月 於雲林

目 錄

序言 把「我們沒有算力」變成一個可以計算的問題

目錄

全書架構總覽

第 18 章 GPU、混合精度、記憶體與高效核心

GPUs, Mixed Precision, Memory, and Efficient Kernels

學習目標

18.1 GPU 的執行模型

18.2 運算強度與屋頂線

18.3 浮點格式

18.4 記憶體去哪裡了

18.5 FlashAttention 與融合核心

18.6 torch.compile 與圖最佳化

18.7 用 profiler 定位瓶頸

程式實作

系統案例：把訓練腳本的 MFU 提升一倍

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

附：效能自檢清單

本章小結

成果檢核

第 19 章 分散式訓練與多卡系統

Distributed Training and Multi-GPU Systems

學習目標

19.1 什麼時候該用多卡

19.2 資料平行

19.3 ZeRO 與分片

19.4 張量平行與管線平行

19.5 3D 平行：把三種組合起來

19.6 分散式環境的正確性

19.7 多卡的除錯

程式實作

系統案例：分散式訓練的決策報告

章末評量

研究延伸與版本注意事項

常見問題
教師使用建議
本章小結
成果檢核

第 20 章 現代架構、長上下文與混合專家

Modern Architectures, Long Context, and Mixture-of-Experts

學習目標

- 20.1 長上下文的三重瓶頸
- 20.2 位置外推
- 20.3 稀疏與局部注意力
- 20.4 混合專家 (MoE)
- 20.5 狀態空間模型與線性注意力
- 20.6 長上下文的評測
- 20.7 架構選型的決策框架

程式實作

系統案例：法規彙編的長上下文評估

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

附：本章技術的瓶頸對照表

本章小結

成果檢核

第 21 章 臺灣算力生態、資源盤點與永續成本

Taiwan Compute Ecosystem, Resource Planning, and Sustainability Costs

學習目標

- 21.1 臺灣的算力地圖
- 21.2 取得國家級算力
- 21.3 從 GPU 小時到電力與碳排
- 21.4 四種方案的總持有成本
- 21.5 把環境成本納入規劃
- 21.6 資料時效與查核清單
- 21.7 實驗室的資源治理

程式實作

系統案例：實驗室三年技術路線規劃

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議
附：本章速查表
本章小結
成果檢核

索引

以下為 Word 自動目錄欄位。在 Word 中開啟後，於下方按右鍵選擇「更新功能變數」即可產生含頁碼的三層目錄；亦可使用左側導覽窗格依書籤瀏覽。

序言 把「我們沒有算力」變成一個可以計算的問題.....	2
一個貫穿全篇的立場.....	2
四章，四個工具箱.....	2
幾個值得先知道的結論.....	3
關於本冊的資料時效.....	3
關於用語.....	3
目 錄.....	6
全書架構總覽.....	15
本冊四章的能力遞進.....	15
第 5 篇.....	17
GPU、混合精度、記憶體與高效核心.....	18
學習目標.....	18
18.1 GPU 的執行模型.....	18
18.1.1 與 CPU 的根本差異.....	18
18.1.2 記憶體階層.....	19
18.1.3 Tensor Core：為矩陣乘法而生的硬體.....	19
18.2 運算強度與屋頂線.....	20
18.2.1 兩種瓶頸.....	20
18.2.2 常見運算的運算強度.....	20
18.2.3 為什麼要融合核心.....	21
18.2.4 把屋頂線用在整個模型上.....	21
18.3 浮點格式.....	22
18.3.1 六種格式的完整比較.....	22
18.3.2 為什麼 BF16 勝過 FP16.....	23
18.3.3 FP8 與更低精度.....	23
18.4 記憶體去哪裡了.....	24
18.4.1 四大類.....	24
18.4.2 實際數字.....	24
18.4.3 activation checkpointing.....	24

18.4.5 記憶體碎片化.....	26
18.4.4 OOM 排除清單.....	27
18.5 FlashAttention 與融合核心.....	28
18.5.1 問題重述.....	28
18.5.2 核心想法：分塊與線上 softmax.....	28
18.5.3 怎麼確認它真的被啟用.....	29
18.5.4 其他可融合的部分.....	30
18.5.5 為什麼記憶體節省能換來更長的上下文.....	30
18.5.6 融合核心的一般原則.....	31
18.6 torch.compile 與圖最佳化.....	31
18.6.1 它做什麼.....	32
18.6.2 實務注意事項.....	32
18.6.3 最佳化器的融合版本.....	33
18.7 用 profiler 定位瓶頸.....	34
18.7.1 診斷流程.....	34
18.7.2 常見的熱點與意義.....	35
18.7.3 從診斷到行動.....	36
程式實作.....	37
程式 18A 記憶體剖析器.....	37
程式 18B 效能矩陣測試.....	37
程式 18C 瓶頸定位與最佳化實戰.....	38
系統案例：把訓練腳本的 MFU 提升一倍.....	39
為什麼設定「提升一倍」這個目標.....	39
必須交付的六項.....	39
評分重點.....	40
章末評量.....	41
一、概念題.....	41
二、計算題.....	41
三、除錯題.....	41
四、系統設計題.....	42
五、臺灣情境與倫理題.....	42
評量提示（給自學者）.....	42
研究延伸與版本注意事項.....	42
常見問題.....	43
教師使用建議.....	43
附：效能自檢清單.....	44
本章小結.....	45
成果檢核.....	46
分散式訓練與多卡系統.....	48

學習目標.....	48
19.1 什麼時候該用多卡.....	48
19.1.1 三個理由，三種答案.....	48
19.1.2 擴展的損益兩平.....	49
19.1.3 多卡帶來的隱性成本.....	49
19.2 資料平行.....	50
19.2.1 原理.....	50
19.2.2 通訊量.....	50
19.2.3 通訊與計算的重疊.....	51
19.2.4 實作.....	51
19.2.5 資料平行的隱形前提.....	53
19.3 ZeRO 與分片.....	54
19.3.1 資料平行的浪費.....	54
19.3.2 三個階段.....	54
19.3.3 ZeRO-3 的代價.....	55
19.4 張量平行與管線平行.....	57
19.4.1 張量平行：把單層切開.....	57
19.4.2 管線平行：把層分給不同的卡.....	57
19.5 3D 平行：把三種組合起來.....	58
19.5.1 組合的原則.....	58
19.5.2 該用幾維.....	59
19.5.3 專家平行：MoE 的特殊需求.....	59
19.5.4 一張決策流程圖.....	60
19.6 分散式環境的正確性.....	60
19.6.1 隨機種子：哪些該相同、哪些該不同.....	60
19.6.2 資料切分.....	62
19.6.3 checkpoint 的處理.....	63
19.7 多卡的除錯.....	63
19.7.1 六種常見問題.....	63
19.7.2 診斷工具.....	64
19.7.3 容錯與長時間訓練.....	65
19.7.4 一份多卡啟動前的檢查清單.....	66
程式實作.....	67
程式 19A 資料平行訓練腳本.....	67
程式 19B 擴展效率量測.....	68
程式 19C ZeRO 記憶體實測.....	68
系統案例：分散式訓練的決策報告.....	69
要回答的五個問題.....	69
交付要求.....	69

評分重點	70
章末評量	71
一、概念題	71
二、計算題	71
三、除錯題	71
四、系統設計題	72
五、臺灣情境與倫理題	72
評量提示（給自學者）	72
研究延伸與版本注意事項	72
常見問題	73
教師使用建議	73
本章小結	74
成果檢核	75
現代架構、長上下文與混合專家	76
學習目標	76
20.1 長上下文的三重瓶頸	76
20.1.1 三個獨立的問題	76
20.1.2 「支援」與「能用」的差距	77
20.1.3 對臺灣應用的意義	77
20.2 位置外推	78
20.2.1 問題的本質	78
20.2.2 三種延長方法	79
20.2.3 外推需要微調	80
20.3 稀疏與局部注意力	80
20.3.1 滑動視窗	80
20.3.2 混合注意力	81
20.3.3 其他稀疏模式	81
20.3.4 KV cache 的縮減	82
20.3.5 注意力沉沒與串流處理	82
20.4 混合專家（MoE）	83
20.4.1 核心想法	83
20.4.2 參數量的計算	84
20.4.3 路由與負載平衡	84
20.4.4 MoE 適合誰	85
20.4.5 MoE 的推論特性	86
20.5 狀態空間模型與線性注意力	86
20.5.1 回到遞迴	86
20.5.2 現代狀態空間模型的關鍵改進	87
20.5.3 混合架構	87

20.6 長上下文的評測	88
20.6.1 為什麼需要專門的評測	88
20.6.2 大海撈針測試	88
20.6.3 多針與多跳測試	89
20.6.5 長度衰減的量化	90
20.6.4 該報告什麼	91
20.7 架構選型的決策框架	91
20.7.1 三個層次的問題	91
20.7.2 一張選型對照表	92
20.7.3 繁體中文的額外考量	92
程式實作	94
程式 20A 位置編碼延長實驗	94
程式 20B 長上下文評測套件	95
程式 20C 架構特性比較	95
系統案例：法規彙編的長上下文評估	96
情境設定	96
要回答的四個問題	96
交付要求	97
評分重點	97
章末評量	98
一、概念題	98
二、計算題	98
三、除錯題	98
四、系統設計題	99
五、臺灣情境與倫理題	99
評量提示（給自學者）	99
研究延伸與版本注意事項	99
常見問題	100
教師使用建議	100
附：本章技術的瓶頸對照表	101
本章小結	102
成果檢核	103
臺灣算力生態、資源盤點與永續成本	104
學習目標	104
21.1 臺灣的算力地圖	104
21.1.1 五種來源	104
21.1.2 國家級設施的角色	105
21.1.3 能效：一個被低估的指標	105
21.1.4 規模的實感：把數字翻譯成經驗	106

21.2	取得國家級算力	107
21.2.1	申請的基本條件	107
21.2.2	審查會看什麼	107
21.2.3	一份申請書的骨架	107
21.2.4	常見的申請失誤	108
21.3	從 GPU 小時到電力與碳排	108
21.3.1	完整的換算鏈	108
21.3.2	電力排碳係數	109
21.3.3	PUE：機房的隱形成本	109
21.3.4	實際算幾個例子	110
21.3.5	電價	110
21.3.6	把碳排寫進實驗紀錄	110
21.4	四種方案的總持有成本	112
21.4.1	比較的架構	112
21.4.2	三年期的總持有成本	112
21.4.3	決策準則	112
21.4.4	一個完整的比較範例	113
21.4.5	混合策略	114
21.5	把環境成本納入規劃	114
21.5.1	什麼規模以上該考慮	114
21.5.2	減少環境成本的手段	115
21.5.3	該不該揭露	116
21.6	資料時效與查核清單	116
21.6.1	本章哪些資料會過期	116
21.6.2	查核腳本	117
21.7	實驗室的資源治理	118
21.7.1	共用制度的三個問題	118
21.7.2	一個可行的時段制度	118
21.7.3	使用率的量測	118
	程式實作	121
	程式 21A 資源盤點與成本計算器	121
	程式 21B 算力申請書產生器	122
	程式 21C 實驗室資源監控	122
	系統案例：實驗室三年技術路線規劃	123
	情境設定	123
	要規劃的三個年度	123
	必須回答的六個問題	124
	交付要求	124
	評分重點	124

章末評量	126
一、概念題	126
二、計算題	126
三、除錯題	126
四、系統設計題	127
五、臺灣情境與倫理題	127
評量提示（給自學者）	127
研究延伸與版本注意事項	127
常見問題	128
教師使用建議	128
附：本章速查表	129
本章小結	130
成果檢核	130
索 引	132
英文詞條	132
中文詞條	132

全書架構總覽

全書分為十篇四十章，另有十四組附錄。本冊為第 5 篇，是第二學期的系統工程主線，也是從「模型」轉向「資源」的視角切換。

篇次	章次	主題	核心產出
第 1 篇	1–4	大模型視野、數學與工程基礎	需求規格書、技術演進圖、數學速查表、可重現環境
第 2 篇	5–9	語言建模、臺灣語料與 Token 工程	基準評量規格、文本工程規格、自訓 tokenizer、語料清冊
第 3 篇	10–13	Transformer 從零實作	MiniFormosaGPT 與可生成繁體中文的模型
第 4 篇	14–17	預訓練、解碼與實驗科學	解碼工具箱、工業級訓練迴圈、實驗紀律、算力預算書
第 5 篇 (本冊)	18–21	算力、系統與現代架構	效能剖析、分散式決策、架構評估、臺灣算力盤點與三年路線
第 6 篇	22–26	本土化後訓練與對齊	繁中適配模型、LoRA、偏好對齊、邊緣小模型
第 7 篇	27–30	推理、知識更新與提示工程	推理實驗、知識更新流程、RAG 原型
第 8 篇	31–34	應用系統：進階 RAG、Agent 與產品開發	知識系統、受限 Agent、可上線的應用
第 9 篇	35–37	部署、評測、安全與治理	量化部署、臺灣題庫、紅隊與影響評估
第 10 篇	38–40	多模態、具身智慧與總整專題	VLM 應用、臺語語音、VLA、總整專題

本冊四章的能力遞進

章	回答什麼問題	你會做出什麼	被誰使用
第 18 章	單卡為什麼慢？	記憶體剖析器、效能矩陣、12 項自檢清單	第 19、26、35 章

章	回答什麼問題	你會做出什麼	被誰使用
第 19 章	該不該用多卡？	分散式腳本、擴展效率報告、決策流程	第 21、22、40 章
第 20 章	架構怎麼選？	長上下文評測套件、瓶頸對照表	第 22、30、35 章
第 21 章	臺灣有什麼資源？	成本計算器、申請書、三年路線規劃	第 22、26、40 章

第 5 篇

算力、系統與現代架構

Compute, Systems, and Modern Architectures

第 18–21 章

本篇承諾：把硬體從黑盒子變成可診斷的系統。效能剖析、分散式決策、架構選型與臺灣算力盤點，讓資源用在刀口上。

第 18 章

GPU、混合精度、記憶體與高效核心

GPUs, Mixed Precision, Memory, and Efficient Kernels

難度：核心 / 進階 | 建議授課：第 18 週（第二學期初） | 硬體需求：A 級可完成概念與 CPU 量測；GPU 實測需 B 級

叫獸開講

GPU 不是「比較快的 CPU」。它是一台需要你不斷餵資料的怪獸，餵不夠快，八萬元的卡就在那裡空轉。第 15 章你算出 MFU 只有 0.2，卻不知道那 0.8 跑到哪裡去了——這一章就是要把它找出來。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明 GPU 的執行模型與記憶體階層，並解釋它與 CPU 的根本差異。
2. 用運算強度與屋頂線模型判斷一個運算是計算受限還是頻寬受限。
3. 比較六種浮點格式的範圍與精度，說明為什麼 BF16 取代了 FP16。
4. 完整拆解訓練的記憶體占用，並用實測驗證你的估算。
5. 說明 activation checkpointing 的原理與取捨，並計算它的節省倍數。
6. 解釋 FlashAttention 與融合核心為什麼能同時省記憶體與加速。
7. 使用 profiler 定位效能瓶頸，並判斷它屬於哪一類。
8. 套用至少三種最佳化手段，並量測前後的 MFU 變化。
9. 建立一份 OOM 排除清單，能在十分鐘內找出記憶體不足的原因。

本章與第 15 章的關係

第 15 章教你量 MFU，本章教你在 MFU 低的時候怎麼辦。兩者的分工是：前者告訴你「有問題」，後者告訴你「問題在哪、怎麼修」。本章的所有工具在第 19 章的分散式訓練與第 35 章的推論部署都會用到——那時候的效能問題更複雜，但診斷方法是同一套。

18.1 GPU 的執行模型

18.1.1 與 CPU 的根本差異

面向	CPU	GPU	意涵
核心數	數個至數十個	數千至上萬	GPU 靠數量取勝
單核能力	強、有複雜的分支預測	弱、簡單	GPU 不適合分支密集的邏輯
執行方式	各核心獨立	成群執行同一指令	同一群內的分支會被序列化
快取	大而多層	小、需手動管理	資料重用要靠程式設計
延遲處理	靠快取與預測降低延遲	靠切換大量執行緒隱藏延遲	需要足夠的平行工作量

最後一列是理解 GPU 的關鍵。GPU 不試圖降低單次記憶體存取的延遲——它的策略是：當一群執行緒在等資料時，就切換到另一群去做事。所以要讓 GPU 跑得快，你必須給它「夠多的平行工作」，否則它會在那裡等待而無事可做。

這解釋了第 4 章程式 4B 的觀察：矩陣越大，達到的 TFLOPS 越高。小矩陣的平行工作量不足以填滿 GPU，也不足以隱藏記憶體延遲。

18.1.2 記憶體階層

層級	容量	相對頻寬	誰在用
暫存器	每執行緒數十個	最快	單一執行緒的區域變數
共享記憶體	每個運算單元數十 KB	很快	同一群執行緒共享的資料
L2 快取	數十 MB	快	自動管理
顯示記憶體	8 至 80 GB	基準 (數百 GB/s 至數 TB/s)	模型權重、activation
主機記憶體	數十至數百 GB	慢 (受 PCIe 限制)	offload 時使用

這個階層有一個關鍵的實務意涵：同樣的資料重複讀取顯示記憶體，遠慢於讀進共享記憶體後重用。FlashAttention 的核心洞見正是這一點 (18.5 節)，而它也是「融合核心」這個概念的基礎。

18.1.3 Tensor Core：為矩陣乘法而生的硬體

現代 GPU 除了一般的運算單元，還有專門處理小矩陣乘加的硬體單元。它們的吞吐量遠高於通用單

元——但只有在特定條件下才會被使用。

條件	要求	不滿足的後果	處置
資料型別	BF16、FP16、TF32 等低精度	退回通用單元，慢數倍	啟用混合精度
維度對齊	通常需為 8 或 16 的倍數	效率下降	d_ff 取整 (第 12.3.3 節)
運算形式	必須是矩陣乘法	逐元素運算用不到	無法改變
記憶體排列	連續且對齊	額外的搬移開銷	避免不必要的 transpose

第二列連結到第 12.3.3 節那個「d_ff 向上取整到 128 的倍數」的設計。那不是強迫症，而是為了讓矩陣維度對齊硬體單元的偏好——一個「數學上無所謂、硬體上有差」的細節。

18.2 運算強度與屋頂線

18.2.1 兩種瓶頸

任何一個運算都可以問兩個問題：它要做多少次浮點運算？它要搬移多少位元組？兩者的比值就是運算強度。

$$\text{運算強度} = \text{FLOPs} \div \text{搬移的位元組數 (單位 : FLOPs/byte)}$$

第 14.6.2 節已經用過這個概念來分析 decode 階段。這裡把它一般化。

$$\text{脊點} = \text{硬體峰值算力} \div \text{記憶體頻寬}$$

硬體	峰值算力	記憶體頻寬	脊點
消費級 (示例)	30 TFLOPS	700 GB/s	42.9 FLOPs/byte
資料中心級 (示例)	300 TFLOPS	2,000 GB/s	150 FLOPs/byte

判讀方式很簡單：運算強度低於脊點就是頻寬受限，高於就是計算受限。低於脊點時，再強的算力也用不上——你的瓶頸在資料搬不進來。

18.2.2 常見運算的運算強度

運算	運算強度	判定	意涵
逐元素加法	0.17	嚴重頻寬受限	幾乎純粹在搬資料
RMSNorm	1.0	頻寬受限	同上
softmax	1.25	頻寬受限	同上
矩陣乘法 512×512	170.7	計算受限	能發揮算力
矩陣乘法 2048×2048	682.7	計算受限	更好
矩陣乘法 4096×4096	1365.3	計算受限	最佳

(矩陣乘法以 BF16 計算： $\text{FLOPs} = 2n^3$ 、位元組 $= 3n^2 \times 2$ 。)

這張表傳達兩個重要訊息。第一，所有的逐元素運算都是頻寬受限的——正規化、激勵函數、殘差加法、dropout 全都是。它們的 FLOPs 很少，卻要把整個張量讀出來再寫回去。

第二，矩陣越大越有效率。這與 18.1.1 節的觀察一致，也解釋了為什麼大模型的 MFU 天然高於小模型（第 15.6.2 節）。

18.2.3 為什麼要融合核心

既然逐元素運算是頻寬受限的，那麼把多個逐元素運算「融合」成一個，就能省下中間結果的讀寫。

做法	記憶體流量	說明	節省
分開執行三個運算	讀 3 次、寫 3 次	每個運算各自進出顯示記憶體	基準
融合成一個核心	讀 1 次、寫 1 次	中間結果留在暫存器	約三分之二

一個具體例子：`x = dropout(gelu(linear(x)))`。分開執行需要把 linear 的輸出寫回顯示記憶體、再讀出來做 gelu、再寫回、再讀出來做 dropout。融合之後，linear 的結果直接在暫存器裡經過 gelu 與 dropout，只寫一次。

這就是 torch.compile 這類工具在做的事之一。它會分析你的計算圖，自動把可以融合的運算合併成單一核心。18.6 節會實測它的效果。

18.2.4 把屋頂線用在整個模型上

前面的分析是逐運算的。實務上更有用的是問：整個訓練步驟的平均運算強度是多少？

第 11.7.1 節拆解過單層的 FLOPs 組成，這裡加上記憶體流量。以 $d = 4096$ 、 $T = 2048$ 、 $B = 4$ 為例：

元件	FLOPs 占比	記憶體流量占比	運算強度
矩陣乘法 (投影與 FFN)	約 86%	約 35%	高
注意力核心	約 14%	約 25%	中 (用 flash 時)
正規化與殘差	不到 1%	約 25%	極低
激勵函數與 dropout	不到 1%	約 15%	極低

(比例為量級示意，實際值請用程式 18C 的 profiler 量測。)

這張表揭示了一個關鍵的現象：占計算不到 2% 的逐元素運算，卻占了記憶體流量的四成。這正是 torch.compile 能帶來一至三成加速的原因——它把那四成的流量壓縮掉。

也因此，一個實用的判斷法則是：如果你的模型有很多逐元素運算（多層正規化、複雜的激勵函數、大量的殘差），融合的收益就大；如果幾乎全是大矩陣乘法，收益就小。這可以在做最佳化之前就大致估出來。

18.3 浮點格式

18.3.1 六種格式的完整比較

第 3.6.2 節給過概覽，這裡給精確的數值。

格式	指數位	尾數位	最大值	最小正規值	相對精度
FP32	8	23	3.40×10^{38}	1.18×10^{-38}	6.0×10^{-8}
TF32	8	10	3.40×10^{38}	1.18×10^{-38}	4.9×10^{-4}
FP16	5	10	6.55×10^4	6.10×10^{-5}	4.9×10^{-4}
BF16	8	7	3.39×10^{38}	1.18×10^{-38}	3.9×10^{-3}
FP8 (E4M3)	4	3	2.40×10^2	1.56×10^{-2}	6.2×10^{-2}

格式	指數位	尾數位	最大值	最小正規值	相對精度
FP8 (E5M2)	5	2	5.73×10^4	6.10×10^{-5}	1.2×10^{-1}

18.3.2 為什麼 BF16 勝過 FP16

看表格的第三與第四列：FP16 的相對精度是 BF16 的八倍好，但它的最大值只有 65,504。

這個數字在訓練中很容易被超過。梯度的平方和、注意力分數的中間值、甚至某些 activation，都可能突破六萬。一旦溢位就變成 inf，接著整個訓練崩潰。

BF16 犧牲尾數精度換取與 FP32 相同的指數範圍。而深度學習的經驗是：範圍比精度重要得多。模型對數值的微小誤差有相當的容忍度（梯度下降本來就是隨機的），但對 inf 沒有任何容忍度。

面向	FP16	BF16	判斷
溢位風險	高（上限 65,504）	極低	BF16 勝
下溢風險	有（需 loss scaling）	低	BF16 勝
數值精度	較好	較差	FP16 勝但影響小
實作複雜度	需 GradScaler	不需要	BF16 勝
硬體支援	較舊的 GPU 也有	需較新的硬體	FP16 勝

本書的建議與第 15.3.1 節一致：硬體支援 BF16 就用 BF16，不支援才退回 FP16 加 loss scaling。

18.3.3 FP8 與更低精度

FP8 有兩種變體：E4M3 精度較好但範圍窄（最大 240），E5M2 範圍較大但精度極差。實務上常見的做法是前向用 E4M3、反向用 E5M2——因為梯度的動態範圍通常較大。

但 FP8 的相對精度只有 6% 左右，這意味著它不能無條件使用。目前的做法多半是逐張量或逐區塊地做動態縮放，把數值調整到 FP8 能表示的範圍內。這增加了實作複雜度，也是為什麼 FP8 尚未像 BF16 那樣普及。

對本書讀者的實務建議

FP8 需要較新的硬體與成熟的函式庫支援，且對訓練穩定性的影響仍在累積實證。本書的建議是：訓練用 BF16、推論可考慮更低精度（第 35 章的量化）。把 FP8 當成一個要知道但暫時不必自己實作的技術。

18.4 記憶體去哪裡了

18.4.1 四大類

第 3.5.5 節與第 15.6.4 節分別談過一部分，這裡做完整的拆解。

類別	大小	隨什麼變化	如何減少
模型狀態	參數、梯度、最佳化器狀態	模型大小	分片 (第 19 章) 、LoRA (第 24 章)
activation	反向傳播所需的中間值	批次 $\times$ 序列 $\times$ 層數	checkpointing (18.4.3)
暫時緩衝	運算過程的中間張量	運算方式	融合核心、in-place 運算
框架開銷	CUDA context、記憶體池碎片	大致固定	設定調整

18.4.2 實際數字

用混合精度 (每參數 16 bytes : BF16 權重 2 + BF16 梯度 2 + FP32 主權重 4 + m 4 + v 4) 計算三種規模：

模型	設定	模型狀態	activation	合計
110 M	B=8, T=1024, L=12	1.76 GB	4.53 GB	6.29 GB
1.3 B	B=4, T=2048, L=24	20.80 GB	24.16 GB	44.96 GB
7 B	B=1, T=4096, L=32	112.00 GB	49.39 GB	161.39 GB

(activation 為粗估，含各層中間值與注意力矩陣，BF16、未使用 checkpointing。實際數字請用程式 18A 量測。)

這張表給出三個重要觀察。第一，小模型的 activation 反而占大宗 (110 M 的例子中 activation 是模型狀態的 2.6 倍)。第二，模型越大，模型狀態的占比越高。第三，7 B 模型即使批次只有 1，總需求仍達 161 GB——遠超單卡容量，這就是第 19 章分散式訓練的必要性。

18.4.3 activation checkpointing

第 15.6.4 節介紹過原理，這裡給出精確的取舍。

層數 L	全部保留	每 $\sqrt{L}$ 層保留一個	節省倍數
12	12	4	3.0×
24	24	5	4.8×
32	32	6	5.3×
80	80	9	8.9×

計算成本的增加約為三分之一（第 15.6.4 節推導過：反向原本是前向的兩倍，加上重算一次前向，從 3 變成 4）。

但這個「增加三分之一」的說法有一個重要的但書：如果省下的記憶體讓你能用更大的批次，整體吞吐量可能反而提升。因為更大的批次意味著更高的運算強度（18.2 節），GPU 的利用率更好。

formosa/training/checkpointing.py

```
import torch
from torch.utils.checkpoint import checkpoint

class CheckpointedBlockStack(torch.nn.Module):
    """對 block 堆疊套用 activation checkpointing
    every_n=1 表示每層都重算，=0 表示不使用"""

    def __init__(self, blocks, every_n: int = 1):
        super().__init__()
        self.blocks = blocks
        self.every_n = every_n

    def forward(self, x, cos=None, sin=None):
        for i, blk in enumerate(self.blocks):
            if self.every_n > 0 and (i % self.every_n == 0) and self.training:
                # use_reentrant=False 是較新的實作，與其他功能相容性較好
                x = checkpoint(blk, x, cos, sin, use_reentrant=False)
            else:
                x = blk(x, cos, sin)
        return x

def suggest_checkpoint_interval(n_layers: int) -> int:
    """理論上的最佳間隔約為 sqrt(L)"""
    import math
    return max(1, round(math.sqrt(n_layers)))
```

use_reentrant=False 值得說明：舊版的實作在與某些功能（例如 RNG 狀態管理、部分的自動微分行為）搭配時有已知的問題。新版較穩健，本書一律使用它。

18.4.5 記憶體碎片化

PyTorch 不會每次都向 CUDA 要記憶體——它維護一個記憶體池，用完的張量回收到池中重用。這通常很有效率，但會產生碎片。

碎片化的典型症狀是：明明 `nvidia-smi` 顯示還有幾 GB 可用，卻仍然 OOM。原因是那幾 GB 被切成許多小塊，沒有一塊夠大來容納你要配置的張量。

成因	機制	徵兆	處置
形狀頻繁變動	不同大小的張量交替配置	reserved 遠大於 allocated	固定形狀、 <code>drop_last</code>
大小張量交錯	大張量之間夾雜小張量	同上	調整配置順序
長生命週期的小張量	卡在池中造成分隔	碎片持續累積	及時釋放參照
峰值後未整理	池保留了峰值時的配置	穩定但浪費	<code>empty_cache</code> (謹慎使用)

碎片化診斷

```
import torch

def fragmentation_report():
    if not torch.cuda.is_available():
        return None
    alloc = torch.cuda.memory_allocated() / 1e9
    reserved = torch.cuda.memory_reserved() / 1e9
    frag = (reserved - alloc) / max(reserved, 1e-9)
    verdict = ("正常" if frag < 0.2 else
               "輕微碎片" if frag < 0.4 else "嚴重碎片")
    return {"allocated_GB": round(alloc, 2),
            "reserved_GB": round(reserved, 2),
            "fragmentation": round(frag, 3),
            "verdict": verdict}

# 環境變數：讓配置器使用可擴充的區段，通常能減少碎片
# 執行前設定：PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

`empty_cache` 要謹慎使用

它會把記憶體池還給 CUDA，看起來釋放了記憶體——但下次配置時又要重新向 CUDA 要，而那個過程有可觀的開銷。在訓練迴圈中頻繁呼叫它會嚴重拖慢速度。正確的做法是找出碎片的根因（通常是形狀變動），而不是反覆清理。只有在階段切換時（例如訓練結束、開始評估）才適合呼叫一次。

18.4.4 OOM 排除清單

記憶體不足是最常見的訓練障礙。以下清單依「成本由低到高」排序，遇到 OOM 時依序嘗試。

順序	做法	記憶體節省	代價
1	降低微批次，用梯度累積補回	正比於批次	略慢（第 15.1 節）
2	確認使用融合注意力	大幅（避免 $T \times T$ 矩陣）	無（第 18.5 節）
3	啟用 activation checkpointing	數倍	計算增加約三分之一
4	縮短序列長度	超線性	影響上下文能力
5	確認評估時使用 no_grad	大幅	無（第 4.2.2 節）
6	清除不必要的 tensor 參照	視情況	無（第 4.2.1 節）
7	改用記憶體較省的最佳化器	約三分之一	可能影響收斂
8	減少模型層數或維度	正比	能力下降
9	分片或 offload	大幅	複雜度與速度（第 19 章）

第五與第六項是「零代價」的——如果你的 OOM 是這兩個原因造成的，修好之後什麼都不會犧牲。

所以雖然它們排在中間，實務上應該優先檢查：先確認自己沒有犯低級錯誤，再考慮真正的取捨。

記憶體剖析器

```
import torch

class MemoryProfiler:
    """逐階段記錄 GPU 記憶體，找出峰值發生在哪裡"""

    def __init__(self, device):
        self.device = device
        self.records = []
        self.enabled = device.type == "cuda"

    def reset(self):
        if self.enabled:
            torch.cuda.empty_cache()
            torch.cuda.reset_peak_memory_stats()
            self.records = []

    def mark(self, label: str):
        if not self.enabled:
            return
        torch.cuda.synchronize()
```

```

self.records.append({
    "label": label,
    "allocated_GB": torch.cuda.memory_allocated() / 1e9,
    "reserved_GB": torch.cuda.memory_reserved() / 1e9,
    "peak_GB": torch.cuda.max_memory_allocated() / 1e9,
})

def report(self):
    if not self.records:
        print("（非 CUDA 裝置，無記憶體資料）"); return
    print(f"{'階段':<20}{'已配置':>10}{'保留':>10}{'峰值':>10}")
    prev = 0.0
    for r in self.records:
        delta = r["allocated_GB"] - prev
        mark = " ←最大增量" if delta > 0.5 else ""
        print(f"{r['label']:<20}{r['allocated_GB']:>9.2f}G"
              f"{r['reserved_GB']:>9.2f}G{r['peak_GB']:>9.2f}G{mark}")
        prev = r["allocated_GB"]

# 使用方式：在訓練步驟的各階段插入 mark
# prof.reset(); prof.mark("初始")
# out = model(x);          prof.mark("前向後")
# loss.backward();        prof.mark("反向後")
# optimizer.step();       prof.mark("更新後")
# prof.report()

```

allocated 與 reserved 的差別值得說明：前者是實際被張量使用的，後者是 PyTorch 向 CUDA 要來的記憶體池。兩者差距大代表有碎片化——這時候 `empty_cache()` 可能有幫助，但更好的做法是找出造成碎片的原因（通常是頻繁配置不同大小的張量）。

18.5 FlashAttention 與融合核心

18.5.1 問題重述

第 11.7.2 節算過： $B = 8$ 、 $H = 32$ 、 $T = 16,384$ 時，單層的注意力分數矩陣就要 137 GB。標準實作要把這個矩陣完整寫進顯示記憶體，然後讀出來做 softmax，再讀出來與 V 相乘。

用 18.2 節的角度看，這個過程的問題不只是記憶體不夠，還在於它的運算強度極低——大量的資料搬移，卻只做了相對少的計算。

18.5.2 核心想法：分塊與線上 softmax

FlashAttention 的洞見是：我們要的是最終輸出，不是那個中間矩陣。如果能一塊一塊地算，算完就丟掉中間結果，就不必儲存整個矩陣。

困難在於 softmax 需要整列的總和才能正規化。解法是線上 softmax：一邊掃描一邊維護「目前的最大值」與「目前的加權和」，遇到更大的值就把先前的結果重新縮放。

若已處理到最大值 m_1 、和 s_1 ，遇到新塊的最大值 m_2 ：新的最大值 $m = \max(m_1, m_2)$ ，先前的結果乘上 $\exp(m_1 - m)$ 後再累加

這與第 3.3.5 節的 log-sum-exp 是同一個數學基礎。差別在於這裡是「增量式」地做，而不是一次看完全部。

面向	標準實作	FlashAttention	結果
T×T 矩陣	完整寫入顯示記憶體	分塊在共享記憶體中處理	記憶體 $O(T^2) \rightarrow O(T)$
記憶體流量	多次進出顯示記憶體	讀一次、在晶片內重用	大幅減少
FLOPs	基準	略增（反向需重算）	用計算換頻寬
數值結果	基準	數學上等價	差異僅浮點誤差
最大可行 T	受記憶體限制	大幅延長	長上下文成為可能

第三列是這個技術最違反直覺的地方：它做了更多次浮點運算，卻跑得更快。因為現代 GPU 的瓶頸是頻寬而非算力（18.2 節），減少資料搬移的收益遠大於多算一些的成本。

18.5.3 怎麼確認它真的被啟用

呼叫 `F.scaled_dot_product_attention` 不保證會使用 FlashAttention——PyTorch 會依據輸入的形狀、型別與硬體選擇後端。以下是確認的方法。

ch18/check_attention_backend.py

```
import torch
import torch.nn.functional as F
from torch.nn.attention import sdpa_kernel, SDPBackend

def which_backend(q, k, v, is_causal=True):
    """逐一嘗試各後端，回報哪些可用"""
    results = {}
    for name, backend in [("flash", SDPBackend.FLASH_ATTENTION),
                          ("mem_efficient", SDPBackend.EFFICIENT_ATTENTION),
                          ("math", SDPBackend.MATH)]:
        try:
            with sdpa_kernel(backend):
                F.scaled_dot_product_attention(q, k, v, is_causal=is_causal)
            results[name] = "可用"
        except RuntimeError as e:
            results[name] = f"不可用：{str(e)[:60]}"
    return results
```

```
def diagnose(B=2, H=8, T=1024, dh=64, dtype=torch.bfloat16,
            device="cuda"):
    q = torch.randn(B, H, T, dh, dtype=dtype, device=device)
    k = torch.randn_like(q); v = torch.randn_like(q)
    print(f"設定：B={B} H={H} T={T} d_head={dh} dtype={dtype}")
    for name, status in which_backend(q, k, v).items():
        print(f" {name:16s} {status}")
```

常見的「無法使用 flash」原因：

- # 1. dtype 為 float32 (flash 需要 fp16/bf16)
- # 2. d_head 不被支援 (通常需 ≤ 128 且為特定值)
- # 3. 使用了自訂的 attn_mask (而非 is_causal)
- # 4. 硬體不支援

第三個原因特別值得注意：如果你傳入自訂的 attn_mask 而非使用 is_causal=True，很可能會退回較慢的後端。所以在只需要因果遮罩的情況下，一定要用 is_causal 參數而非自己建一個遮罩張量——這是一個「寫法不同、效能差好幾倍」的例子。

18.5.4 其他可融合的部分

可融合的組合	節省什麼	如何取得	章節
注意力全流程	T×T 矩陣的讀寫	F.scaled_dot_product_attention	18.5.3
正規化 + 殘差	一次張量讀寫	torch.compile 或專用核心	18.6
SwiGLU 的三個矩陣	中間結果的讀寫	同上	第 12.3 節
交叉熵 + softmax	大型 logits 的中間值	F.cross_entropy (已融合)	第 3.4.4 節
最佳化器更新	多次參數讀寫	fused/foreach 版本的 AdamW	18.6.3

最後一列是一個容易被忽略的最佳化。標準的 AdamW 對每個參數張量逐一更新，而模型可能有數百個張量——每一個都是一次核心啟動。使用 fused 或 foreach 版本可以把它們批次處理，在參數多的模型上有可觀的效益。

18.5.5 為什麼記憶體節省能換來更長的上下文

FlashAttention 最重要的實務影響，不是「同樣的設定跑得快一點」，而是「原本做不到的事變成可

能」。

序列長度	標準實作的分數矩陣	FlashAttention	差別
2,048	0.54 GB	幾乎為零	兩者皆可行
8,192	8.59 GB	幾乎為零	標準版已吃緊
32,768	137.44 GB	幾乎為零	標準版完全不可行

($B = 8$ 、 $H = 32$ 、BF16、單層。第 11.7.2 節算過同樣的數字。)

「幾乎為零」的意思是：FlashAttention 的額外記憶體只與 T 成正比（用來存輸出與統計量），而非 T^2 。所以在長上下文的情境下，注意力不再是記憶體瓶頸——瓶頸回到 KV cache（第 14.5.4 節）與 activation。

這對臺灣的應用有直接意義。法規彙編、契約全文、長篇技術文件都是數萬字的量級，沒有長上下文能力就必須切塊處理（第 30 章的 RAG），而切塊必然損失跨段的關聯。FlashAttention 讓「整份讀進去」變得可行——雖然還有第 20 章要處理的位置編碼外推問題。

18.5.6 融合核心的一般原則

從 FlashAttention 可以抽出一個通用的最佳化思路，適用於任何頻寬受限的運算。

1. 找出被寫進顯示記憶體、然後又立刻被讀出來的中間結果。
2. 判斷它能否留在晶片內（共享記憶體或暫存器）。
3. 若中間結果太大放不下，考慮分塊處理，並找出「可增量計算」的形式。
4. 接受多做一些計算來換取少搬一些資料——因為前者便宜。
5. 驗證數值結果與原版等價（在浮點誤差內）。

第三步的「可增量計算」是關鍵，也是最難的部分。softmax 之所以能分塊，是因為 log-sum-exp 有增量形式（18.5.2 節）；不是所有運算都有這個性質。這也是為什麼融合核心通常由函式庫提供，而不是每個人自己寫。

但理解這個原則仍然有價值：當你看到某個運算特別慢時，你會知道該問「它有沒有不必要的中間結果進出記憶體」，而那往往就是答案。

18.6 torch.compile 與圖最佳化

18.6.1 它做什麼

階段	做什麼	效益	代價
圖捕捉	把 Python 程式碼轉成計算圖	—	需要程式可被追蹤
圖最佳化	消除冗餘、重排運算	減少不必要的計算	編譯時間
核心融合	把相鄰的逐元素運算合併	大幅減少記憶體流量	同上
程式碼生成	為特定形狀產生最佳化核心	接近手寫核心的效能	形狀改變需重編譯

最後一列是使用 `torch.compile` 時最需要注意的：每一種新的輸入形狀都會觸發一次重新編譯。如果你的批次大小或序列長度經常變動，編譯開銷可能吃掉所有效益。

這解釋了第 13.3.3 節那個 `drop_last=True` 的建議——保持形狀一致不只是為了程式碼整潔，也是為了避免重複編譯。

18.6.2 實務注意事項

問題	症狀	原因	處置
首次執行極慢	第一步花數十秒	編譯開銷	正常，暖機時排除
反覆重新編譯	每隔幾步就變慢	輸入形狀變動	<code>drop_last</code> 、固定序列長度
圖中斷	效益遠低於預期	程式碼有無法追蹤的部分	用 <code>fullgraph=True</code> 診斷
數值差異	結果與未編譯版本不同	運算重排造成的浮點差異	通常可接受，需驗證
與其他功能衝突	報錯或行為異常	與 <code>checkpointing</code> 等的相容性	逐一測試

安全地使用 `torch.compile`

```
import torch, time

def compile_with_fallback(model, mode="default", verbose=True):
    """嘗試編譯，失敗時退回原模型而非讓訓練中斷"""
    if not hasattr(torch, "compile"):
        return model, "torch.compile 不可用"
    try:
        compiled = torch.compile(model, mode=mode)
        return compiled, f"已編譯 (mode={mode})"
    except:
```

```

except Exception as e:
    if verbose:
        print(f"[警告] 編譯失敗，使用未編譯版本：{e}")
    return model, f"編譯失敗：{type(e).__name__}"

def verify_compile_equivalence(model, compiled, sample_input,
                               atol=1e-3):
    """編譯前後的數值必須接近（不會完全相同，因運算可能被重排）"""
    model.eval(); compiled.eval()
    with torch.no_grad():
        a = model(sample_input)
        b = compiled(sample_input)
    diff = (a - b).abs().max().item()
    ok = diff < atol
    print(f"編譯前後最大差異 {diff:.2e} {'通過' if ok else '需檢查'}")
    return ok

def measure_compile_benefit(model, compiled, sample, steps=30,
                             warmup=10):
    """量測實際效益，暖機必須足夠涵蓋編譯時間"""
    def bench(m):
        for _ in range(warmup):
            m(sample)
        if sample.is_cuda:
            torch.cuda.synchronize()
        t0 = time.perf_counter()
        for _ in range(steps):
            m(sample)
        if sample.is_cuda:
            torch.cuda.synchronize()
        return (time.perf_counter() - t0) / steps

    base = bench(model)
    comp = bench(compiled)
    return {"baseline_ms": round(base * 1000, 3),
            "compiled_ms": round(comp * 1000, 3),
            "speedup": round(base / comp, 3)}

```

`compile_with_fallback` 的設計理由：編譯可能因為各種原因失敗（不支援的運算、版本問題、記憶體不足），而那不應該讓你的訓練整個中斷。退回未編譯版本雖然慢，但至少能跑完。

18.6.3 最佳化器的融合版本

啟用 fused AdamW

```

import torch, inspect

def build_optimizer(model, lr, weight_decay, device):
    """自動選用可用的最快實作"""
    from formosa.utils import build_param_groups # 第 10.2.4 節
    groups = build_param_groups(model, weight_decay)

    # fused 版本把多個張量的更新合併成少數核心
    sig = inspect.signature(torch.optim.AdamW)

```

```
kwargs = {"lr": lr, "betas": (0.9, 0.95), "eps": 1e-8}
if "fused" in sig.parameters and device.type == "cuda":
    kwargs["fused"] = True
    note = "fused"
elif "foreach" in sig.parameters:
    kwargs["foreach"] = True
    note = "foreach"
else:
    note = "標準"
print(f"[最佳化器] 使用 {note} AdamW")
return torch.optim.AdamW(groups, **kwargs)
```

這是一個「幾乎零成本」的最佳化：加一個參數就可能省下數個百分點的訓練時間，而且不改變任何數值行為（除了浮點誤差）。建議在所有訓練腳本中都加上這個自動偵測。

18.7 用 profiler 定位瓶頸

18.7.1 診斷流程

第 15.6.3 節列過常見瓶頸，這裡給出系統性的定位方法。

步驟	做什麼	判斷依據	排除什麼
1	用假資料取代真實 DataLoader	速度是否大幅提升	資料載入瓶頸
2	看 GPU 使用率的時間分布	是否有明顯的空檔	同步點、CPU 開銷
3	算 MFU 並與 15.6.2 節對照	是否在合理範圍	整體效率
4	用 profiler 看各核心的耗時	哪些核心占最多時間	找出熱點
5	檢查注意力後端	flash 是否可用	未使用融合核心
6	看記憶體流量與運算強度	是否頻寬受限	確認瓶頸類型

第一步是最便宜也最有效的：把 DataLoader 換成一個直接回傳固定張量的假迭代器，跑一次看速度。如果速度大幅提升，那你的瓶頸就在資料管線，而不在模型——那是一個完全不同的問題，且通常更好修。

ch18/profile_training.py

```
import torch
from torch.profiler import profile, ProfilerActivity, schedule

def fake_loader(batch_size, seq_len, vocab_size, device, n=1000):
    """假資料迭代器：用於排除資料載入瓶頸（診斷第 1 步）"""
```

```

x = torch.randint(0, vocab_size, (batch_size, seq_len), device=device)
y = torch.randint(0, vocab_size, (batch_size, seq_len), device=device)
for _ in range(n):
    yield x, y

def profile_steps(model, optimizer, loader, device, out_dir,
                  wait=2, warmup=3, active=5):
    """對數個訓練步驟做完整剖析"""
    activities = [ProfilerActivity.CPU]
    if device.type == "cuda":
        activities.append(ProfilerActivity.CUDA)

    sched = schedule(wait=wait, warmup=warmup, active=active, repeat=1)
    with profile(activities=activities, schedule=sched,
                record_shapes=True, profile_memory=True,
                with_stack=False) as prof:
        for i, (x, y) in enumerate(loader):
            if i >= wait + warmup + active:
                break
            _, loss = model(x, targets=y)
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            optimizer.step()
            prof.step()

    key = "cuda_time_total" if device.type == "cuda" else "cpu_time_total"
    print(prof.key_averages().table(sort_by=key, row_limit=20))
    prof.export_chrome_trace(str(out_dir / "trace.json"))
    return prof

def summarize_hotspots(prof, device, top_k=10):
    """把剖析結果整理成可讀的熱點清單"""
    key = "cuda_time_total" if device.type == "cuda" else "cpu_time_total"
    events = sorted(prof.key_averages(),
                    key=lambda e: getattr(e, key), reverse=True)
    total = sum(getattr(e, key) for e in events)
    rows = []
    for e in events[:top_k]:
        t = getattr(e, key)
        rows.append({"name": e.key[:50],
                    "time_ms": round(t / 1000, 2),
                    "share": round(t / total, 4),
                    "calls": e.count})
    return rows

```

18.7.2 常見的熱點與意義

熱點	正常嗎	若占比過高代表	處置
矩陣乘法核心	正常・應占大宗	—	這是你要的
逐元素運算	應占少數	未融合	啟用 torch.compile
記憶體複製	應極少	不必要的搬移或型別轉換	檢查 dtype 一致性

熱點	正常嗎	若占比過高代表	處置
注意力的 math 後端	不正常	未使用 flash	見 18.5.3
同步等待	應極少	過多的 .item() 或 print	批次記錄
最佳化器	應占少數	未使用 fused	見 18.6.3

第四列是一個很常見的問題。如果 profiler 顯示注意力用的是 math 後端（也就是原生的矩陣乘法實作），那你就沒有享受到 FlashAttention——而在長序列上這個差距可能是數倍。

18.7.3 從診斷到行動

把診斷結果對應到具體的最佳化手段。

診斷結果	主要手段	預期改善	副作用
資料載入瓶頸	預先編碼、增加 workers	可能數倍	無
逐元素運算占比高	torch.compile	一至三成	編譯時間
未使用 flash	修正呼叫方式	長序列上數倍	無
批次太小	加大批次或梯度累積	視情況	記憶體
記憶體不足	checkpointing 換更大批次	可能淨提升	計算增加
同步點過多	批次記錄指標	數個百分點	紀錄粒度變粗
最佳化器慢	fused AdamW	數個百分點	無

這張表的排序大致就是「投資報酬率」的順序。前三項通常是零成本或低成本的，應該優先處理；後面幾項涉及取捨，需要量測後再決定。

程式實作

本章三個實作的產出一份效能剖析報告與一套可重複使用的診斷工具。它們會被第 19 章的分散式訓練與第 35 章的推論部署直接沿用。

程式 18A 記憶體剖析器

目標：逐項拆解訓練的記憶體占用，並與 18.4.2 節的估算對照。

項目	要量測什麼	驗收標準	節次
模型狀態	權重、梯度、最佳化器狀態	與公式估算差異 < 10%	18.4.2
activation	前向後與反向前的差值	隨批次與序列長度的成長符合預期	18.4.2
峰值位置	記憶體峰值出現在哪一階段	通常在反向傳播開始時	18.4.1
checkpointing	啟用前後的差異	節省倍數接近 18.4.3 節	18.4.3
碎片化	reserved 與 allocated 的差距	差距小於 20%	18.4.4

延伸挑戰：掃描批次大小從 1 到 OOM 為止，畫出「批次對峰值記憶體」的曲線。它應該接近線性——如果不是，找出偏離的原因（提示：與注意力矩陣的平方成長有關）。

程式 18B 效能矩陣測試

目標：在你的裝置上系統性地量測不同精度與核心的效能，建立一份可查詢的參考表。

ch18/lab18b_perf_matrix.py (節錄)

```
import json, time, torch
from pathlib import Path

def bench_matmul(n, dtype, device, iters=20, warmup=5):
    a = torch.randn(n, n, dtype=dtype, device=device)
    b = torch.randn(n, n, dtype=dtype, device=device)
    for _ in range(warmup):
        _ = a @ b
    if device.type == "cuda":
        torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(iters):
        c = a @ b
    if device.type == "cuda":
        torch.cuda.synchronize()
    dt = (time.perf_counter() - t0) / iters

    flops = 2 * n ** 3
    bytes_moved = 3 * n * n * a.element_size()
```

```

return {
    "n": n, "dtype": str(dtype).replace("torch.", ""),
    "seconds": round(dt, 6),
    "TFLOPS": round(flops / dt / 1e12, 2),
    "arithmetic_intensity": round(flops / bytes_moved, 1),
}

def bench_elementwise(n, dtype, device, iters=50):
    """對照組：頻寬受限的運算"""
    a = torch.randn(n, n, dtype=dtype, device=device)
    b = torch.randn_like(a)
    for _ in range(5):
        _ = a + b
    if device.type == "cuda":
        torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(iters):
        c = a + b
    if device.type == "cuda":
        torch.cuda.synchronize()
    dt = (time.perf_counter() - t0) / iters

    bytes_moved = 3 * n * n * a.element_size()
    return {"n": n, "dtype": str(dtype).replace("torch.", ""),
            "GB_per_sec": round(bytes_moved / dt / 1e9, 1),
            "arithmetic_intensity": round(n * n / bytes_moved, 3)}

```

必須產出的四項：

- 矩陣乘法在四種尺寸 × 三種精度下的 TFLOPS 表，並與規格書上的峰值對照算出達成率。
- 逐元素運算的有效頻寬，並與規格書上的記憶體頻寬對照。這個數字就是你的實際頻寬上限。
- 用前兩項算出你自己裝置的脊點，並與 18.2.1 節的示例對照。
- 注意力後端診斷：在你的硬體與設定下，flash 是否可用？如果不可用，是哪一個條件不滿足？

一個重要的量測紀律

本章的所有量測都必須遵守第 2 章與第 4 章建立的三條規則：暖機並捨棄前幾次、在 CUDA 上呼叫 `synchronize`、記錄裝置與版本。特別是第二條——不加 `synchronize` 量到的是「發出指令的時間」而非「算完的時間」，在 GPU 上會得到荒謬的樂觀結果。

程式 18C 瓶頸定位與最佳化實戰

目標：對第 15 章的訓練腳本做完整剖析，套用至少三種最佳化，並量測前後的 MFU。

必須完成的六個步驟：

1. 建立基準：記錄未最佳化版本的 MFU、tokens/sec、峰值記憶體。
2. 執行 18.7.1 節的六步診斷，判定主要瓶頸類型。
3. 依 18.7.3 節選擇至少三種手段，逐一套用並各自量測（不可一次全開，否則無法歸因）。

4. 每一項都記錄：改善幅度、副作用、以及是否有數值差異。
5. 找出對你的設定效益最大的一項，並解釋為什麼是它。
6. 產出前後對照表與一份最佳化決策紀錄。

第三步的「逐一套用」是第 16.1.2 節控制變數原則的直接應用。如果你一次開啟三個最佳化，看到 MFU 從 0.18 升到 0.34，你不會知道哪一項真正有效——而下次換一個模型時，你也不知道該優先做什麼。

最佳化項目	量測什麼	預期	若不如預期
torch.compile	MFU、編譯時間、數值差異	一至三成	檢查是否有圖中斷
fused AdamW	MFU、最佳化器耗時占比	數個百分點	確認參數是否生效
flash attention	長序列的時間與記憶體	長序列上數倍	檢查後端診斷
checkpointing + 大批次	整體吞吐量	可能淨提升	批次沒加大就沒意義
減少同步點	MFU	數個百分點	檢查是否仍有 .item()

系統案例：把訓練腳本的 MFU 提升一倍

本章的系統案例是一個明確的工程目標：拿第 15 章的訓練腳本，透過本章的方法把 MFU 提升至少一倍，並完整記錄過程與決策。

為什麼設定「提升一倍」這個目標

這個目標聽起來很高，但在多數學生的初始腳本上其實不難達成——因為初始版本通常有幾個明顯的問題：沒有啟用融合注意力、沒有用 `compile`、每步都呼叫 `.item()`、批次設得太小。修掉這些，MFU 從 0.15 升到 0.30 是常見的結果。

而如果你的初始 MFU 已經是 0.35，那麼提升一倍就不現實——這時候正確的做法是誠實報告「已接近合理上限」，並用 15.6.2 節的表格佐證。這也是本案例的一部分：知道什麼時候該停止最佳化。

必須交付的六項

1. 基準報告：初始 MFU、tokens/sec、峰值記憶體、以及 profiler 的熱點清單。

2. 診斷結論：主要瓶頸是什麼？依據是什麼？
3. 逐項最佳化紀錄：每一項的改善幅度、副作用、數值驗證。
4. 最終對照表：初始 vs 最終的所有指標。
5. 決策紀錄：哪些手段你試了但沒採用？為什麼？
6. 停止判準：你認為還有多少改善空間？依據是什麼？

評分重點

- 是否逐項量測而非一次全開。無法歸因的改善不予計分。
- 數值驗證：最佳化後的模型輸出是否與原版一致（在浮點誤差內）。
- 是否誠實報告失敗的嘗試。只列出有效的手段者扣分。
- 停止判準的品質。「感覺差不多了」不算，要有 MFU 與合理範圍的對照。
- 若初始 MFU 已高而無法提升一倍，是否正確地論證了這一點。

第五項是這個案例最重要的訓練。工程上有一個常見的陷阱：最佳化可以無止境地做下去，但邊際效益會急速遞減。能判斷「這裡已經夠好了，該去處理別的問題」，是一項比技術本身更難的能力。

章末評量

一、概念題

1. 為什麼說「GPU 靠切換執行緒隱藏延遲」？這對「該給它多大的工作量」有什麼意涵？
2. 什麼是運算強度與脊點？如何用它們判斷一個運算是計算受限還是頻寬受限？
3. 為什麼所有的逐元素運算都是頻寬受限的？這對最佳化有什麼指引？
4. 為什麼 BF16 取代了 FP16 成為訓練的主流？請從範圍與精度兩方面說明。
5. 訓練的記憶體由哪四大類組成？為什麼小模型的 activation 反而可能占大宗？
6. activation checkpointing 的原理是什麼？為什麼「計算增加三分之一」有時反而讓整體變快？
7. FlashAttention 為什麼「做更多次浮點運算卻更快」？這反映了現代 GPU 的什麼特性？
8. 使用 torch.compile 時，為什麼輸入形狀變動會是個問題？
9. 診斷效能瓶頸的第一步為什麼是「用假資料取代 DataLoader」？

二、計算題

1. 一張 GPU 峰值 40 TFLOPS、記憶體頻寬 900 GB/s。計算它的脊點。一個運算強度為 30 的核心在這張卡上是什麼受限？
2. 計算 BF16 下 3072×3072 矩陣乘法的運算強度。它高於上題的脊點嗎？
3. 一個 350 M 參數的模型，混合精度訓練。計算模型狀態的記憶體。若 $B=8$ 、 $T=2048$ 、 $L=24$ 、 $d=1024$ 、 $\text{heads}=16$ ，用 18.4.2 節的粗估公式算 activation。
4. FP16 的最大值是多少？如果訓練中某個 activation 達到 10^5 ，會發生什麼？BF16 呢？
5. 一個 48 層的模型使用每 $\sqrt{L}$ 層保留一次的 checkpointing。保留幾個檢查點？記憶體節省約幾倍？
6. 未最佳化時 MFU 為 0.16、tokens/sec 為 12,000。最佳化後 MFU 為 0.33。新的 tokens/sec 約是多少？（假設模型與硬體不變）

三、除錯題

1. profiler 顯示注意力使用 math 後端而非 flash。請列出四個可能的原因與檢查順序。
2. 啟用 torch.compile 後，前幾步很慢，之後正常，但每隔約 50 步又會慢一次。請診斷。
3. MFU 只有 0.08，但 GPU 使用率顯示 90%。請說明這兩者為何不矛盾，以及可能的原因。
4. 記憶體剖析顯示 reserved 是 allocated 的兩倍。請說明這代表什麼與可能的處置。
5. 啟用 activation checkpointing 後，記憶體確實下降但訓練時間增加了 80%，遠高於預期的三成。請列出三個可能原因。

6. 同學把 dtype 從 FP32 改成 BF16，但速度幾乎沒變。請列出三個可能原因。

四、系統設計題

1. 你的實驗室要為學生寫一份「訓練效能自檢腳本」，能在五分鐘內跑完並產出診斷報告。請設計它要檢查哪些項目、各自的判準、以及報告該長什麼樣。
2. 你有一張 24 GB 的 GPU，要訓練一個 1.3 B 的模型。用 18.4.2 節的估算，這個配置放不下。請提出至少四種讓它可行的方案，各自計算記憶體節省與代價，並排出你的優先順序。

五、臺灣情境與倫理題

1. 本章的最佳化手段大多是「零成本」的（改個參數、修正呼叫方式）。請討論：為什麼許多實務專案仍然跑在很低的 MFU 上？這是技術問題、教育問題、還是誘因問題？
2. MFU 提升一倍等同於電費減半、碳排減半。請以第 17 章的換算工具估算：如果臺灣所有的學術訓練任務平均 MFU 從 0.2 提升到 0.35，一年可以省下多少電力？這個數字對推動效能最佳化的教育有什麼意義？

評量提示（給自學者）

- 計算題第 1 題：脊點 = $40e12 \div 900e9 \approx 44.4$ FLOPs/byte。運算強度 30 低於脊點，屬頻寬受限。
- 計算題第 2 題：FLOPs = 2×3072^3 、位元組 = $3 \times 3072^2 \times 2$ ；運算強度 = $2 \times 3072 \div 6 = 1024$ ，遠高於脊點，計算受限。
- 計算題第 3 題：模型狀態 = $350e6 \times 16 = 5.6$ GB；activation $\approx (14 \times 8 \times 2048 \times 1024 \times 2 + 8 \times 16 \times 2048^2 \times 2) \times 24 \approx 37.0$ GB。
- 計算題第 4 題：FP16 最大值 $65,504 \cdot 10^5$ 會溢位成 inf；BF16 最大值約 3.39×10^{38} ，完全沒問題。
- 計算題第 5 題： $\sqrt{48} \approx 6.93$ ，取 7 個檢查點；節省約 $48/7 \approx 6.9$ 倍。
- 計算題第 6 題：tokens/sec 與 MFU 成正比， $12,000 \times (0.33/0.16) \approx 24,750$ 。
- 除錯題第 6 題：可能是模型太小未受益於 Tensor Core、autocast 未正確包住前向、或瓶頸本來就在資料載入而非計算。

研究延伸與版本注意事項

- FlashAttention 系列論文：核心貢獻是把注意力重新表述為一個 IO 感知的演算法。第二版與第三版在平行化與硬體適配有進一步改進，值得追蹤。

- 屋頂線模型 (Roofline Model) 的原始提出：這是一個通用的效能分析框架，不限於 GPU 或深度學習，值得完整理解。
- GPU 架構白皮書：各代硬體的 Tensor Core 支援、記憶體頻寬與功耗規格，是做 18.2 節計算的資料來源。
- FP8 訓練的實證研究：這是一個快速發展的方向，本章的保守建議可能在一兩年內需要更新。
- PyTorch 的效能調校文件與 `torch.compile` 說明：本章 18.6 節的實作以它為準，API 與行為在版本間變動較快。
- 本章的物理原理（頻寬與算力的關係）不會過時，但具體的硬體規格、後端選擇邏輯與 API 都會變。所有效能數字都必須在你自己的裝置上重測。

常見問題

問：我只有 CPU，這一章還有用嗎？答：概念上完全適用——運算強度、記憶體階層、融合的價值在 CPU 上同樣成立，只是脊點的位置不同。實作上，程式 18B 的量測可以在 CPU 上跑（會得到低兩個數量級的數字），而記憶體剖析需要改用系統記憶體的量測工具。第 18.2 節的觀念是通用的，值得完整讀。

問：MFU 要到多少才算好？答：第 15.6.2 節給過範圍：大模型長序列 0.4 到 0.55、中型 0.25 到 0.4、小模型 0.1 到 0.25。重點是與「你自己模型規模的合理範圍」對照，而不是追求絕對數字。小模型的 MFU 天然較低，那不是你的錯。

問：`torch.compile` 一定要用嗎？答：不一定。它在逐元素運算占比高的模型上效益明顯，但也帶來編譯時間與相容性風險。本書的建議是：先用 `profiler` 確認你的瓶頸確實是未融合的逐元素運算，再決定要不要用。盲目啟用可能得到零效益卻多了一堆問題。

問：最佳化該做到什麼程度？答：本章系統案例的第六項要求你回答這個問題。實務的判準是：當你的 MFU 已在合理範圍內，且剩餘的最佳化需要顯著的複雜度時，就該停止。把時間花在資料品質（第 9 章）或評測（第 36 章）上，通常比再擠出 5% 的 MFU 更有價值。

教師使用建議

本章排在第 18 週，是第二學期的第一章，建議 2 小時講授加 1 小時實驗。本章的內容較硬，但每一個概念都能用實測驗證，教學上應該大量使用現場量測。

時段	內容	互動設計	注意事項
前 25 分鐘	18.1 至 18.2 節，執行模型與屋頂線	現場量測矩陣乘法與逐元素運算	運算強度的對比極有說服力
中間 25 分鐘	18.3 至 18.4 節，浮點格式與記憶體	現場示範 FP16 溢位	65504 這個數字要讓學生記住
後 30 分鐘	18.5 至 18.6 節，FlashAttention 與 compile	現場切換後端比較長序列	數倍的差距很震撼
最後 30 分鐘	18.7 節與作業說明	一起看一份真實的 profiler 輸出	找熱點比講原則有效
實驗課 1 小時	程式 18B 與 18C	兩人一組，比較不同硬體的量測結果	硬體差異本身就是教材

一個效果極佳的示範：在課堂上準備一個 $T = 8192$ 的注意力運算，分別用 math 後端與 flash 後端各跑一次，把時間與峰值記憶體並排顯示。學生會看到記憶體從數十 GB 降到不到 1 GB、時間縮短數倍——而程式碼幾乎沒變。這比任何說明都能讓他們理解「同樣的數學、不同的實作」有多大差距。

另一個建議：讓學生在不同的硬體上跑程式 18B（有些人用筆電 CPU、有些用實驗室 GPU），然後把全班的結果彙整成一張表。他們會看到脊點、達成率、頻寬在不同裝置上的巨大差異——而那正是第 21 章要談的臺灣算力生態的具體體現。

高中授課的調整：18.1 用「GPU 是一群只會做簡單事但人數極多的工人」的比喻；18.2 完整教（運算強度的概念高中生完全可懂，而且很有啟發性）；18.3 只講「FP16 會爆掉、BF16 不會」與 65504 這個數字；18.4 講記憶體去哪裡了，checkpointing 用「不記筆記、需要時重推一次」的比喻；18.5 講「不要把中間結果寫下來」的直覺；18.6、18.7 略過。程式做 18B 的簡化版，量測矩陣乘法與加法的差異。

附：效能自檢清單

把本章的診斷方法濃縮成一份可以在五分鐘內跑完的自檢清單。建議在每次開始長時間訓練之前執行一次。

項次	檢查什麼	通過標準	節次
1	MFU 是否在合理範圍	對照第 15.6.2 節的表格	第 15.6 節

項次	檢查什麼	通過標準	節次
2	換成假資料後速度是否大幅提升	提升 < 20% 才算通過	18.7.1
3	注意力是否使用 flash 後端	which_backend 回報可用	18.5.3
4	是否使用 is_causal 而非自訂遮罩	程式碼檢查	18.5.3
5	是否啟用混合精度	autocast 包住前向	第 15.3 節
6	最佳化器是否為 fused 或 foreach	建立時的輸出訊息	18.6.3
7	訓練迴圈中是否有多餘的 .item()	每步不超過一次	第 15.6.3 節
8	批次形狀是否固定	drop_last=True	第 13.3.3 節
9	評估是否使用 no_grad 或 inference_mode	程式碼檢查	第 4.2.2 節
10	記憶體碎片是否過高	reserved – allocated < 20%	18.4.5
11	峰值記憶體是否留有餘裕	至少 10%	第 15.7 節
12	是否記錄了裝置與版本	環境指紋完整	第 4.7.3 節

這十二項中，第 2 到第 9 項都是「零代價的修正」——如果沒通過，修好之後不會犧牲任何東西。第 1、10、11 項是狀態檢查，第 12 項是為了讓你的量測結果日後仍可解讀。

一個實務建議：把這份清單寫成一支腳本，接在第 15 章的訓練啟動流程之前。花五分鐘跑完，可能省下數小時的低效訓練——而且它會在你換硬體、換函式庫版本時自動抓出退化。

本章小結

1. GPU 靠大量執行緒隱藏記憶體延遲，因此需要足夠的平行工作量才能發揮效能；小矩陣填不滿 GPU。
2. 記憶體階層中，重複讀取顯示記憶體遠慢於讀進共享記憶體後重用——這是融合核心的基礎。
3. Tensor Core 只在低精度、維度對齊、且運算為矩陣乘法時才會被使用。
4. 運算強度等於 FLOPs 除以搬移位元組數；脊點等於峰值算力除以記憶體頻寬。低於脊點即頻寬受限。
5. 所有逐元素運算（正規化、激勵函數、殘差、dropout）的運算強度都在 1 附近，嚴重頻寬受限。

6. 矩陣乘法的運算強度隨尺寸線性成長，這解釋了為什麼大模型的 MFU 天然較高。
7. BF16 的最大值與 FP32 同級，FP16 只有 65,504——範圍比精度重要，這是 BF16 勝出的關鍵。
8. 訓練記憶體分四類：模型狀態、activation、暫時緩衝、框架開銷。小模型的 activation 可能占大宗。
9. activation checkpointing 用約三分之一的額外計算換取數倍的記憶體；若能因此加大批次，整體可能反而更快。
10. OOM 排除清單中，「評估時忘記 no_grad」與「把帶梯度的 tensor 存進 list」是零代價的修正，應優先檢查。
11. FlashAttention 用分塊與線上 softmax 把記憶體從 $O(T^2)$ 降到 $O(T)$ ；它做更多計算卻更快，因為瓶頸在頻寬。
12. 使用 is_causal=True 而非自訂遮罩張量，否則可能退回較慢的後端。
13. torch.compile 靠圖最佳化與核心融合加速，但每種新的輸入形狀都會觸發重新編譯。
14. fused 或 foreach 版本的 AdamW 是幾乎零成本的最佳化，應在所有訓練腳本中自動偵測啟用。
15. 診斷瓶頸的第一步是用假資料取代 DataLoader——最便宜也最常命中。
16. 最佳化必須逐項套用並各自量測，否則無法歸因；而知道何時停止最佳化，是比技術更難的能力。

成果檢核

完成本章後你必須繳交：

- 記憶體剖析報告（程式 18A）：四大類的實測拆解、與估算的對照、以及批次對記憶體的曲線。
- 效能矩陣（程式 18B）：矩陣乘法與逐元素運算在多種尺寸與精度下的量測，以及你自己裝置的脊點。
- 注意力後端診斷：flash 在你的設定下是否可用，若否是哪個條件不滿足。
- 瓶頸定位報告（程式 18C）：六步診斷的結果與判定依據。
- 最佳化實戰紀錄：至少三項手段的逐項量測，含改善幅度、副作用與數值驗證。
- 前後對照表：MFU、tokens/sec、峰值記憶體的初始與最終值。
- 停止判準：你認為還有多少改善空間，以及為什麼決定停在這裡。

本章的診斷工具會被第 19 章（分散式訓練的效能分析）、第 26 章（邊緣裝置的效能量測）與第 35 章（推論服務的壓力測試）直接沿用。從這一章開始，「跑得慢」對你而言不再是一個模糊的抱怨——

—它是一個可以被拆解、定位、並逐項修復的工程問題。

第 19 章

分散式訓練與多卡系統

Distributed Training and Multi-GPU Systems

難度：進階 | 建議授課：第 19 週 | 硬體需求：C 級（多卡）；單卡者可用 CPU gloo 後端驗證正確性

叫獸開講

八張卡不會讓你快八倍。第一次做分散式訓練的人，通常會在某個下午發現：加了四張卡，速度只快了兩倍多——而那多出來的電費與複雜度，可能不值得。這一章要教你的，是在按下多卡之前，先算清楚它能給你什麼、又要你付出什麼。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明四種平行策略的原理、通訊模式與適用情境。
2. 計算資料平行的通訊量，並判斷在給定頻寬下的擴展效率。
3. 解釋 ZeRO 三個階段各自分片什麼，並計算記憶體節省。
4. 說明張量平行為何需要高速互連，以及它的通訊量如何隨模型變化。
5. 計算管線平行的氣泡比例，並用微批次數量控制它。
6. 設計 3D 平行的組合，並說明各維度該優先分配到哪一層互連。
7. 正確處理分散式環境下的隨機種子、資料切分與 checkpoint。
8. 診斷多卡訓練的常見問題：不同步、死結、與單卡結果不一致。
9. 判斷一個專案是否真的需要多卡，並算出擴展的損益兩平點。

本章的前提

本章假設你已完成第 18 章的單卡最佳化。理由很實際：在一張沒有調好的卡上加卡，只是把低效率乘以卡數。如果你的 MFU 只有 0.12，先把它提升到 0.30 再考慮多卡——那通常比多買三張卡便宜得多，也簡單得多。這是本章最重要的一句話，值得寫在前面。

19.1 什麼時候該用多卡

19.1.1 三個理由，三種答案

你的理由	真正的問題	多卡是解方嗎	替代方案
訓練太慢	想縮短牆鐘時間	是，但有效率損失	先做第 18 章的最佳化
記憶體不夠	模型或批次放不下	是，這是最強的理由	checkpointing、LoRA、量化
別人都用	不是技術問題	否	先算清楚需求

第二列是最正當的理由。第 18.4.2 節算過：一個 7 B 模型的訓練需求達 161 GB，任何單卡都放不下。這種情況下多卡不是「加速」，而是「唯一可行的方式」。

第一列則需要謹慎評估。加卡確實能縮短時間，但效率一定小於一倍——你付出的是卡數，得到的是小於卡數的加速。19.2.3 節會給出精確的計算。

19.1.2 擴展的損益兩平

假設你考慮從一張卡擴展到 n 張。決策的關鍵是「擴展效率」：

$$\text{擴展效率} = \text{單卡的步驟時間} \div (n \text{ 卡的步驟時間} \times 1) \quad ; \quad \text{理想值為 } 1$$

$$\text{有效加速倍數} = n \times \text{擴展效率}$$

擴展效率	4 卡的有效加速	8 卡的有效加速	判讀
0.95	3.8×	7.6×	很好
0.85	3.4×	6.8×	可接受
0.70	2.8×	5.6×	偏低，需檢查
0.50	2.0×	4.0×	不值得，先找瓶頸

最後一列值得警惕：如果 8 張卡只換來 4 倍加速，那你的電費是四倍、複雜度是十倍，卻只得到四倍的產出。這種情況下應該先找出效率損失的來源（通常是通訊或負載不均），而不是繼續加卡。

19.1.3 多卡帶來的隱性成本

成本	具體表現	常被低估的程度	章節
除錯難度	錯誤可能只在某一張卡上發生	高	19.7
可重現性	通訊順序影響浮點加總	高	第 4.6.2 節
checkpoint 複雜度	分片的存檔與載入	中	19.6
資料管線	每張卡要拿到不同的資料	中	19.6.2
環境一致性	所有節點的版本必須相同	高	第 4.5 節
失敗處理	一張卡掛掉整個任務就停	高	19.7.3

第一列的除錯難度是最容易被低估的。單卡訓練出問題時，你可以加 `print`、可以下中斷點；多卡時，八個行程同時輸出，而錯誤可能只發生在 rank 3 上。本章 19.7 節會給出具體的處理方法，但你應該事先知道：多卡的除錯成本大約是單卡的三到五倍。

19.2 資料平行

19.2.1 原理

最直觀的平行方式：每張卡放一份完整的模型，各自處理不同的資料，然後把梯度平均起來。

1. 每張卡收到不同的微批次。
2. 各自前向、算損失、反向，得到自己的梯度。
3. 所有卡交換梯度並取平均（all-reduce）。
4. 各自用相同的平均梯度更新參數——因為初始參數相同、梯度相同，更新後參數也相同。

第四步是關鍵：因為所有卡的參數始終保持一致，所以任何一張卡的模型都是完整可用的。這讓資料平行的實作與 checkpoint 都相對單純。

19.2.2 通訊量

每一步都要對全部梯度做一次 all-reduce。以 7 B 參數、BF16 梯度為例，資料量是 14 GB。

但實際的傳輸量不是 14 GB。ring all-reduce 的演算法把資料切成 n 份，經過兩輪環狀傳遞完成——每張卡實際收發的量是：

$$\text{每卡傳輸量} = 2 \times (n - 1) / n \times \text{資料量}$$

卡數 n	係數 $2(n-1)/n$	每卡傳輸量	說明
2	1.00	14.0 GB	—
4	1.50	21.0 GB	—
8	1.75	24.5 GB	趨近 2 倍
很大	趨近 2.00	趨近 28 GB	上限

好消息是這個量有上界：無論多少張卡，每卡的傳輸量都不會超過資料量的兩倍。這是 ring all-reduce 相對於「全部送給一張卡再廣播」的優勢——後者會讓中心節點成為瓶頸。

19.2.3 通訊與計算的重疊

如果通訊要等計算全部完成才開始，那它就是純粹的額外開銷。但反向傳播是逐層進行的——最後一層的梯度先算出來，此時就可以開始傳送它，而不必等前面的層。

這種重疊能大幅降低通訊的影響。用實際數字看（假設單卡計算 2 秒）：

卡數	互連頻寬	通訊量	無重疊效率	80% 重疊
4	300 GB/s	21.0 GB	96.6%	99.3%
8	300 GB/s	24.5 GB	96.1%	99.2%
8	50 GB/s	24.5 GB	80.3%	95.3%

這張表傳達兩個重要訊息。第一，在高速互連（如同機內的專用連線）上，資料平行的效率極高——即使不做重疊也有 96%。第二，在低速互連（如一般網路）上，重疊變得關鍵：從 80.3% 提升到 95.3%。

對臺灣的實驗室而言，第三列是常見情境：用一般乙太網路連接數台工作站。此時務必確認你的框架有啟用梯度重疊（PyTorch 的 DDP 預設會做），並考慮把梯度壓縮成更低精度傳輸。

19.2.4 實作

`formosa/distributed/setup.py`

```
import os
import torch
import torch.distributed as dist
```

```

from torch.nn.parallel import DistributedDataParallel as DDP

def setup_distributed():
    """從環境變數初始化；由 torchrun 啟動時會自動設定"""
    if "RANK" not in os.environ:
        return {"distributed": False, "rank": 0, "world_size": 1,
                "local_rank": 0}

    rank = int(os.environ["RANK"])
    world_size = int(os.environ["WORLD_SIZE"])
    local_rank = int(os.environ["LOCAL_RANK"])

    backend = "nccl" if torch.cuda.is_available() else "gloo"
    dist.init_process_group(backend=backend)
    if torch.cuda.is_available():
        torch.cuda.set_device(local_rank)

    return {"distributed": True, "rank": rank,
            "world_size": world_size, "local_rank": local_rank,
            "backend": backend}

def is_main(info):
    """只有主行程才做記錄、存檔、印訊息"""
    return info["rank"] == 0

def wrap_ddp(model, info, gradient_as_bucket_view=True):
    if not info["distributed"]:
        return model
    device_ids = ([info["local_rank"]] if torch.cuda.is_available()
                  else None)
    return DDP(model, device_ids=device_ids,
               # 讓梯度直接使用通訊桶的記憶體，省一次複製
               gradient_as_bucket_view=gradient_as_bucket_view,
               # 若模型有未使用的參數才設為 True，否則會變慢
               find_unused_parameters=False)

def cleanup():
    if dist.is_initialized():
        dist.destroy_process_group()

```

`find_unused_parameters` 的預設值值得說明。設為 `True` 時，DDP 會在每一步掃描哪些參數沒有梯度——這有開銷。只有當你的模型確實有某些分支不一定被執行時才需要它。多數 Transformer 不需要，所以應該保持 `False`。

一個容易踩的坑

梯度累積（第 15.1 節）與 DDP 一起用時，每一個微批次都會觸發一次 `all-reduce`——那是不必要的，因為只有最後一次的梯度才會被用來更新。正確做法是在前 $n-1$ 個微批次上使用 `model.no_sync()` 情境管理器，

只在最後一個微批次進行同步。忘記這件事會讓通訊量變成累積步數倍，是多卡訓練中最常見的效能問題之一。

梯度累積與 DDP 的正確組合

```
from contextlib import nullcontext

def accumulate_with_ddp(model, batches, optimizer, info, grad_clip=1.0):
    optimizer.zero_grad(set_to_none=True)
    n = len(batches)
    token_counts = [int((y != -100).sum()) for _, y in batches]
    denom = sum(token_counts)

    for i, (x, y) in enumerate(batches):
        is_last = (i == n - 1)
        # 只有最後一個微批次才同步梯度
        ctx = (nullcontext() if (is_last or not info["distributed"])
              else model.no_sync())
        with ctx:
            _, loss_sum, _ = model(x, targets=y, reduction="sum")
            (loss_sum / denom).backward()

    gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
    optimizer.step()
    return float(gnorm)
```

19.2.5 資料平行的隱形前提

資料平行看起來很單純，但它建立在三個前提上。任何一個被違反，訓練就會出錯——而且往往不會報錯。

前提	為什麼必要	違反的後果	如何驗證
各卡初始參數相同	否則梯度平均沒有意義	模型行為變成隨機混合	verify_params_identical
各卡看到不同資料	否則等於白白浪費卡數	有效批次沒有真的變大	收集索引檢查交集
各卡執行相同的計算圖	否則集體操作對不上	死結或錯誤的歸約	避免依 rank 分支

第二列的檢查方法很簡單，卻很少人做：在訓練開始時，讓每張卡把自己前 100 個樣本的索引收集起來，檢查各卡之間的交集是否為空。如果不是空的，你的資料切分沒有生效。

一個容易產生這個問題的情境：使用 DataLoader 但忘記傳入 sampler，或是傳了 shuffle=True 卻同

時傳 sampler (PyTorch 會報錯，但有些寫法會靜默地讓 sampler 失效)。這個錯誤的症狀是「各卡的 loss 完全相同」——因為它們在算同一件事。

資料切分的驗證

```
import torch
import torch.distributed as dist

def verify_data_disjoint(loader, info, n_batches=10):
    """檢查各卡拿到的樣本索引沒有重疊"""
    if not info["distributed"]:
        return True

    # 收集本卡的索引 (需 dataset 回傳索引，或用 sampler 直接取)
    my_idx = []
    for i, idx in enumerate(loader.sampler):
        if i >= n_batches * loader.batch_size:
            break
        my_idx.append(idx)

    t = torch.tensor(my_idx, dtype=torch.long,
                     device=torch.cuda.current_device())
    gathered = [torch.zeros_like(t) for _ in range(info["world_size"])]
    dist.all_gather(gathered, t)

    if info["rank"] == 0:
        sets = [set(g.tolist()) for g in gathered]
        overlaps = []
        for i in range(len(sets)):
            for j in range(i + 1, len(sets)):
                inter = sets[i] & sets[j]
                if inter:
                    overlaps.append((i, j, len(inter)))
        if overlaps:
            print(f"[錯誤] 資料切分失效，重疊：{overlaps[:5]}")
            return False
        print(f"[檢查] 各卡資料無重疊 (各 {len(sets[0])} 筆)")
    return True
```

19.3 ZeRO 與分片

19.3.1 資料平行的浪費

資料平行有一個明顯的冗餘：每張卡都存了一份完整的模型狀態，而它們完全相同。八張卡就是八份相同的參數、梯度與最佳化器狀態。

ZeRO (Zero Redundancy Optimizer) 的想法是：既然它們相同，何不各存一部分、需要時再交換？

19.3.2 三個階段

階段	分片什麼	額外通訊	適用
ZeRO-1	最佳化器狀態 (m、v、主權重)	幾乎無	記憶體略不足時
ZeRO-2	再加上梯度	略增	記憶體明顯不足
ZeRO-3	再加上參數本身	明顯增加	模型單卡放不下

用 7 B 模型 (混合精度、每參數 16 bytes) 計算各階段的記憶體：

卡數	無分片	ZeRO-1	ZeRO-2	ZeRO-3
1	112.0 GB	112.0 GB	112.0 GB	112.0 GB
2	112.0 GB	70.0 GB	63.0 GB	56.0 GB
4	112.0 GB	49.0 GB	38.5 GB	28.0 GB
8	112.0 GB	38.5 GB	26.2 GB	14.0 GB

(模型狀態的組成：BF16 權重 2 + BF16 梯度 2 + FP32 主權重 4 + m 4 + v 4。ZeRO-1 分片後三項、ZeRO-2 再加梯度、ZeRO-3 全部分片。)

注意 ZeRO-1 的效益已經相當可觀：四張卡就從 112 GB 降到 49 GB。這是因為最佳化器狀態占了每參數 16 bytes 中的 12 bytes——四分之三。而 ZeRO-1 的額外通訊幾乎為零，所以它是一個「幾乎免費」的最佳化。

19.3.3 ZeRO-3 的代價

ZeRO-3 把參數本身也分片，這意味著計算某一層時必須先把該層的參數從其他卡收集過來 (all-gather)，用完再丟掉。

時機	通訊動作	資料量	能否重疊
前向，每層	all-gather 該層參數	該層參數量	可以 (預取下一層)
反向，每層	再次 all-gather	同上	可以
反向後	reduce-scatter 梯度	該層梯度量	可以

所以 ZeRO-3 的通訊量約為資料平行的 1.5 倍。但它換來的是「模型可以大於單卡記憶體」——那是一個質變而非量變。

實務上的建議是階梯式採用：先試 ZeRO-1，不夠再 ZeRO-2，真的放不下才 ZeRO-3。每往上一階，複雜度與通訊都增加，不該一開始就用最重的方案。

formosa/distributed/fsdp.py (PyTorch 的分片實作)

```
import functools
import torch
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy, MixedPrecision
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from formosa.model.block import TransformerBlock

STRATEGY = {
    "zero1": ShardingStrategy.SHARD_GRAD_OP, # 近似 ZeRO-2
    "zero2": ShardingStrategy.SHARD_GRAD_OP,
    "zero3": ShardingStrategy.FULL_SHARD,
    "none": ShardingStrategy.NO_SHARD,
}

def wrap_fsdp(model, stage="zero3", precision="bf16"):
    """以 TransformerBlock 為單位分片，這是常見的粒度"""
    wrap_policy = functools.partial(
        transformer_auto_wrap_policy,
        transformer_layer_cls={TransformerBlock},
    )

    mp = None
    if precision == "bf16":
        mp = MixedPrecision(
            param_dtype=torch.bfloat16,
            reduce_dtype=torch.float32, # 梯度歸約用 FP32 較穩
            buffer_dtype=torch.bfloat16,
        )

    return FSDP(
        model,
        sharding_strategy=STRATEGY[stage],
        auto_wrap_policy=wrap_policy,
        mixed_precision=mp,
        device_id=torch.cuda.current_device(),
        # 預取下一層的參數，讓通訊與計算重疊
        forward_prefetch=True,
        limit_all_gathers=True,
    )
```

reduce_dtype 設為 FP32 值得說明：梯度歸約是把多張卡的梯度相加，而那是一個大量小數累加的運算——第 3.6.1 節說過這種情況下精度很重要。用 FP32 做歸約、再轉回 BF16 使用，是一個成本很低

的穩定性保險。

19.4 張量平行與管線平行

19.4.1 張量平行：把單層切開

前面的方法都是「每張卡有完整的層」。張量平行則是把單一層的矩陣切開，讓多張卡各算一部分。

以前饋網路為例： W_1 是 (d, d_{ff}) ，可以按 d_{ff} 切成 n 份，每張卡算 $(d, d_{ff}/n)$ ； W_2 是 (d_{ff}, d) ，按 d_{ff} 切，最後把各卡的結果相加。

元件	怎麼切	需要的通訊	時機
注意力的頭	按頭切分	輸出投影後 all-reduce	每層一次
FFN 的 d_{ff}	第一層按列切、第二層按行切	第二層後 all-reduce	每層一次
嵌入與輸出投影	按詞彙表切	all-gather 或 all-reduce	首尾各一次

通訊量的計算：每次 all-reduce 傳的是 activation，大小為 $B \times T \times d$ 。以 $B = 4$ 、 $T = 2048$ 、 $d = 4096$ 、BF16 計算，單次約 0.067 GB。每層前向反向各兩次、32 層，總計約 8.6 GB。

這個數字看起來比資料平行的 24.5 GB 小，但有一個關鍵差異：張量平行的通訊發生在每一層的計算過程中，無法與計算重疊——因為下一步的計算需要這次通訊的結果。這使它對互連速度極度敏感。

策略	通訊時機	能否重疊	對互連的要求
資料平行	每步一次，反向結束時	可以（逐層重疊）	中
ZeRO-3	每層多次	部分可以（預取）	中高
張量平行	每層計算中	幾乎不能	極高
管線平行	階段邊界	可以	低

這張表給出一個重要的部署原則：張量平行只應該用在同一台機器內的高速互連上，跨機器使用會嚴重劣化。而管線平行對互連要求最低，適合跨機器。19.5 節的 3D 平行就是依這個原則組合的。

19.4.2 管線平行：把層分給不同的卡

把模型的層分成 p 段，每張卡負責一段。資料像生產線一樣依序通過各段。

問題是：當第一張卡在算第一段時，其他卡在等；當最後一張卡在算時，前面的卡也在等。這些等待時間稱為氣泡。

$$\text{氣泡比例} = (p - 1) / (m + p - 1) \quad \cdot p \text{ 為階段數、} m \text{ 為微批次數}$$

階段數 p	微批次 m	氣泡比例	效率	判讀
4	4	42.9%	57.1%	極差
4	16	15.8%	84.2%	可接受
4	64	4.5%	95.5%	很好
8	8	46.7%	53.3%	極差
8	32	17.9%	82.1%	可接受
8	64	9.9%	90.1%	好

結論很明確：微批次數必須遠大於階段數，通常建議 $m \geq 4p$ 。如果你的 m 只有 p 的兩倍，將近一半的算力會浪費在氣泡上。

這也解釋了管線平行的一個限制：它需要大的有效批次才能有效率，而那未必符合你的訓練需求（第 15.1.3 節談過批次大小有其最適範圍）。

19.5 3D 平行：把三種組合起來

19.5.1 組合的原則

大規模訓練通常同時使用三種平行：張量平行（TP）、管線平行（PP）、資料平行（DP）。組合的關鍵是把通訊需求高的維度放在互連快的層級。

維度	通訊特性	應放在哪一層	理由
張量平行	每層多次、無法重疊	同一機器內	需要最快的互連
管線平行	階段邊界、量小	跨機器	對頻寬要求最低
資料平行	每步一次、可重疊	最外層	可容忍較慢的網路

$$\text{總卡數} = TP \times PP \times DP$$

一個典型的配置：每台機器有 8 張卡， $TP = 8$ （機器內）、 $PP = 4$ （跨 4 台機器）、 $DP = 2$ （兩組這樣的配置），總共 64 張卡。

19.5.2 該用幾維

卡數	建議配置	理由	複雜度
2 至 8（同機）	DP 或 ZeRO	單一維度已足夠	低
8 至 32	ZeRO-3 或 $TP + DP$	記憶體與速度兼顧	中
32 至數百	$TP + PP + DP$	單一維度會撞牆	高
數百以上	3D + 專家平行等	需專門的框架與團隊	極高

對本書讀者而言，第一列與第二列涵蓋了絕大多數的實際情境。臺灣的大學實驗室通常是 2 到 8 張卡，此時 ZeRO 或單純的資料平行就足夠，不需要碰 3D 平行的複雜度。

國網中心的大型配置（第 21 章會詳述）可能達到第三列的規模，但那通常是有專人維運的環境，而非個別研究生自己組。本書的立場是：知道 3D 平行存在與它的組合原則就好，除非你的專案真的到了那個規模，否則不必自己實作。

19.5.3 專家平行：MoE 的特殊需求

第 20 章會介紹 MoE 架構。它有一個特殊的平行維度：把不同的專家放在不同的卡上。

面向	一般密集模型	MoE	意涵
每個詞元用到的參數	全部	只有被選中的專家	計算量較小
需要的通訊	all-reduce 梯度	額外的 all-to-all 路由	通訊模式不同
負載平衡	天然平衡	可能某些專家過載	需要輔助損失
記憶體	正比於參數	同樣正比，但參數多得多	需要專家平行

第三列是 MoE 訓練最麻煩的地方：如果路由把大部分詞元都送到少數幾個專家，那些卡會過載而其他卡閒置。這需要額外的機制來平衡，也是 MoE 比密集模型難訓練的主因之一。第 20 章會完整處理。

19.5.4 一張決策流程圖

把前面四節整合成一個可以照著走的決策流程。

步驟	問題	若「是」	若「否」
1	單卡 MFU 已在合理範圍嗎	進入步驟 2	先做第 18 章的最佳化
2	模型加上訓練狀態放得下單卡嗎	進入步驟 3	跳到步驟 5
3	訓練時間可以接受嗎	不需要多卡，結束	進入步驟 4
4	有多卡且互連良好嗎	用 DDP 資料平行	考慮縮小規模或租用
5	加上 checkpointing 後放得下嗎	回到步驟 3	進入步驟 6
6	ZeRO-1 或 ZeRO-2 夠嗎	採用，結束	進入步驟 7
7	ZeRO-3 夠嗎	採用，結束	進入步驟 8
8	有同機多卡的高速互連嗎	加上張量平行	加上管線平行

這個流程的設計邏輯是「由簡到繁」：每一步都先問「更簡單的方案夠不夠」，只有不夠才往下走。

這與 19.3.3 節說的「階梯式採用」是同一個精神。

對本書多數讀者而言，流程會在步驟 3 或步驟 4 結束——那是完全正常的，也是最省事的結果。步驟 8 需要的規模與維運能力，通常已經超出個別研究生的範圍。

19.6 分散式環境的正確性

19.6.1 隨機種子：哪些該相同、哪些該不同

這是分散式訓練最容易出錯、且錯了很難發現的地方。第 16.2.5 節談過單機的原則，多卡的情況更複雜。

項目	各卡應相同嗎	為什麼	做法
參數初始化	必須相同	否則各卡的模型不同	固定種子，或由 rank 0 廣播

項目	各卡應相同嗎	為什麼	做法
資料取樣	必須不同	否則各卡看到同樣的資料	DistributedSampler
dropout 遮罩	通常應不同	增加有效的正則化多樣性	seed + rank
資料切分	必須相同	否則驗證集不一致	固定鹽值 (第 5.4.5 節)
評測時的取樣	必須相同	確保各卡評的是同一件事	固定種子

第一列的錯誤最嚴重也最隱蔽：如果各卡的初始參數不同，資料平行的前提（參數始終一致）就被破壞了。訓練仍會進行，loss 也會下降，但模型的行為是各卡梯度的某種混合——而那不等於任何一個有意義的訓練過程。

formosa/distributed/seed.py

```
import random
import numpy as np
import torch
import torch.distributed as dist

def setup_distributed_seed(base_seed: int, info: dict):
    """參數初始化用相同種子，資料與 dropout 用 rank 相關的種子"""
    rank = info["rank"]

    # 階段一：所有卡用相同種子建立模型
    torch.manual_seed(base_seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(base_seed)
    return base_seed

def switch_to_per_rank_seed(base_seed: int, info: dict):
    """模型建立完成後才切換，供 dropout 與資料使用"""
    s = base_seed + info["rank"] * 10007 # 質數避免碰撞
    random.seed(s)
    np.random.seed(s % (2 ** 32))
    torch.manual_seed(s)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(s)

@torch.no_grad()
def verify_params_identical(model, info, tol=0.0):
    """驗證所有卡的參數確實相同——建模後必做"""
    if not info["distributed"]:
        return True
    checksum = sum(float(p.double().sum()) for p in model.parameters())
    t = torch.tensor([checksum], dtype=torch.float64,
                     device=torch.cuda.current_device()
                     if torch.cuda.is_available() else "cpu")
    gathered = [torch.zeros_like(t) for _ in range(info["world_size"])]
```

```

dist.all_gather(gathered, t)
values = [float(g) for g in gathered]
ok = max(values) - min(values) <= tol
if info["rank"] == 0:
    print(f"[檢查] 各卡參數校驗和：{'一致' if ok else '不一致！'}")
    if not ok:
        print(f" 範圍：{min(values)} 至 {max(values)}")
return ok

```

verify_params_identical 應該在建立模型後立刻執行。它只花幾毫秒，卻能抓出一整類難以診斷的問題。本書要求：任何分散式訓練腳本都必須包含這個檢查。

19.6.2 資料切分

可續訓的分散式取樣器

```

import torch
from torch.utils.data import Sampler

class DistributedResumableSampler(Sampler):
    """結合第 15.5.3 節的續訓能力與分散式切分"""

    def __init__(self, dataset_size, rank, world_size,
                 seed=42, shuffle=True):
        self.n = dataset_size
        self.rank = rank
        self.world_size = world_size
        self.seed = seed
        self.shuffle = shuffle
        self.consumed = 0  # 全域已消耗的樣本數

    def set_position(self, global_consumed: int):
        self.consumed = global_consumed

    def __iter__(self):
        per_rank = self.n // self.world_size
        epoch = self.consumed // self.n
        offset = (self.consumed % self.n) // self.world_size
        while True:
            g = torch.Generator().manual_seed(self.seed + epoch)
            order = (torch.randperm(self.n, generator=g) if self.shuffle
                     else torch.arange(self.n))
            # 交錯切分：rank 0 拿 0, n, 2n...；確保各卡不重疊
            mine = order[self.rank::self.world_size][:per_rank]
            for idx in mine[offset:].tolist():
                yield idx
            epoch += 1
            offset = 0

    def __len__(self):
        return self.n // self.world_size

```

交錯切分 (rank::world_size) 而非連續切分是刻意的：如果資料有某種順序性（例如按來源排列），連續切分會讓每張卡只看到某一類資料，而交錯切分能保證分布相近。

19.6.3 checkpoint 的處理

策略	做法	優點	缺點
僅 rank 0 存	主行程存完整模型	單一檔案、易載入	ZeRO-3 時需先收集
分片存檔	每張卡存自己的分片	快、省記憶體	載入時卡數必須相同
分片存 + 轉換	訓練時分片、結束時合併	兩者兼顧	需額外步驟

本書建議第三種：訓練期間用分片存檔（快速、可續訓），訓練結束後產出一份合併的完整權重（供部署與分享）。這樣既不拖慢訓練，也讓最終產物可以在任何配置下使用。

分散式 checkpoint 的存取

```
import torch
import torch.distributed as dist
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import StateDictType, FullStateDictConfig

def save_full_from_fsdp(model, path, info):
    """把分片的參數收集成完整的 state_dict（僅 rank 0 寫檔）"""
    cfg = FullStateDictConfig(offload_to_cpu=True, rank0_only=True)
    with FSDP.state_dict_type(model, StateDictType.FULL_STATE_DICT, cfg):
        state = model.state_dict()
        if info["rank"] == 0:
            torch.save(state, path)
            print(f"[存檔] 完整權重已寫入 {path}")
        dist.barrier()  # 確保所有卡等主行程寫完

def all_ranks_barrier(info, label=""):
    """在關鍵節點同步，避免某些卡跑太前面"""
    if info["distributed"]:
        dist.barrier()
        if info["rank"] == 0 and label:
            print(f"[同步] {label}")
```

`dist.barrier()` 在存檔後的呼叫是必要的。如果沒有它，非主行程會繼續訓練，而主行程還在寫一個可能數 GB 的檔案——結果是各卡的進度不同步，且下次同步時會有一張卡莫名地慢。

19.7 多卡的除錯

19.7.1 六種常見問題

問題	症狀	常見原因	診斷
參數不一致	loss 異常、行為古怪	初始化種子不同	verify_params_identical
死結	程式卡住不動	各卡執行的集體操作不一致	看每張卡卡在哪一行
某卡 OOM	單一 rank 崩潰	負載不均或該卡有其他行程	逐卡記錄記憶體
速度遠低於預期	擴展效率差	通訊未重疊、梯度累積未 no_sync	看通訊占比
結果與單卡不同	同 seed 卻不一致	歸約順序、資料切分	先確認是否在誤差內
某卡特別慢	整體被拖慢	硬體差異、散熱降頻	逐卡量測吞吐量

第二列的死結值得特別說明。集體操作 (all-reduce、barrier、broadcast) 必須被所有卡以相同的順序呼叫。如果你寫了 `if rank == 0: dist.broadcast(...)`，那麼只有 rank 0 會進入這個呼叫，其他卡不會——結果是 rank 0 永遠等不到別人，整個任務卡死。

這類錯誤的典型來源是把記錄或存檔的邏輯與集體操作混在一起。原則是：條件分支中不可包含集體操作。如果某個操作只需要主行程做，那就把集體操作放在分支外面。

19.7.2 診斷工具

formosa/distributed/diagnostics.py

```
import time
import torch
import torch.distributed as dist

def per_rank_report(info, **metrics):
    """收集各卡的指標並由主行程統一輸出"""
    if not info["distributed"]:
        print(metrics); return metrics

    keys = sorted(metrics)
    t = torch.tensor([metrics[k] for k in keys], dtype=torch.float64,
                     device=torch.cuda.current_device())
    gathered = [torch.zeros_like(t) for _ in range(info["world_size"])]
    dist.all_gather(gathered, t)

    if info["rank"] == 0:
        print(f"{'rank':>5}" + "".join(f"{k:>16}" for k in keys))
        vals = {k: [] for k in keys}
        for r, g in enumerate(gathered):
            row = [float(x) for x in g]
            print(f"{'r':>5}" + "".join(f"{v:>16.3f}" for v in row))
            for k, v in zip(keys, row):
                vals[k].append(v)
        # 找出離群的卡
```

```

        for k, vs in vals.items():
            spread = (max(vs) - min(vs)) / max(abs(sum(vs) / len(vs)), 1e-9)
            if spread > 0.2:
                slow = vs.index(max(vs))
                print(f"  [警告] {k} 各卡差異達 {spread:.1%}, "
                      f"    最慢為 rank {slow}")
        return metrics

def measure_comm_overhead(info, tensor_gb=1.0, iters=10):
    """量測實際的互連頻寬"""
    if not info["distributed"]:
        return None
    n = int(tensor_gb * 1e9 / 2)
    t = torch.ones(n, dtype=torch.bfloat16,
                   device=torch.cuda.current_device())
    for _ in range(3):
        dist.all_reduce(t)
    torch.cuda.synchronize()

    t0 = time.perf_counter()
    for _ in range(iters):
        dist.all_reduce(t)
    torch.cuda.synchronize()
    dt = (time.perf_counter() - t0) / iters

    w = info["world_size"]
    effective_gb = 2 * (w - 1) / w * tensor_gb    # 見 19.2.2 節
    return {"seconds": round(dt, 5),
            "effective_GB": round(effective_gb, 3),
            "achieved_GB_per_sec": round(effective_gb / dt, 1)}

```

`measure_comm_overhead` 量到的是你的實際互連頻寬，而不是規格上的。這個數字應該用來代入 19.2.3 節的公式，估算你的擴展效率——用規格值算出來的預期通常過於樂觀。

19.7.3 容錯與長時間訓練

狀況	影響	處置	章節
單卡硬體故障	整個任務中止	從 checkpoint 續訓 (可能減少卡數)	第 15.5 節
網路中斷	集體操作逾時	設定合理的 timeout 並重試	19.7.2
某節點被搶占	同上	彈性訓練 (可變卡數)	進階議題
某卡降頻	整體變慢但不中止	偵測並排除該卡	per_rank_report

第一列在共用叢集中很常見。本書的建議與第 15.5 節一致：把存檔間隔設在能接受的損失範圍內（例如 30 分鐘），並確認續訓能在不同卡數下進行——那需要使用「合併的完整權重」而非分片檔

(19.6.3 節) 。

19.7.4 一份多卡啟動前的檢查清單

延續第 18 章的效能自檢清單，這裡給出多卡專用的版本。建議在每次啟動分散式訓練前執行。

項次	檢查什麼	通過標準	節次
1	單卡自檢清單已通過	第 18 章的 12 項	第 18 章
2	各卡參數是否一致	校驗和完全相同	19.6.1
3	各卡資料是否不重疊	索引交集為空	19.2.5
4	梯度累積是否使用 no_sync	程式碼檢查	19.2.4
5	條件分支中是否有集體操作	程式碼檢查	19.7.1
6	所有節點的版本是否一致	環境指紋比對	第 4.5 節
7	實際互連頻寬是否量測過	measure_comm_overhead	19.7.2
8	擴展效率是否符合預期	對照 19.1.2 節的表格	19.1.2
9	各卡的吞吐量是否相近	差異小於 20%	19.7.2
10	checkpoint 策略是否確認	分片或完整，能否跨卡數載入	19.6.3
11	存檔後是否有 barrier	程式碼檢查	19.6.3
12	失敗時的續訓路徑是否測過	實際中斷一次	19.7.3

第 1 項是前提，第 2 到 5 項是正確性，第 6 到 9 項是效能，第 10 到 12 項是韌性。四組的優先順序就是這個順序——一個跑得快但參數不一致的訓練，比一個慢但正確的訓練糟糕得多。

第 12 項特別建議在正式訓練前實際演練一次：故意中斷、然後續訓、確認 loss 接得上。多卡的續訓比單卡複雜（涉及分片、資料位置、各卡同步），第一次通常會失敗——而你不希望是在跑了二十小時之後才發現。

程式實作

本章的實作需要至少兩張 GPU。若你只有單卡，可以用 CPU 的 gloo 後端模擬多行程——雖然速度沒有意義，但正確性的邏輯完全相同，仍值得跑一次。

程式 19A 資料平行訓練腳本

目標：把第 15 章的訓練腳本改為支援多卡，並通過五項正確性驗收。

驗收項	測試方式	通過標準	節次
參數一致	verify_params_identical	各卡校驗和完全相同	19.6.1
資料不重疊	收集各卡的樣本索引	交集為空	19.6.2
梯度同步	比較各卡更新後的參數	完全相同	19.2.1
等價性	單卡 B=32 vs 四卡 B=8	loss 曲線接近	19.2
續訓	中斷後以相同卡數續訓	loss 序列接得上	第 15.5 節

scripts/train_ddp.py (主要差異處)

```
def main():
    info = setup_distributed()
    device = torch.device(f"cuda:{info['local_rank']}")
        if torch.cuda.is_available() else "cpu")

    # 步驟一：所有卡用相同種子建模
    setup_distributed_seed(tcfg.seed, info)
    model = MiniFormosaGPT(mcfg).to(device)
    assert verify_params_identical(model, info), "各卡參數不一致"

    # 步驟二：包裝 DDP，之後切換為各卡不同的種子
    model = wrap_ddp(model, info)
    switch_to_per_rank_seed(tcfg.seed, info)

    # 步驟三：資料切分
    sampler = DistributedResumableSampler(
        len(train_ds), info["rank"], info["world_size"], seed=tcfg.seed)
    loader = DataLoader(train_ds, batch_size=tcfg.micro_batch,
        sampler=sampler, drop_last=True,
        num_workers=tcfg.num_workers, pin_memory=True)

    # 步驟四：只有主行程做記錄與存檔
    log = setup_logging(out, tcfg.run_name) if is_main(info) else None

    for step in range(start_step, tcfg.max_steps):
        batches = [next(it) for _ in range(tcfg.grad_accum)]
        gnorm = accumulate_with_ddp(model, batches, opt, info)
```

```

if step % tcfg.log_every == 0 and is_main(info):
    log.info(f"step {step} gnorm {gnorm:.3f}")

if step % tcfg.save_every == 0:
    # 注意：存檔本身在主行程，但 barrier 所有卡都要呼叫
    if is_main(info):
        save_checkpoint(...)
    all_ranks_barrier(info, f"存檔完成 step {step}")

cleanup()

```

請特別注意最後那段存檔的寫法：`if is\_main` 只包住寫檔，而 `all\_ranks\_barrier` 在外面——所有卡都會呼叫它。這正是 19.7.1 節說的「條件分支中不可包含集體操作」的正確實踐。

程式 19B 擴展效率量測

目標：實測 1、2、4、8 卡（依你的硬體）的擴展效率，並與 19.2.3 節的理論值對照。

必須產出的五項：

- 各卡數的 tokens/sec 與擴展效率曲線。
- 通訊占比：用 `measure_comm_overhead` 量到的實際頻寬，代入公式算出預期通訊時間。
- 重疊效果：分別量測有無梯度重疊的差異（可用環境變數關閉 DDP 的重疊）。
- 「no_sync 錯誤」的代價：故意在梯度累積時不使用 `no_sync`，量測速度差異。
- 一段分析：你的擴展效率在哪個卡數開始明顯下降？瓶頸是通訊還是別的？

第四項的預期結果

如果你的梯度累積步數是 4，忘記使用 `no_sync` 會讓通訊量變成四倍。在高速互連上可能只慢一成，但在一般網路上可能慢一倍以上。這個實驗的價值在於：它讓一個「看不見的錯誤」變成一個可量測的數字。

程式 19C ZeRO 記憶體實測

目標：實測 ZeRO 各階段的記憶體節省，並與 19.3.2 節的理論值對照。

量測項	方法	對照	節次
各階段峰值記憶體	程式 18A 的剖析器	19.3.2 節的表格	19.3.2
通訊量增加	<code>measure_comm_overhead</code>	ZeRO-3 約 1.5 倍	19.3.3
吞吐量代價	tokens/sec 對照	視互連速度而定	19.3.3

量測項	方法	對照	節次
最大可訓練模型	逐步加大直到 OOM	應接近理論值	19.3.2

最後一項是最有價值的：找出「在你的硬體上，用 ZeRO-3 最大能訓練多大的模型」。這個數字直接餵回第 17 章的算力預算——它是資料之外的另一個硬約束。

系統案例：分散式訓練的決策報告

本章的系統案例不是「做出一個分散式訓練」，而是「決定要不要做、以及怎麼做」。這反映了本章的核心立場：多卡是手段而非目的。

要回答的五個問題

問題	需要什麼資料	工具	節次
單卡最佳化過了嗎	第 18 章的自檢清單結果	程式 18C	第 18 章
為什麼需要多卡	記憶體或時間的具體缺口	第 17 章的預算	19.1.1
該用哪種策略	模型大小、卡數、互連速度	19.4.1 的對照表	19.3–19.5
效率是多少	實測的擴展效率	程式 19B	19.2.3
值得嗎	成本與加速的比較	第 17.6 節	19.1.2

第一列是本案例的前提。如果你的單卡 MFU 只有 0.12，那麼「加卡」的正確答案通常是「不要」——先把單卡調到 0.30，那等同於免費得到 2.5 倍的加速，而且不增加任何複雜度。

交付要求

- 單卡最佳化紀錄：第 18 章自檢清單的結果，證明你已經做過。
- 需求論證：具體的記憶體缺口或時間需求，附第 17 章的計算。
- 策略選擇：依 19.4.1 與 19.5.2 的原則選定，並說明為什麼不選其他方案。
- 擴展效率實測：程式 19B 的完整結果，含理論值對照。
- 正確性驗收：程式 19A 的五項全部通過的紀錄。
- 成本效益分析：加卡的成本（硬體、電力、複雜度）與得到的加速，以及你的結論。

- 若結論是「不需要多卡」，同樣是合格的交付——但必須有資料支撐。

評分重點

- 是否先做過單卡最佳化。跳過這一步直接上多卡者，本案例不予計分。
- 正確性驗收是否全數通過，特別是參數一致性與資料不重疊。
- 擴展效率是否實測而非引用文獻。
- 成本效益分析是否誠實。若效率只有 0.6 仍主張值得，必須說明理由。
- 「不需要多卡」的結論是否被公平對待——這常常是正確答案。

最後一項是本案例刻意設計的。學生往往覺得「做出分散式訓練」比「論證不需要它」更有成就感，但後者通常更接近正確的工程判斷。能誠實地說「算過之後，我們的專案不需要多卡」，是一種成熟。

章末評量

一、概念題

1. 為什麼說「在一張沒調好的卡上加卡，只是把低效率乘以卡數」？
2. 資料平行為什麼能保證各卡的參數始終一致？這個性質帶來什麼好處？
3. 解釋 ring all-reduce 的傳輸量公式，並說明為什麼它有上界。
4. 為什麼梯度重疊對低速互連特別重要？請用 19.2.3 節的數字說明。
5. ZeRO 三個階段各自分片什麼？為什麼 ZeRO-1 的效益已經相當可觀？
6. 為什麼張量平行只適合同機內使用，而管線平行適合跨機器？
7. 管線平行的氣泡從何而來？為什麼微批次數必須遠大於階段數？
8. 在分散式環境中，哪些隨機種子該相同、哪些該不同？各自的理由是什麼？
9. 為什麼「條件分支中不可包含集體操作」？違反會發生什麼？

二、計算題

1. 一個 3 B 參數的模型用 BF16 梯度做資料平行。計算 4 卡與 8 卡時每卡的傳輸量。
2. 承上題，若互連頻寬為 100 GB/s、單卡計算時間 1.5 秒。計算 8 卡在無重疊與 80% 重疊下的擴展效率。
3. 一個 13 B 模型混合精度訓練（每參數 16 bytes）。計算無分片、ZeRO-1、ZeRO-2、ZeRO-3 在 8 卡下的每卡記憶體。
4. 管線平行 $p = 6$ 、微批次 $m = 24$ 。計算氣泡比例與效率。若要達到 90% 效率， m 至少要多少？
5. 總共 32 張卡，每台機器 8 張。若採用 $TP = 8$ 、 $PP = 2$ ， DP 是多少？
6. 梯度累積 8 步，忘記使用 no_sync。通訊量變成正確做法的幾倍？若原本通訊占步驟時間的 10%，現在占多少？

三、除錯題

1. 四卡訓練的 loss 明顯高於單卡，且波動較大。請列出四個可能原因與檢查順序。
2. 程式在第一個 all-reduce 就卡住不動。請列出三個可能原因。
3. 八卡訓練中，rank 5 特別慢，拖慢了整體。請列出三個可能原因與診斷方式。
4. 從 checkpoint 續訓時，四卡的結果與原本八卡的不一致。請說明可能的原因。
5. 啟用 ZeRO-3 後記憶體確實下降，但速度慢了三倍。請列出三個可能原因。
6. 各卡的 loss 印出來完全相同，一點差異都沒有。請說明這可能代表什麼問題。

四、系統設計題

1. 你的實驗室有四台工作站，各有兩張 GPU，用一般乙太網路連接。請設計一個訓練方案：該用什麼平行策略、預期的擴展效率、以及你會如何驗證。並說明這個配置的主要限制。
2. 你要為一個共用叢集寫一份「分散式訓練規範」，讓多個團隊能公平地使用資源。請設計：資源申請規則、失敗處理、以及如何避免某個團隊的任務影響其他人。

五、臺灣情境與倫理題

1. 第 21 章會談國網中心的資源。請討論：對一個只有 2 到 4 張卡的臺灣大學實驗室，本章的哪些內容最實用、哪些可以略過？並說明你的判斷依據。
2. 多卡訓練的電力消耗是單卡的數倍，而擴展效率往往小於 1——這意味著單位產出的能耗更高。請討論：在什麼情況下這個額外的環境成本是正當的？研究團隊該如何做這個判斷？

評量提示（給自學者）

- 計算題第 1 題：梯度 6 GB。4 卡為 $2 \times (3/4) \times 6 = 9$ GB；8 卡為 $2 \times (7/8) \times 6 = 10.5$ GB。
- 計算題第 2 題：通訊時間 $10.5/100 = 0.105$ 秒。無重疊： $1.5/(1.5+0.105) \approx 93.5\%$ ；80% 重疊： $1.5/(1.5+0.021) \approx 98.6\%$ 。
- 計算題第 3 題：總 208 GB。無分片 208；ZeRO-1 為 $13e9 \times (2+2+12/8)/1e9 = 71.5$ ；ZeRO-2 為 $13e9 \times (2+(2+12)/8)/1e9 = 48.75$ ；ZeRO-3 為 $208/8 = 26$ GB。
- 計算題第 4 題：氣泡 $= 5/29 \approx 17.2\%$ ，效率 82.8%。要達 90% 需 $(p-1)/(m+p-1) \leq 0.1$ ，解得 $m \geq 45$ 。
- 計算題第 5 題： $DP = 32/(8 \times 2) = 2$ 。
- 計算題第 6 題：8 倍。原本 10% 表示計算占 90%，現在通訊為 $0.8/0.9 \approx 0.89$ 個計算單位，占比約 47%。
- 除錯題第 6 題：各卡 loss 完全相同代表它們看到了相同的資料——資料切分沒有生效，等於白白浪費了卡數。

研究延伸與版本注意事項

- ZeRO 的原始論文清楚說明了三個階段的設計與記憶體分析，是本章 19.3 節的直接來源。
- Megatron-LM 的系列論文：張量平行與 3D 平行的經典實作，19.4 與 19.5 節的內容多來自此。
- 管線平行的排程策略（如 1F1B、交錯排程）：本章只介紹了最基本的形式，實務上有多種降低氣泡的技巧。

- PyTorch FSDP 的官方文件：本章 19.3.3 節的實作以它為準，API 在版本間變動較快。
- NCCL 的效能調校：集體通訊的實際效能受環境變數與拓撲影響甚大，若你的擴展效率不如預期值得查閱。
- 本章的原理穩定，但框架 API 與最佳實務變動快速。所有效能數字都必須在你自己的環境上重測。

常見問題

問：我只有兩張卡，需要學 3D 平行嗎？答：不需要實作，但值得知道原則。兩張卡用 DDP 或 ZeRO-1 就足夠，而 19.4.1 節的「通訊需求對應互連層級」是一個通用的思考框架，即使你不用 3D 平行也有幫助。

問：多卡訓練的結果與單卡不同，是 bug 嗎？答：不一定。第 4.6.2 節說過浮點加法不滿足結合律，而多卡的歸約順序與單卡不同——微小的差異是預期的。判準是：差異是否在合理範圍內（通常小數點後四五位），以及訓練曲線是否重合。如果 loss 差了 0.5，那就是 bug。

問：DDP 與 FSDP 該選哪個？答：如果模型放得下單卡，DDP 較簡單且通常較快。放不下才用 FSDP。一個常見的錯誤是「聽說 FSDP 比較先進所以用它」——在小模型上它只會增加通訊而沒有好處。

問：擴展效率多少才算好？答：同機內高速互連上，8 卡應該能達到 0.9 以上；跨機器的一般網路上，0.7 到 0.85 是常見範圍。如果你的數字明顯低於這些，先檢查 19.7.1 節的六種問題——特別是梯度累積的 no_sync。

教師使用建議

本章排在第 19 週，建議 2 小時講授加 1 小時實驗。本章的挑戰在於多數學生沒有多卡環境，教學上需要用模擬與計算彌補。

時段	內容	互動設計	注意事項
前 20 分鐘	19.1 節，什麼時候該用多卡	請學生算自己專題的擴展效益	「不需要多卡」要被正常化
中間 30 分鐘	19.2 至 19.3 節，資料平行與 ZeRO	現場算 ZeRO 各階段的記憶體	表格的對比很有說服力
後 30 分鐘	19.4 至 19.5 節，張量與管線平行	請學生算不同 m 的氣泡比例	氣泡公式要親手代一次

時段	內容	互動設計	注意事項
最後 30 分鐘	19.6 至 19.7 節，正確性與除錯	示範一個死結的例子	死結的示範印象深刻
實驗課 1 小時	程式 19A (可用 CPU gloo 模擬)	兩人一組，一人故意引入錯誤	互相找錯很有效

關於沒有多卡環境的處理：用 CPU 的 gloo 後端可以在單機上啟動多個行程，完整驗證所有的正確性邏輯（參數一致、資料切分、集體操作）。速度沒有意義，但那不是重點——本章的正確性部分遠比效能部分難學，而它完全可以在 CPU 上練習。

一個效果很好的示範：現場寫一個包含 `if rank == 0: dist.barrier()` 的程式並執行，讓學生看到它卡死。然後把 barrier 移到條件外面，程式就正常了。這個五分鐘的示範，比講十分鐘的原則有效得多。

高中授課的調整：本章整體偏進階，建議只教 19.1 與 19.2.1（用「多人分工做同一份考卷」的比喻），以及 19.4.2 的氣泡概念（用「接力賽的交接空檔」比喻）。19.3 至 19.7 略過。程式部分不做，改為讓學生計算「四張卡能加速幾倍」的練習。

本章小結

1. 在一張沒調好的卡上加卡，只是把低效率乘以卡數——多卡之前必須先完成第 18 章的單卡最佳化。
2. 多卡最正當的理由是記憶體不足；為了加速而多卡時，必須先算清楚擴展效率。
3. 多卡的隱性成本包括除錯難度（約單卡的三到五倍）、可重現性、與失敗處理。
4. 資料平行讓各卡參數始終一致，實作與 checkpoint 都相對單純。
5. ring all-reduce 的每卡傳輸量為 $2(n-1)/n$ 倍資料量，有 2 倍的上界。
6. 梯度重疊在低速互連上是關鍵：8 卡 50 GB/s 的效率可從 80.3% 提升到 95.3%。
7. 梯度累積時必須使用 no_sync，否則通訊量會變成累積步數倍——這是最常見的多卡效能問題。
8. ZeRO-1 分片最佳化器狀態（占每參數 16 bytes 中的 12），效益大而通訊幾乎為零，應優先採用。
9. ZeRO-3 讓模型可以大於單卡記憶體，代價是約 1.5 倍的通訊量。
10. 張量平行的通訊發生在每層計算中、無法重疊，只適合同機內；管線平行對頻寬要求最低，適合跨機。
11. 管線氣泡為 $(p-1)/(m+p-1)$ ；微批次數應至少為階段數的四倍。

12. 3D 平行的組合原則是把通訊需求高的維度放在互連快的層級；2 至 8 卡通常不需要它。
13. 參數初始化的種子必須各卡相同、資料取樣必須不同；建模後應立即驗證參數一致。
14. 條件分支中不可包含集體操作，否則會死結；存檔的 barrier 必須在條件外。
15. checkpoint 建議訓練時分片、結束時合併，兼顧速度與可攜性。
16. 若計算後的結論是「不需要多卡」，那同樣是一個合格且常常正確的工程判斷。

成果檢核

完成本章後你必須繳交：

- 分散式訓練腳本（程式 19A）：通過五項正確性驗收，含參數一致性與資料不重疊。
- 擴展效率報告（程式 19B）：各卡數的實測曲線、實際互連頻寬、以及與理論值的對照。
- no_sync 實驗：故意不用 no_sync 的速度差異量測。
- ZeRO 記憶體實測（程式 19C）：各階段的峰值記憶體與理論對照，以及最大可訓練模型的實測。
- 分散式訓練決策報告：五個問題的完整回答與結論。
- 若你的結論是「不需要多卡」，請附上支撐的計算——這與做出多卡實作同樣合格。

本章的工具會被第 21 章（臺灣算力生態的實務評估）、第 22 章（大規模持續預訓練）與第 40 章（總整專題的資源規劃）使用。但更重要的是本章的判斷框架：多卡是解決特定問題的手段，而不是技術能力的展示。能誠實評估它是否必要，比會用它更接近工程的本質。

第 20 章

現代架構、長上下文與混合專家

Modern Architectures, Long Context, and Mixture-of-Experts

難度：進階 | 建議授課：第 20 週 | 硬體需求：A 級可完成位置編碼實驗；長上下文評測可用 API 模型

叫獸開講

「我們的模型支援 128K 上下文」——這句話的真正意思，可能是「它不會報錯」，而不是「它在第 100,000 個字的位置還記得你說過什麼」。這一章要教你怎麼分辨這兩件事，以及怎麼在有限的資源下真的把上下文拉長。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明長上下文的三重瓶頸：計算、記憶體與位置外推。
2. 比較位置內插、NTK 感知縮放與 YaRN 三種延長方法的原理與代價。
3. 實作滑動視窗與稀疏注意力，並計算它們的複雜度節省。
4. 說明 MoE 的路由機制、負載平衡問題與參數量的計算方式。
5. 區分總參數量與活躍參數量，並解釋兩者對成本的不同意涵。
6. 比較狀態空間模型與注意力的取捨，說明各自擅長什麼。
7. 設計長上下文的評測方法，辨識「支援」與「真的能用」的差別。
8. 為繁體中文的長文件應用選擇合適的架構與上下文策略。
9. 在架構選型時做出有資料支撐的取捨，而非追隨潮流。

本章的性質

這是全書最「不穩定」的一章——它介紹的技術有相當比例在近幾年才出現，而且仍在快速演進。因此本章的重點不是讓你記住每一個方法，而是建立兩件事：一是理解每個技術在解決什麼瓶頸，二是掌握評估它們的方法。技術會過時，但「這個方法省了什麼、付出什麼」的分析框架不會。

20.1 長上下文的三重瓶頸

20.1.1 三個獨立的問題

「把上下文從 4K 拉到 128K」聽起來是一件事，實際上是三個必須分別解決的問題。

瓶頸	問題	解法方向	本章節次
計算	注意力的 FLOPs 隨 T^2 成長	稀疏化、滑動視窗	20.3
記憶體	KV cache 隨 T 線性成長	GQA、視窗、壓縮	20.3.4
位置外推	模型沒見過那麼遠的位置	內插、縮放、微調	20.2

這三者的獨立性很重要。FlashAttention (第 18.5 節) 解決了記憶體中的注意力矩陣問題，但沒有解決 KV cache 與位置外推；位置內插解決了外推，但沒有降低計算量。一個宣稱支援長上下文的模型，可能只解決了其中一項。

20.1.2 「支援」與「能用」的差距

一個模型可以在 128K 的輸入上跑完而不報錯，但那不代表它在使用那些資訊。

宣稱	技術上的意思	未必成立的是	如何檢驗
支援 128K	不會因為長度而崩潰	真的在使用遠處的資訊	大海撈針測試
位置編碼可外推	能算出任意位置的編碼	注意力分布仍然合理	注意力熵隨距離的變化
通過長文件測試	在某些任務上表現尚可	所有位置都同樣可靠	分位置的準確率

第一列的「大海撈針」(needle in a haystack) 測試是業界常用的方法：在一份長文件的某個位置埋入一句特定的資訊，然後問模型那句話的內容。改變埋入的位置與文件長度，畫出一張二維的成功率熱圖。

典型的結果會呈現「兩端好、中間差」的形狀——模型對開頭與結尾的資訊記得清楚，中段容易遺漏。這個現象在多份研究中被觀察到，也是本章 20.6 節評測設計的重點。

20.1.3 對臺灣應用的意義

應用	典型長度	需要長上下文嗎	替代方案
公文摘要	1K 至 5K 字	不需要	直接處理
法規彙編查詢	數十萬字	需要，但可切塊	RAG (第 30 章)

應用	典型長度	需要長上下文嗎	替代方案
契約全文審閱	2 萬至 10 萬字	有幫助	長上下文或分段
多輪長對話	隨輪次累積	需要	摘要壓縮或視窗
程式碼庫理解	極長	需要	檢索加長上下文

第二列值得特別說明。很多人以為「法規太長所以需要長上下文」，但實際上 RAG 通常是更好的方案——因為你只需要相關的幾條，而不是整部法規。長上下文與檢索不是互斥的，第 30、31 章會處理它們的組合。

真正需要長上下文的，是那些「無法事先知道哪一段相關」的任務——例如契約的前後條款一致性檢查、或跨章節的邏輯矛盾偵測。這類任務切塊會遺失關聯，而那正是長上下文的價值所在。

20.2 位置外推

20.2.1 問題的本質

第 10.4.4 節提過 RoPE 的外推限制。這裡把原因講清楚。

RoPE 把位置編碼成旋轉角度，不同維度用不同的頻率（第 10.4.1 節）。訓練長度為 4,096 時，模型見過的是「相對距離 0 到 4,095」對應的角度組合。超出這個範圍，高頻的維度已經轉了很多圈，產生模型從未見過的角度組合——注意力分布就會失控。

用具體數字看各頻率分量的週期（ $\text{head_dim} = 128$ 、 $\text{base} = 10000$ ）：

維度 j	週期（位置數）	在 4K 訓練中	外推時
0（最高頻）	6.3	轉了約 650 圈	仍在見過的範圍內
16	62.8	轉了約 65 圈	同上
32	628.3	轉了約 6.5 圈	同上
63（最低頻）	54,410	只轉了 0.075 圈	超出即為未見過

關鍵在最後一列：低頻維度在訓練中根本沒轉完一圈。它們負責編碼長距離的資訊，但模型只見過角度 0 到 0.075 圈的部分。要求它處理 32K 的距離，等於要求它外推到 0.6 圈——那是完全陌生的區域。

20.2.2 三種延長方法

方法	做什麼	優點	代價
位置內插 (PI)	把位置索引除以擴展倍數 s	簡單、只需少量微調	短距離解析度下降 s 倍
NTK 感知縮放	調高 base，讓低頻轉得更慢	短距離幾乎不受影響	需要調參
YaRN	對不同頻率分別處理，加注意力溫度	兼顧長短距離	實作較複雜

► 位置內插

最直觀的做法：想把 4K 擴展到 32K ($s = 8$)，就把位置索引除以 8。這樣第 32,000 個位置的角度，等同於原本第 4,000 個位置——完全落在訓練見過的範圍內。

代價是相鄰位置的角度差變成原本的八分之一。這意味著模型區分「相鄰」與「隔兩個」的能力下降——短距離的解析度被犧牲了。實務上需要少量微調來讓模型適應新的尺度。

► NTK 感知縮放

這個方法的洞見是：不該對所有頻率一視同仁。高頻維度在訓練中已經轉了幾百圈，它們處理的是短距離關係，不需要縮放；只有低頻維度需要延長。

做法是調高 base，這會讓所有頻率都變慢，但低頻受影響更大。理論上的建議值是：

$$base_new = base \times s^{(d/(d-2))}$$

以 $s = 8$ 、 $d = 128$ 計算，base 從 10,000 變成約 82,685 (8.3 倍)。看它對各頻率的影響：

維度 j	原週期	NTK 後週期	變化
0	6.3	6.3	幾乎不變
16	62.8	106.5	1.7 倍
32	628.3	1,806.7	2.9 倍
63	54,410	435,281	8.0 倍

這正是設計目標：高頻幾乎不動（保留短距離解析度），低頻延長八倍（支援長距離）。這比位置內插的「全部除以 8」精細得多。

► YaRN

YaRN 進一步把頻率分成三段：完全不縮放的高頻、完全內插的低頻、以及中間的漸變區。此外它還調整注意力的溫度，補償因為序列變長而導致的注意力熵上升。

這三種方法的關係是遞進的：位置內插是最粗糙但最簡單的，NTK 加入了頻率的區分，YaRN 再加上熵的補償。實務上的選擇取決於你能投入多少微調成本。

20.2.3 外推需要微調

一個常見的誤解是「換個位置編碼設定就能支援長上下文」。實際上三種方法都需要在目標長度上做一些訓練，讓模型適應新的位置分布。

方法	所需微調量	效果	風險
完全不微調	0	通常明顯劣化	短上下文能力也可能受損
少量微調	數百至數千步	多數情況可接受	需要長文本語料
充分微調	數萬步	最佳	成本高

第二列的「需要長文本語料」是臺灣情境的一個實際困難。要微調到 32K 上下文，你需要大量長度接近 32K 的繁體中文文本——而那類語料比一般文本更稀缺。第 9 章的語料工作在這裡會直接影響你能做到什麼。

一個必做的回歸測試

延長上下文之後，務必回頭測試短上下文的表現。位置內插會壓縮短距離的解析度，可能讓模型在原本擅長的短文本任務上退步——而那個退步很容易被「我們現在支援 32K 了」的興奮掩蓋。第 22 章的適配回歸測試在這裡同樣適用。

20.3 稀疏與局部注意力

20.3.1 滑動視窗

最簡單的稀疏化：每個位置只注意鄰近的 W 個位置。複雜度從 $O(T^2)$ 降到 $O(T \cdot W)$ 。

序列長度 T	全域注意力	視窗 $W=512$	節省
4,096	1.68×10^7	2.10×10^6	8 倍

序列長度 T	全域注意力	視窗 $W=512$	節省
32,768	1.07×10^9	1.68×10^7	64 倍
131,072	1.72×10^{10}	6.71×10^7	256 倍

節省倍數是 T/W ，所以序列越長效益越大。但代價是每個位置只能直接看到鄰近的 W 個。

這個限制有一個緩解機制：多層疊加會擴大有效感受野。第一層看 W 個位置、第二層的每個位置又看了 W 個「已經看過 W 個位置」的位置——所以 L 層之後的有效感受野約為 $L \times W$ 。以 $W = 512$ 、 $L = 32$ 計算，理論上可達 16,384。

但這只是理論上的路徑存在，不代表資訊真的有效傳遞。第 11.1.1 節說過，注意力的優勢在於 $O(1)$ 的路徑長度；滑動視窗把它變回 $O(T/W)$ ，資訊在多層傳遞中會衰減。這是一個真實的能力損失。

20.3.2 混合注意力

既然全域注意力貴、局部注意力有損，一個自然的折衷是：部分層用全域、部分層用局部。

配置	做法	效益	取捨
全部全域	每層都是完整注意力	能力最強	最貴
全部局部	每層都是滑動視窗	最便宜	長距離能力弱
交錯	每 N 層一個全域層	折衷	需調整 N
前局部後全域	淺層局部、深層全域	符合資訊聚合的直覺	較少見

第三列是目前較常見的做法。直覺是：多數的語言關係是局部的（詞與詞、句與句），只有少數需要跨越整份文件——所以不需要每一層都付全域的代價。

20.3.3 其他稀疏模式

模式	做法	動機	限制
滑動視窗	只看鄰近 W 個	局部關係最重要	長距離弱
全域詞元	少數位置被所有位置注意	提供資訊匯集點	需選定哪些是全域

模式	做法	動機	限制
擴張視窗	間隔取樣（如每 2 個取 1）	用同樣成本看更遠	解析度下降
區塊稀疏	把序列分塊，塊間稀疏連接	對硬體友善	模式需設計
學習式稀疏	讓模型自己決定看哪裡	最有彈性	訓練困難、硬體不友善

第二列的「全域詞元」是一個實用的技巧：保留少數幾個位置（例如序列開頭或特殊標記）讓所有位置都能看到，作為資訊的匯集與分發點。這與第 11 章觀察到的「注意力沉沒」現象有關——模型本來就傾向於把注意力放在開頭，全域詞元等於把這個行為制度化。

20.3.4 KV cache 的縮減

第 14.5.4 節算過 KV cache 的大小。滑動視窗在這裡有一個額外的好處：你只需要保留視窗內的 KV。

序列長度	全域 KV cache	視窗 4,096	節省
4,096	0.54 GB	0.54 GB	無
32,768	4.29 GB	0.54 GB	8 倍
131,072	17.18 GB	0.54 GB	32 倍

（ $L = 32$ 、 $H_{kv} = 8$ 、 $d_{head} = 128$ 、BF16、批次 1。）

這個性質對部署極為重要：使用滑動視窗時，KV cache 的大小有上界，不隨對話長度無限增長。這讓長時間的多輪對話變得可行——而那正是第 33 章的應用場景。

代價當然是模型會「忘記」視窗之外的內容。實務上的折衷是：保留少數的全域詞元（例如系統提示與最初幾輪）加上滑動視窗，讓最重要的資訊不被丟棄。

20.3.5 注意力沉沒與串流處理

第 11 章觀察到的「注意力沉沒」現象（大量注意力集中在第一個詞元）在長上下文中有一個實用的後果。

如果你用滑動視窗做串流處理——也就是不斷丟棄最舊的 KV、保留最近的視窗——會發生一件奇怪的事：當最初的幾個詞元被丟棄時，模型的輸出品質會突然崩壞。

原因就是注意力沉沒。模型習慣把「無處可去」的注意力放在開頭幾個位置；一旦它們消失，那些注

意力被迫重新分配到其他位置，擾亂了整個分布。

做法	效果	成本	說明
純滑動視窗	丟棄開頭後品質崩壞	低	不可行
保留開頭數個詞元	品質穩定	幾乎為零	常見的做法
重新計算位置編碼	同上，但更正確	略高	需重算相對位置

第二列是一個成本極低、效果顯著的技巧：在滑動視窗之外，額外永久保留最初的四到八個詞元。它們占的記憶體微不足道，卻能讓串流處理穩定運作。

這是一個很好的例子，說明為什麼要理解模型的內部行為：如果你不知道注意力沉沒的存在，遇到這個崩壞會完全不知道從何查起——而解法只是「多留四個詞元」。

含沉沒詞元的滑動視窗快取

```
class SlidingKVCache:
    """滑動視窗 + 保留開頭的沉沒詞元"""

    def __init__(self, n_layers, window=4096, n_sink=4, **kw):
        self.window = window
        self.n_sink = n_sink
        self.k = [[] for _ in range(n_layers)]
        self.v = [[] for _ in range(n_layers)]

    def update(self, layer, k_new, v_new):
        self.k[layer].append(k_new)
        self.v[layer].append(v_new)
        total = sum(t.size(2) for t in self.k[layer])
        # 超出上限時，丟棄中段但保留最前面的 n_sink 個
        if total > self.window + self.n_sink:
            self._evict(layer)
        return (torch.cat(self.k[layer], dim=2),
                torch.cat(self.v[layer], dim=2))

    def _evict(self, layer):
        full_k = torch.cat(self.k[layer], dim=2)
        full_v = torch.cat(self.v[layer], dim=2)
        sink_k, sink_v = full_k[:, :, :self.n_sink], full_v[:, :, :self.n_sink]
        recent_k = full_k[:, :, -self.window:]
        recent_v = full_v[:, :, -self.window:]
        self.k[layer] = [torch.cat([sink_k, recent_k], dim=2)]
        self.v[layer] = [torch.cat([sink_v, recent_v], dim=2)]
```

20.4 混合專家 (MoE)

20.4.1 核心想法

密集模型的每個詞元都要經過全部的參數。MoE 的想法是：把前饋網路換成多個「專家」，每個詞元只經過其中少數幾個。

元件	密集模型	MoE	差別
前饋網路	一組 W_1 、 W_2 、 W_3	E 組 (E 個專家)	參數 $\times$ E
路由器	無	一個小型網路決定用哪些專家	額外的參數與計算
每詞元的計算	全部參數	top-k 個專家	計算量約 k/E

這帶來一個關鍵的性質：總參數量與活躍參數量分離。模型可以有非常多的參數（記憶容量大），但每次前向只用其中一小部分（計算量小）。

20.4.2 參數量的計算

以一個 $d = 4096$ 、 $d_{ff} = 11008$ 、 $L = 32$ 、 $V = 128000$ 的配置為例（相當於 7 B 密集模型），每兩層放一個 MoE 層：

配置	總參數量	活躍參數量	比值
密集 (基準)	7.00 B	7.00 B	1.0×
MoE E=8, top-2	22.2 B	9.16 B	2.4×
MoE E=64, top-2	143.4 B	9.16 B	15.6×
MoE E=128, top-8	281.9 B	22.15 B	12.7×

看第三列：一個總參數 143 B 的模型，每次前向只用 9.16 B——計算成本接近一個 9 B 的密集模型，但知識容量遠大於它。這就是 MoE 的吸引力。

但要注意活躍參數量仍高於密集基準（9.16 vs 7.00）。原因是路由器本身有參數，且 top-2 意味著要算兩個專家而非一個前饋網路。所以 MoE 不是「免費的午餐」——它是用記憶體換計算。

20.4.3 路由與負載平衡

MoE 最麻煩的問題是：路由器可能把大部分詞元都送到少數幾個專家。

問題	成因	後果	對策
專家坍縮	少數專家被過度使用	其他專家學不到東西	負載平衡輔助損失
卡間負載不均	專家分散在不同卡上	部分卡閒置、部分過載	容量上限與丟棄
訓練不穩	路由決策是離散的	梯度訊號不連續	軟性路由、輔助損失
推論時的變動	不同輸入用不同專家	延遲不可預測	容量規劃需保守

第一列的處置是加入一個輔助損失，鼓勵路由器均勻地分配詞元。這個損失有一個權重，而那個權重需要調——太小則不起作用，太大則干擾主要的學習目標。這是 MoE 訓練比密集模型難的主要原因之一。

第二列的「容量上限」是一個工程妥協：每個專家設定一個最多能處理的詞元數，超過的部分直接丟棄（那些詞元只走殘差路徑）。這保證了負載可預測，代價是部分詞元沒有經過前饋網路。

20.4.4 MoE 適合誰

情境	適合 MoE 嗎	理由	章節
記憶體充足、要最大知識量	適合	總參數大是優勢	本節
邊緣部署	不適合	總參數仍需載入記憶體	第 26 章
單卡推論	通常不適合	所有專家都要放進記憶體	第 35 章
大規模訓練	適合	算力效率高	第 19.5.3 節
資源受限的實驗室	不建議	訓練複雜度高、收益不確定	本節

對本書多數讀者的建議很明確：MoE 值得理解，但不建議自己從零訓練。它的訓練難度、超參數敏感度與工程複雜度都遠高於密集模型，而在資源受限的情況下，一個訓練良好的密集小模型通常更實用。

但你可能會使用別人訓練的 MoE 模型——那時候理解它的記憶體特性（總參數要載入、活躍參數決定速度）就很重要。第 35 章的部署會回到這個議題。

20.4.5 MoE 的推論特性

訓練 MoE 困難，但使用別人訓練好的 MoE 模型是很可能發生的事。這時候需要理解它的推論特性。

面向	密集模型	MoE	實務意涵
記憶體需求	正比於總參數	同樣正比於總參數	MoE 需要更多記憶體
每詞元計算	全部參數	只有活躍參數	MoE 較快
批次的效益	穩定	不同詞元用不同專家	批次大時所有專家都會被用到
延遲的可預測性	高	較低	路由決策依輸入而變
量化的難度	標準	較高	專家間的分布差異大

第一列是最關鍵的限制：MoE 省的是計算，不是記憶體。一個總參數 143 B 的 MoE 模型，即使活躍參數只有 9 B，你仍然需要把 143 B 全部載入——因為你不知道下一個詞元會路由到哪些專家。

這使 MoE 不適合記憶體受限的部署（第 26 章的邊緣裝置）。它適合的是「記憶體充足但想省計算」的場景——例如有大量記憶體的伺服器要服務高併發。

第三列有一個有趣的後果：批次為 1 時只有 top-k 個專家被用到，但批次為 64 時，不同詞元可能路由到不同專家，結果幾乎所有專家都被啟用。這意味著 MoE 在小批次時的計算優勢最明顯，大批次時優勢減弱——與第 14.6.2 節談的批次效益方向相反。

20.5 狀態空間模型與線性注意力

20.5.1 回到遞迴

第 2 章講過 RNN 被 Transformer 取代的過程。但近年有一批研究試圖把遞迴的優點找回來——因為遞迴有一個注意力永遠沒有的性質：推論時的記憶體是固定的。

面向	注意力	遞迴 / 狀態空間	意涵
訓練時的平行度	高（全序列平行）	需特殊設計才能平行	注意力較易訓練
推論記憶體	隨長度線性成長（KV cache）	固定（只有狀態）	遞迴適合長對話
推論計算	每步與已生成長度成正比	每步固定	遞迴適合長生成

面向	注意力	遞迴 / 狀態空間	意涵
長距離檢索	直接連結， $O(1)$ 路徑	需經過狀態壓縮	注意力較強
資訊保留	完整保留所有位置	狀態是壓縮的	注意力較強

第二與第三列是遞迴的決定性優勢，第四與第五列是它的決定性劣勢。這個取捨可以用一句話總結：注意力保留一切但成本隨長度增長；遞迴成本固定但必須壓縮。

20.5.2 現代狀態空間模型的關鍵改進

傳統 RNN 無法平行訓練（第 2.4.5 節），這是它被淘汰的主因。近年的狀態空間模型解決了這個問題。

問題	傳統 RNN	現代 SSM	如何做到
訓練平行度	無	有	把遞迴改寫成可平行的形式
長距離記憶	梯度消失	較好	特殊的狀態轉移設計
選擇性	固定的轉移	依輸入調整	讓狀態轉移與輸入相關

第三列的「選擇性」是關鍵改進。傳統 RNN 的狀態更新方式對所有輸入都一樣，而現代 SSM 讓模型能依據輸入決定「這個資訊該記多久」——這在功能上接近注意力的選擇能力，但保留了固定記憶體體的優勢。

20.5.3 混合架構

目前較被看好的方向不是取代注意力，而是混用。

配置	做法	效益	適用
純注意力	所有層都是注意力	能力最強	中短上下文
純 SSM	所有層都是狀態空間	推論成本固定	極長生成
混合	多數層 SSM、少數層注意力	兼顧兩者	長上下文應用

混合的直覺與 20.3.2 節的混合注意力相同：多數的處理是局部與順序的（SSM 擅長），少數需要精

確的長距離檢索（注意力擅長）。少數幾層注意力就足以提供「大海撈針」的能力，而其餘的層用 SSM 省下成本。

本章對這個方向的態度

狀態空間模型是一個活躍且結論尚未穩定的方向。本書的建議是：理解它的取捨（固定記憶體 vs 壓縮損失），知道混合架構是目前的主流思路，但不要在教學或專題中把它當成已驗證的選擇。第 22 章之後你會用到的開放權重模型，絕大多數仍是純注意力架構。

20.6 長上下文的評測

20.6.1 為什麼需要專門的評測

一般的評測（第 5、36 章）用的是短文本，無法檢驗長上下文能力。而長上下文有一些特有的失效模式，必須專門設計測試。

失效模式	症狀	一般評測抓得到嗎	需要的測試
中段遺失	只記得開頭與結尾	否	分位置的大海撈針
注意力稀釋	長度增加時整體品質下降	否	同任務不同長度的對照
位置外推失敗	超過某長度後輸出崩壞	否	長度階梯測試
多跳失敗	需要串連多處資訊時失敗	否	多針測試

20.6.2 大海撈針測試

基本設計：在一份長文件的位置 p 埋入一句特定資訊，然後詢問模型。改變 p 與文件長度 L ，畫出二維的成功率熱圖。

ch20/needle_test.py (節錄)

```
import random

NEEDLE_TEMPLATE = "特別注意：{fact}"
QUESTION_TEMPLATE = "根據上文，{query}請只回答關鍵資訊，不要解釋。"

# 臺灣情境的針：使用不容易從常識猜到的具體資訊
NEEDLES = [
    {"fact": "本校 115 學年度的畢業門檻為 128 學分",
     "query": "本校 115 學年度的畢業門檻是幾學分？",
     "answer": "128"},
    {"fact": "雲林廠的三號產線最大進給速度為每分鐘 4200 毫米",
```

```

    "query": "雲林廠三號產線的最大進給速度是多少?",
    "answer": "4200"},
{"fact": "本要點自民國 115 年 3 月 1 日起生效",
 "query": "本要點自何時起生效?",
 "answer": "115 年 3 月 1 日"},
]

def build_haystack(filler_docs, target_chars, needle, position_ratio,
                  seed=0):
    """在指定的相對位置埋入針"""
    rng = random.Random(seed)
    text, used = [], 0
    pool = list(filler_docs)
    rng.shuffle(pool)
    for d in pool:
        if used >= target_chars:
            break
        text.append(d)
        used += len(d)

    body = "\n\n".join(text)[:target_chars]
    insert_at = int(len(body) * position_ratio)
    # 對齊到段落邊界，避免把針插進句子中間
    insert_at = body.rfind("\n\n", 0, insert_at) + 2 if insert_at > 0 else 0
    sentence = NEEDLE_TEMPLATE.format(fact=needle["fact"])
    return body[:insert_at] + sentence + "\n\n" + body[insert_at:]

def run_grid(model, tok, filler, lengths, positions, needles,
            n_repeat=3):
    """產生二維熱圖所需的完整結果"""
    rows = []
    for L in lengths:
        for p in positions:
            hits = 0
            total = 0
            for needle in needles:
                for r in range(n_repeat):
                    ctx = build_haystack(filler, L, needle, p, seed=r)
                    q = QUESTION_TEMPLATE.format(query=needle["query"])
                    out = generate(model, tok, ctx + "\n\n" + q,
                                temperature=0.0, max_new_tokens=48)
                    hits += needle["answer"] in out
                    total += 1
            rows.append({"length": L, "position": p,
                        "accuracy": hits / total, "n": total})
    return rows

```

兩個設計細節值得說明。第一，針的內容必須是「不能從常識猜到」的——如果你問「臺灣的首都是哪裡」，模型不看上下文也答得出來。第二，插入位置要對齊段落邊界，否則針會被切在句子中間，測的就變成「模型能不能處理破碎的句子」。

20.6.3 多針與多跳測試

單針測試只檢驗「能不能找到一個東西」。實際應用往往需要串連多處資訊。

測試類型	設計	檢驗什麼	難度
單針	埋一句・問那句	基本檢索	低
多針	埋數句・全部問	同時記住多處	中
多跳	埋 $A \rightarrow B$ 、 $B \rightarrow C$ ・問 $A \rightarrow C$	串連推理	高
干擾針	埋相似但錯誤的資訊	能否分辨	高

第四列的「干擾針」對臺灣情境特別有意義：法規常有相似的條文（例如不同年度的版本、不同學制的規定），模型必須能分辨「115 學年度」與「114 學年度」的差別。這是一個實際會出錯的場景，而單針測試抓不到。

20.6.5 長度衰減的量化

大海撈針給的是二維熱圖，但決策時常常需要一個更簡單的數字：「這個模型在什麼長度以內是可靠的」。

ch20/decay_analysis.py

```
def find_reliable_length(results, threshold=0.9, min_positions=0.8):
    """找出「多數位置仍達到門檻」的最大長度"""
    by_length = {}
    for r in results:
        by_length.setdefault(r["length"], []).append(r["accuracy"])

    reliable = None
    for L in sorted(by_length):
        accs = by_length[L]
        ok_ratio = sum(a >= threshold for a in accs) / len(accs)
        if ok_ratio >= min_positions:
            reliable = L
        else:
            break
    # 一旦跌破就停止，不看更長的
    return {"reliable_length": reliable,
            "threshold": threshold,
            "min_position_ratio": min_positions}

def positional_profile(results, length):
    """某個長度下，各相對位置的準確率——看中段遺失"""
    rows = [r for r in results if r["length"] == length]
    rows.sort(key=lambda r: r["position"])
    head = [r["accuracy"] for r in rows if r["position"] <= 0.2]
    mid = [r["accuracy"] for r in rows if 0.3 <= r["position"] <= 0.7]
    tail = [r["accuracy"] for r in rows if r["position"] >= 0.8]
    avg = lambda xs: round(sum(xs) / len(xs), 3) if xs else None
    return {"head": avg(head), "middle": avg(mid), "tail": avg(tail),
            "mid_penalty": (round(avg(head) - avg(mid), 3)
                            if head and mid else None)}
```

mid_penalty 這個指標把「中段遺失」變成一個數字。如果它是 0.05，那中段遺失不嚴重；如果是 0.35，那你的長文件應用必須把重要資訊放在開頭或結尾——這是一個可以直接轉成提示設計建議的發現（第 29 章會用到）。

find_reliable_length 中「一旦跌破就停止」的設計是刻意的：長度衰減通常是單調的，如果 32K 不可靠而 64K 又「可靠」，那多半是量測雜訊而非真的回升。

20.6.4 該報告什麼

- 二維熱圖：長度 × 位置的成功率，而非單一的平均值。
- 分位置的統計：開頭、中段、結尾各自的準確率——中段通常最差。
- 長度衰減曲線：固定位置，看準確率隨長度的變化。
- 與短上下文的對照：同樣的問題在短文本中的準確率是多少（作為上界）。
- 延遲與記憶體：不同長度下的推論成本，讓讀者判斷實用性。
- 樣本數與信賴區間：依第 16 章的要求，不可只報單次結果。

第四項的對照特別重要。如果模型在短文本上的準確率就只有 0.85，那麼長文本的 0.70 就不能全歸咎於長度——你必須把「任務本身的難度」與「長度造成的衰減」分開。這是第 16.1.2 節控制變數原則的應用。

20.7 架構選型的決策框架

20.7.1 三個層次的問題

層次	問題	誰能決定	本書章節
要不要自己訓練	從零、適配、還是直接用	專案負責人	第 1.4 節
選什麼底模	規模、架構、授權	技術負責人	第 22 章
要不要改架構	長上下文、MoE、混合	本章	20.7.2

第一列與第二列的重要性遠高於第三列。對絕大多數專案而言，架構的選擇是由底模決定的——你選了某個開放權重模型，就繼承了它的架構。真正需要做架構決策的，只有從零訓練的專案。

這也是為什麼本章的態度偏向「理解而非實作」：你更需要判斷「這個底模的架構適不適合我的

應用」，而不是自己設計一個新架構。

20.7.2 一張選型對照表

你的需求	關鍵指標	建議	注意
短上下文、要最強	一般評測	標準密集架構	不需要本章的技巧
長文件、可切塊	檢索品質	RAG 加標準架構	第 30 章
長文件、不可切塊	大海撈針結果	長上下文模型	務必實測而非看宣稱
長對話、記憶體受限	KV cache 大小	滑動視窗或 GQA	20.3.4
要最大知識量	總參數量	MoE	記憶體需求高
邊緣部署	總參數與延遲	小密集模型	第 26 章

第三列的「務必實測」是本章最實際的建議。模型卡上寫的上下文長度是一個技術規格，不是能力保證。用你自己的資料跑一次大海撈針，那個熱圖比任何宣稱都可靠。

20.7.3 繁體中文的額外考量

考量	為什麼特殊	影響	章節
token 稅	同樣的字數需要更多詞元	有效上下文更短	第 7.7.3 節
長文本語料稀缺	微調長上下文需要長文本	難以做充分的適配	第 9 章
公文與法規的結構	有明確的條號與層級	可利用結構切塊	第 6.5.2 節
相似條文的干擾	不同年度、學制的版本	干擾針測試特別重要	20.6.3

第一列的影響常被低估。一個宣稱 32K 上下文的模型，若 fertility 是 1.4，實際能放入的繁中內容約 22,800 字（第 7.7.3 節算過）。如果你的文件是 3 萬字，它其實放不下——而那個限制在英文使用者身上不存在。

第三列則是一個機會。臺灣的公文與法規有高度結構化的格式（第六章處理過），這讓「依結構切塊」

比「依固定長度切塊」有效得多——第 30 章的 RAG 會利用這一點。有時候好的切塊策略比長上下文更實用，也便宜得多。

程式實作

本章三個實作的產出一份架構評估報告。它們不要求你訓練新模型，而是要求你有能力評估既有模型的長上下文與架構特性。

程式 20A 位置編碼延長實驗

目標：實作位置內插與 NTK 感知縮放，量測它們對短距離與長距離的影響。

formosa/model/rope_scaling.py

```
import torch

def build_rope_cache_scaled(seq_len, head_dim, base=10000.0,
                           method="none", scale=1.0, device=None):
    """支援三種延長方法的 RoPE 快取"""
    half = head_dim // 2
    idx = torch.arange(0, half, device=device).float()

    if method == "ntk":
        # 調高 base: base * s^(d/(d-2))
        d = head_dim
        base = base * (scale ** (d / (d - 2)))
        inv_freq = base ** (-idx / half)

    t = torch.arange(seq_len, device=device).float()
    if method == "linear":
        t = t / scale  # 位置內插

    freqs = torch.outer(t, inv_freq)
    return freqs.cos(), freqs.sin()

def measure_resolution(cos, sin, gaps=(1, 2, 8, 64, 512)):
    """量測不同相對距離下的內積差異—解析度的代理指標"""
    import torch
    g = torch.Generator().manual_seed(0)
    dh = cos.shape[-1] * 2
    q = torch.randn(dh, generator=g)
    k = torch.randn(dh, generator=g)

    def rot(v, pos):
        half = dh // 2
        v1, v2 = v[:half], v[half:]
        c, s = cos[pos], sin[pos]
        return torch.cat([v1 * c - v2 * s, v1 * s + v2 * c])

    base_pos = 100
    results = {}
    for gap in gaps:
        ip = float(rot(q, base_pos + gap) @ rot(k, base_pos))
        results[gap] = round(ip, 4)
    # 相鄰距離的區辨度: gap=1 與 gap=2 的差異
    results["discriminability"] = round(
        abs(results[1] - results[2]), 4)
    return results
```

必須完成的四項：

- 對三種方法（無、linear、ntk）各建立快取，比較 $\text{gap} = 1$ 與 $\text{gap} = 2$ 的內積差異。位置內插的區辨度應明顯下降。
- 驗證 NTK 方法對高頻維度的影響確實較小（用 20.2.2 節的週期表對照）。
- 畫出「內積對相對距離」的曲線，比較三種方法在短距離與長距離的行為。
- 若你有可用的模型，實測延長前後的短文本困惑度變化——這是 20.2.3 節說的回歸測試。

程式 20B 長上下文評測套件

目標：建立臺灣情境的大海撈針測試，產出二維熱圖與分位置統計。

要素	設計要求	驗收標準	節次
填充文本	真實的繁中長文件	至少能組出目標長度	第 9 章的語料
針	不可從常識推得	至少 5 種、涵蓋數字與日期	20.6.2
位置	至少 7 個相對位置	含 0%、50%、100%	20.6.2
長度	至少 5 個長度階梯	從短到模型上限	20.6.2
重複	每格至少 3 次	報告信賴區間	第 16.2 節
對照	短文本的同題準確率	作為上界參照	20.6.4

延伸挑戰：加入干擾針。埋入「114 學年度的畢業門檻為 132 學分」作為干擾，同時埋入正確的「115 學年度為 128 學分」，然後問 115 學年度。統計模型答錯成 132 的比例——那就是它被干擾的程度。

一個容易被忽略的變因

大海撈針的結果會受填充文本的內容影響。如果填充的是與針高度相關的內容（例如都在談學分），模型較難分辨；如果完全無關（例如填充是食譜），則較容易。所以報告時必須說明填充文本的性質，否則不同研究的結果無法比較。

程式 20C 架構特性比較

目標：對至少兩個可取得的模型，量測它們的長上下文特性與資源需求，產出一份選型報告。

量測項	方法	意義	節次
宣稱上下文	讀模型卡	規格	20.1.2
實測上下文	大海撈針的衰減點	實際可用長度	20.6.2
繁中有效長度	除以 fertility	真正能放的字數	第 7.7.3 節
KV cache 大小	依 L、H_kv、d_head 計算	併發能力	第 14.5.4 節
總參數 vs 活躍	讀設定或量測	記憶體與速度	20.4.2
延遲隨長度	實測不同長度的生成時間	使用體驗	第 14.6 節

第三列是本書一貫強調的：把英文的規格換算成繁中的實際能力。一個 32K 的模型對繁中使用者可能只有 22K 的有效容量，而那個差距應該出現在你的選型報告中。

系統案例：法規彙編的長上下文評估

本章的系統案例是一個實際的技術決策：某單位要建置法規問答系統，需要判斷該用長上下文、RAG、還是兩者組合。

情境設定

條件	數值	意涵	章節
法規彙編總長	約 80 萬字	遠超任何模型的上下文	需要切塊或檢索
單一法規長度	3 千至 5 萬字	部分可整份放入	長上下文有機會
查詢類型	單條查詢、跨條比對、修訂差異	需求不同	20.7.2
硬體	單張 24 GB GPU	KV cache 受限	第 14.5.4 節
併發需求	同時 10 人	記憶體乘以 10	第 35 章

要回答的四個問題

1. 單條查詢：整份法規放得進上下文嗎？繁中的實際容量是多少？
2. 跨條比對：需要同時看到多條時，長上下文與 RAG 哪個更可靠？用干擾針測試佐證。

3. 併發限制：在 24 GB 的卡上，用你選定的上下文長度，能支援幾個並行請求？
4. 成本比較：長上下文方案與 RAG 方案的每次查詢成本各是多少？

第二個問題是這個案例的核心。法規有大量相似條文（不同年度、不同適用對象），而 RAG 的檢索可能取回錯誤的版本，長上下文則可能在中段遺失。兩者的失效模式不同，必須分別測試。

交付要求

- 長上下文實測：大海撈針的二維熱圖，含干擾針的結果。
- 繁中有效容量換算：宣稱長度、fertility、實際可放字數。
- RAG 對照：同樣的查詢用檢索方案的準確率（可用第 10 章的檢索原型）。
- 併發計算：KV cache 大小與可支援的請求數。
- 成本比較表：兩種方案的每次查詢成本與延遲。
- 建議方案：明確的選擇與理由，含在什麼條件下該改變決定。
- 限制說明：你的評測沒有涵蓋什麼、結論適用到哪裡。

評分重點

- 是否實測而非引用模型卡的宣稱。
- 是否做了繁中的容量換算——只用詞元數討論者扣分。
- 干擾針測試是否包含，以及結果的解讀。
- 併發計算是否正確，特別是 KV cache 隨請求數的成長。
- 是否公平評估了 RAG 方案。只論證長上下文者，本案例不予計分。

最後一項延續第 17 章系統案例的精神：一份誠實的技術評估必須認真對待你不打算採用的方案。很多情況下，RAG 加上好的切塊策略會勝過長上下文——而能算出這一點並接受它，比堅持使用比較新的技術更專業。

章末評量

一、概念題

1. 長上下文有哪三重瓶頸？為什麼解決其中一項不代表解決其他兩項？
2. 為什麼 RoPE 在超出訓練長度後會失效？請用頻率分量的週期說明。
3. 比較位置內插與 NTK 感知縮放，說明後者為什麼對短距離的影響較小。
4. 滑動視窗的有效感受野為什麼是 $L \times W$ ？為什麼這不等同於全域注意力的能力？
5. 滑動視窗對 KV cache 有什麼特殊好處？這對長對話應用有什麼意義？
6. 解釋 MoE 的總參數量與活躍參數量，並說明為什麼 MoE 不是「免費的午餐」。
7. MoE 的負載平衡問題是什麼？容量上限的做法有什麼代價？
8. 狀態空間模型相對於注意力的兩個決定性優勢與兩個決定性劣勢各是什麼？
9. 為什麼「支援 128K」不等於「在 128K 上能用」？該用什麼方法檢驗？

二、計算題

1. 用 NTK 公式計算：head_dim = 128、原 base = 10000，要擴展 4 倍時新的 base 是多少？
2. $T = 65,536$ 、滑動視窗 $W = 1,024$ 。與全域注意力相比，注意力的計算量節省幾倍？
3. $L = 40$ 、 $H_{kv} = 8$ 、 $d_{head} = 128$ 、BF16。計算 $T = 65,536$ 的全域 KV cache，以及視窗 4,096 的大小。
4. 一個 $d = 4096$ 、 $d_{ff} = 11008$ 的層改成 MoE， $E = 32$ 、top-2。單層的總參數與活躍參數各是多少（不含注意力）？
5. 某模型宣稱 32K 上下文，繁中 fertility 為 1.5。實際能放入多少繁中字？若文件有 30,000 字，放得下嗎？
6. 承第 3 題，在 24 GB 的卡上（模型權重占 14 GB），用全域 KV cache 最多能支援幾個 $T = 8,192$ 的並行請求？

三、除錯題

1. 同學把模型的 max_seq_len 從 4096 改成 32768，但沒有任何其他調整。請說明會發生什麼與為什麼。
2. 同學做了位置內插並延長到 32K，長文本表現不錯，但短文本的困惑度上升了 0.3。請說明原因與處置。
3. 大海撈針測試顯示所有位置的準確率都是 1.0，包括 128K 的位置。請列出三個應該懷疑的地方。

4. 同學的 MoE 訓練中，八個專家裡有兩個處理了七成的詞元。請說明後果與三種對策。
5. 滑動視窗模型在需要跨越整份文件的任務上表現很差，但在摘要任務上正常。請解釋這個現象。
6. 同學比較兩個模型的長上下文能力，用了不同的填充文本。請說明為什麼這個比較無效。

四、系統設計題

1. 你要為一個「契約條款一致性檢查」的應用選擇技術方案。契約長度 2 萬至 10 萬字，需要偵測前後條款的矛盾。請比較長上下文、RAG、與分段加彙整三種方案，並說明你的選擇與驗證方式。
2. 你的團隊要評估三個候選底模的長上下文能力，但算力只夠做有限的測試。請設計一個「最小可行的評測方案」：測哪些長度、哪些位置、幾次重複，以及你如何確保結論可信。

五、臺灣情境與倫理題

1. 本章 20.7.3 節指出，繁中使用者的有效上下文比宣稱值短。請討論：模型提供者是否應該揭露不同語言的實際容量？這是技術問題還是資訊揭露問題？
2. 一個宣稱支援長上下文但實際上中段遺失嚴重的模型，若被用於法規查詢，可能造成什麼後果？請討論：技術人員在採購或推薦時有什麼責任？

評量提示 (給自學者)

- 計算題第 1 題： $10000 \times 4^{(128/126)} \approx 10000 \times 4.089 \approx 40,890$ 。
- 計算題第 2 題：全域 $65536^2 \approx 4.29 \times 10^9$ ；視窗 $65536 \times 1024 \approx 6.71 \times 10^7$ ；節省 64 倍 (即 T/W)。
- 計算題第 3 題：全域 = $2 \times 65536 \times 40 \times 8 \times 128 \times 2 \approx 10.74 \text{ GB}$ ；視窗 4096 為其 1/16，約 0.67 GB。
- 計算題第 4 題：總參數 = $32 \times 3 \times 4096 \times 11008 \approx 43.3 \text{ 億}$ ；活躍 = $2 \times 3 \times 4096 \times 11008 \approx 2.7 \text{ 億}$ 。
- 計算題第 5 題： $32000 \div 1.5 \approx 21,333 \text{ 字}$ ；30,000 字放不下。
- 計算題第 6 題：可用 10 GB；每請求 T=8192 的 KV cache = $2 \times 8192 \times 40 \times 8 \times 128 \times 2 \approx 1.34 \text{ GB}$ ；約 7 個請求。
- 除錯題第 3 題：填充文本可能與針高度可分辨 (太容易)、針的答案可能從常識推得、或評分方式過於寬鬆 (例如只要輸出包含數字就算對)。

研究延伸與版本注意事項

- 位置內插與 NTK 縮放的原始提案：兩者都不長，數學推導清楚，是 20.2 節的直接來源。
- YaRN 論文：對前兩者的整合與改進，並加入了注意力溫度的補償。
- 長上下文評測的相關工作：大海撈針測試與「中段遺失」現象的實證研究，是 20.6 節的基礎。
- MoE 的路由與負載平衡：這是一個技術細節多、實作困難的領域，若你要使用 MoE 模型值得深入。
- 狀態空間模型與混合架構：這是本章最快變動的部分，結論尚未穩定，引用時務必註明時間。
- 本章介紹的技術有相當比例在近幾年才出現。本書的建議是：把重點放在「每個方法解決什麼瓶頸」的分析框架，而非記住特定方法的細節——後者的半衰期可能只有一兩年。

常見問題

問：我該用長上下文還是 RAG？答：先問「你能不能事先知道哪一段相關」。如果能（多數查詢型應用），RAG 通常更好——它更便宜、更快、且可以引用出處。如果不能（一致性檢查、跨章節推理），才需要長上下文。兩者也可以組合：用 RAG 縮小範圍，再用長上下文處理縮小後的內容。

問：MoE 值得學嗎？答：值得理解，不建議自己訓練。你會遇到的情境多半是「使用別人的 MoE 模型」，那時候重要的是知道它的記憶體特性（總參數要全部載入）與速度特性（由活躍參數決定）。自己從零訓練 MoE 的難度與資源需求，通常超出本書讀者的範圍。

問：我的模型宣稱支援 128K，但實測只有 32K 可用，這正常嗎？答：相當常見。宣稱的長度通常代表「技術上不會崩潰」，而非「所有位置都同樣可靠」。這正是本章要求你自己做大海撈針測試的理由——用你自己的資料測出來的數字，比任何宣稱可靠。

問：狀態空間模型會取代 Transformer 嗎？答：目前看不出來。它在「推論記憶體固定」上有明確優勢，在「精確的長距離檢索」上有明確劣勢。目前較被看好的是混合架構。本書的建議是保持關注但不要押注——你在第六篇之後會用到的開放權重模型，絕大多數仍是純注意力架構。

教師使用建議

本章排在第 20 週，建議 2 小時講授加 1 小時實驗。本章的內容較新且變動快，教學上應強調分析框架而非技術細節。

時段	內容	互動設計	注意事項
前 20 分鐘	20.1 節，三重瓶頸	請學生指出某個技術解決了哪一項	三者的獨立性是關鍵

時段	內容	互動設計	注意事項
中間 30 分鐘	20.2 節・位置外推	現場算 NTK 對各頻率的影響	週期表的對比很清楚
後 25 分鐘	20.3 至 20.4 節・稀疏與 MoE	算一次 MoE 的總參數與活躍參數	兩者的分離要講清楚
最後 35 分鐘	20.6 至 20.7 節・評測與選型	展示一張真實的大海撈針熱圖	「中段遺失」的圖很有說服力
實驗課 1 小時	程式 20B (可用 API 模型)	各組設計自己領域的針	針的設計本身就是訓練

一個效果很好的活動：讓各組設計自己專題領域的「針」，然後互相交換測試。學生會發現設計一個好的針比想像中難——太簡單的會被常識猜到，太複雜的模型根本讀不懂。這個過程本身就在訓練評測設計的能力（第 16、36 章）。

另一個建議：如果課程有 API 額度，讓學生對一個商業模型做大海撈針測試。用真實模型測出「宣稱與實際的落差」，比任何說明都有說服力——而且那個落差通常存在。

高中授課的調整：20.1 用「讀一本很厚的書，你會記得中間第 300 頁寫什麼嗎」的比喻；20.2 略過數學，只講「模型沒見過那麼遠的位置」；20.3 用「只看鄰座 vs 看全班」的比喻講滑動視窗；20.4 用「一群專家各有專長，每個問題只找兩位」講 MoE；20.5 略過；20.6 完整教（大海撈針的設計對高中生很有啟發性，且與科展的實驗設計相通）；20.7 挑重點。程式只做 20B 的簡化版。

附：本章技術的瓶頸對照表

本章介紹了十幾種技術。把它們依「解決哪個瓶頸」整理成一張表，比逐一記憶有用得多。

技術	省計算	省記憶體	解決外推	主要代價
FlashAttention	否	是（注意力矩陣）	否	幾乎無
GQA	否	是（KV cache）	否	品質略降
位置內插	否	否	是	短距解析度
NTK 縮放	否	否	是	需調參
YaRN	否	否	是	實作複雜
滑動視窗	是	是（KV cache）	部分	長距能力

技術	省計算	省記憶體	解決外推	主要代價
稀疏注意力	是	是	否	模式設計
混合注意力	部分	部分	否	需調配比
MoE	是	否 (反而更多)	否	訓練困難
狀態空間模型	是	是 (固定狀態)	不適用	長距檢索弱

這張表可以回答一個常見的問題：「為什麼一個模型同時用了 FlashAttention、GQA 與 NTK 縮放？」——因為它們解決的是三個不同的瓶頸，彼此互補而非重複。

反過來說，如果有人告訴你「用了某個技術就能支援長上下文」，你可以用這張表檢查它到底解決了哪一項。多數情況下答案是「只有一項」，而其他兩項仍需要另外處理。

本章小結

1. 長上下文有三重獨立的瓶頸：計算 (T^2)、記憶體 (KV cache)、位置外推；解決一項不代表解決其他。
2. 「支援 128K」通常只代表不會崩潰，不代表所有位置都可靠；必須用大海撈針測試檢驗。
3. RoPE 外推失效的原因是低頻維度在訓練中未轉完一圈，超出即為模型未見過的角度組合。
4. 位置內插把所有位置除以 s ，簡單但犧牲短距離解析度；NTK 縮放調高 base，對高頻影響小。
5. NTK 的建議值為 $\text{base} \times s^{(d/(d-2))}$ ；以 $s = 8$ 、 $d = 128$ 計算，base 從 10,000 變成約 82,685。
6. 三種延長方法都需要在目標長度上微調；延長後必須做短上下文的回歸測試。
7. 滑動視窗把複雜度降到 $O(T \cdot W)$ ，節省倍數為 T/W ；多層疊加可擴大感受野但資訊會衰減。
8. 滑動視窗讓 KV cache 有上界、不隨對話長度增長，這對長對話部署極為重要。
9. MoE 分離了總參數量與活躍參數量；但活躍參數仍高於同等密集模型，它是用記憶體換計算。
10. MoE 的主要困難是負載平衡；容量上限的代價是部分詞元被丟棄。
11. MoE 值得理解但不建議自行訓練；使用他人模型時要注意總參數決定記憶體、活躍參數決定速度。
12. 狀態空間模型的優勢是推論記憶體固定，劣勢是必須壓縮資訊；混合架構是目前的主流思路。
13. 長上下文評測需要二維熱圖、分位置統計、長度衰減曲線與短文本對照，並包含干擾針。
14. 繁中的有效上下文因 token 稅而短於宣稱值；32K 配 fertility 1.5 只有約 21,000 字。

15. 架構選擇多數時候由底模決定；真正需要架構決策的只有從零訓練的專案。
16. 很多情況下 RAG 加好的切塊策略勝過長上下文，而且便宜得多——誠實評估這一點是專業的表現。

成果檢核

完成本章後你必須繳交：

- 位置編碼延長實驗（程式 20A）：三種方法的區辨度對照、頻率影響驗證、以及短文本回歸測試。
- 長上下文評測套件（程式 20B）：臺灣情境的針庫、二維熱圖、分位置統計、干擾針結果。
- 架構特性比較（程式 20C）：至少兩個模型的宣稱與實測對照，含繁中有效容量換算。
- 法規彙編評估報告：四個問題的完整回答、長上下文與 RAG 的公平比較、以及建議方案。
- KV cache 與併發計算：在你的硬體上能支援的請求數。
- 一頁反思：本章哪一個技術你認為最可能在兩年內過時？為什麼？

最後一項是刻意設計的。本章是全書技術半衰期最短的一章，而能判斷「什麼會過時、什麼不會」本身就是一種能力。我的看法是：具體的方法會變，但「這個技術解決哪個瓶頸、付出什麼代價」的分析框架不會——而那正是本章真正想教會你的東西。

第 21 章

臺灣算力生態、資源盤點與永續成本

Taiwan Compute Ecosystem, Resource Planning, and Sustainability Costs

難度：核心 | 建議授課：第 21 週 | 硬體需求：A 級 | 注意：本章含版本敏感資料，查核日 2026 年 8 月

叫獸開講

「我們沒有算力」是臺灣做大模型時最常聽到的一句話。但如果你真的坐下來把數字算清楚，往往會發現：問題不是完全沒有，而是不知道有什麼、不知道怎麼申請、以及沒有把手上這幾張卡用滿。這一章要做的，是把這三件事變清楚。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 盤點臺灣可取得的算力來源，並比較各自的取得條件與適用情境。
2. 說明國家級算力設施的申請流程與常見的審查重點。
3. 把 GPU 小時換算成電力、電費與碳排放，並使用官方公告的係數。
4. 解釋 PUE 對總能耗的影響，並判讀資料中心的能效指標。
5. 設計一份能通過審查的算力申請書。
6. 評估自建、共用、國家設施與雲端四種方案的總持有成本。
7. 為一個實驗室設計 GPU 共用制度與排程規則。
8. 在資源受限的前提下，規劃一條務實的技術路線。
9. 把環境成本納入實驗規劃，並說明什麼規模以上應該這麼做。

關於本章的資料時效

本章引用了多項具體的設施規格、電價與碳排係數。這些數字都標註了查核日期（2026 年 8 月），但它們會變動——電價每年審議兩次、碳排係數每年公告一次、設施每隔幾年更新。本章的價值在於方法與框架，不在於特定數字。使用前請依第 21.6 節的清單重新查核，並在你的報告中註明查核日。這與全書一貫的版本管理原則一致。

21.1 臺灣的算力地圖

21.1.1 五種來源

來源	典型規模	取得方式	適合誰
個人設備	1 至 2 張消費級 GPU	自購	學習與小型實驗
實驗室自建	2 至 16 張卡	計畫經費採購	持續性的研究
校級共用叢集	數十張卡	校內申請	全校研究者
國家級設施	數千張卡以上	計畫申請與審查	大型研究與產學合作
商業雲端	彈性	付費	尖峰需求與短期實驗

這張表的重要訊息是：臺灣的算力不是「有或沒有」的二元問題，而是一個分層的光譜。多數學生的專題落在第一到第三層，而那三層加起來已經足夠完成本書第一到第四篇的所有實作。

21.1.2 國家級設施的角色

國家高速網路與計算中心（國網中心）是臺灣唯一提供跨領域高速計算服務的國家實驗室，也是學研界取得大規模算力的主要管道。

以下是幾個具代表性的系統（查核日：2026 年 8 月，最新狀態請查閱官方網站）：

系統	主要規格	定位	備註
台灣杉二號	2,016 顆 V100 32GB、9 PFLOPS	AI 超級電腦	2018 年底 TOP500 第 20 名、Green500 第 10 名
晶創 26 (Nano4)	H200 架構，Rpeak 117.92 PFLOPS	通用科學模擬與 AI 訓練	2025 年 11 月首次進入 Top50
晶創 26 (GB200 NVL72)	每座 72 顆 Blackwell GPU、13.5 TB 記憶體	旗艦級 AI 運算平台	全臺首次導入

（資料來源：國網中心官方網站，查核日 2026 年 8 月。）

從台灣杉二號的 9 PFLOPS 到晶創 26 的 117.92 PFLOPS（峰值），約七年間成長了十三倍。這個成長速度值得記住——它意味著你今天覺得做不到的規模，幾年後可能變成常規，而這會影響你的技術路線規劃（第 17 章）。

21.1.3 能效：一個被低估的指標

晶創 26 的 Nano4 系統實測效能（Rmax）為 81.55 PFLOPS，實測最佳用電功率為 2.214 百萬瓦。

用這兩個數字可以算出能效：

$$\text{能效} = 81.55 \times 10^{15} \div (2.214 \times 10^6) \approx 36.8 \text{ GFLOPS/W}$$

這個數字有兩個用途。第一，它是評估「這座機器有多綠」的標準指標（Green500 排行榜用的就是它）。第二，它讓你能反推電力成本——第 21.3 節會做這個計算。

對臺灣而言能效特別重要，原因很直接：電力供應是本地的實質限制。一座 2.2 MW 的超級電腦持續運轉一年約需 1,900 萬度電，而那是一個必須被規劃的量。第 21.3.4 節會回到這個議題。

21.1.4 規模的實感：把數字翻譯成經驗

超級電腦的規格數字（PFLOPS、MW）對多數人沒有實感。用第 17 章的公式把它們翻譯成「能做什麼」，會清楚得多。

以晶創 26 的 Nano4 為例，實測效能 81.55 PFLOPS。假設 MFU 為 0.35，用第 17.2.1 節的最適配比公式計算它在不同時間內能訓練多大的模型：

時間	總算力 (FLOPs)	最適參數量	最適訓練詞元
1 小時	約 1.03×10^{20}	約 926 M	約 18.5 B
1 天	約 2.47×10^{21}	約 4.53 B	約 90.7 B
1 週	約 1.73×10^{22}	約 12.0 B	約 240 B

（假設全機滿載且 MFU 0.35，實際上不會有單一計畫獨占全機。這個計算的目的是建立量級感，不是可行性評估。）

這張表傳達兩個訊息。第一，國家級設施的一小時，大約等於一張消費級 GPU 的數千小時——這就是為什麼某些工作只能在那裡做。第二，即使是這樣的規模，一週也只夠訓練一個 12 B 的模型——而國際大廠的旗艦模型規模遠大於此。

第二點值得認真對待。它說明了一個結構性的事實：臺灣的算力規模不足以在「訓練最大的通用模型」這條路上競爭。這與第 1.7 節的定位一致——我們的機會在應用、在邊緣、在別人不會做的差異化資料，而不在參數量的競賽。

一個對學生的重要提醒

看到上表可能會覺得沮喪，但請注意一件事：這本書的所有實作，包括第 13 章訓練出能生成繁體中文的模

型，都是在一張筆電上完成的。規模不等於價值——一個 200 M 參數但真正理解臺灣公文格式的模型，在特定場景下比一個 200 B 參數但不熟悉本地制度的模型更有用。這是第六篇之後的主線。

21.2 取得國家級算力

21.2.1 申請的基本條件

國網中心的計算資源對學研界開放申請。基本的資格條件包括在國內外擔任教學或研究職務，或取得博士學位後一定年限內的研究人員（詳細條件請查閱官方公告，因為它會調整）。

這對學生的實務意涵是：你通常需要透過指導老師申請。這不是障礙，但它意味著申請的時程要納入指導老師的行程——而那往往比技術準備花更多時間。

21.2.2 審查會看什麼

雖然各計畫的審查重點不同，但以下幾項是共通的。這也是第 17 章那份算力預算書真正派上用場的地方。

審查面向	審查者想知道	你該提供什麼	章節
必要性	為什麼一定要用這裡的資源	自有資源的盤點與缺口計算	第 17.6 節
用量估算	你要用多少、憑什麼這樣估	FLOPs 換算與 MFU 依據	第 17.6.1 節
可行性	你有能力用得起來嗎	單卡實作的既有成果	第 18、19 章
產出	會產生什麼成果	明確的交付與時程	第 16.7 節
資料合規	你的語料來源合法嗎	第 8 章的清冊	第 8 章
效率	你會不會浪費資源	MFU 實測與最佳化紀錄	第 18 章

第三列與第六列常被低估。審查者最擔心的情境是「申請了大量資源，結果因為不會用而閒置」——所以能證明你已經在小規模上跑通，比宣稱你會做什麼更有說服力。第 13 章的 MiniFormosaGPT 與第 18 章的 MFU 報告，在這裡就是你的信用證明。

21.2.3 一份申請書的骨架

1. 目標與必要性：要做什麼、為什麼需要這個規模、自有資源為何不足。
2. 技術方案：模型規格、資料規格、訓練策略，以及它們的依據。
3. 資源需求：GPU 小時、儲存空間、期程，附完整的換算過程。

4. 可行性佐證：已完成的小規模實作、實測的 MFU、團隊的相關經驗。
5. 資料合規說明：語料來源、授權狀態、個資處理（第 8 章的清冊）。
6. 產出與時程：階段性的交付物與檢核點。
7. 風險與備案：若資源不足或延遲，你的降級方案是什麼。
8. 效率承諾：你會如何確保資源不被浪費（監控、回滾、定期檢視）。

第七項的「降級方案」是一個容易加分的細節。它顯示你理解資源是有限的、也想過如果拿不到全部會怎麼辦——而審查者經常需要削減申請量，一個已經想好降級路徑的申請案，比一個「全有或全無」的容易通過。

21.2.4 常見的申請失誤

失誤	為什麼是問題	正確做法	章節
用規格書的峰值算	嚴重低估所需時間	用實測 MFU	第 15.6 節
沒有餘裕	一次發散就跑不完	保留 20% 至 30%	第 17.6.3 節
沒盤點自有資源	顯得沒做功課	先算清楚缺口	第 17.6 節
資料來源含糊	合規風險	附第 8 章的清冊	第 8 章
沒有小規模成果	可行性存疑	先做出 tiny 版	第 13 章
申請量遠超需求	審查者會質疑估算能力	誠實估算並說明依據	第 17 章

最後一列值得說明：多要一點看似安全，但它會損害你估算的可信度。一個能說出「我們需要 3,200 GPU 小時，其中 2,500 是訓練、400 是評測、300 是餘裕」的申請，遠比一個要求「一萬 GPU 小時」而說不清用途的可信。

21.3 從 GPU 小時到電力與碳排

21.3.1 完整的換算鏈

第 17.6.1 節給過從 FLOPs 到新臺幣的換算。這裡補上碳排放這一段，並使用官方公告的係數。

$$\text{用電度數} = \text{GPU 小時} \times \text{單卡功耗(kW)} \times PUE$$

$$\text{碳排放} = \text{用電度數} \times \text{電力排碳係數}$$

21.3.2 電力排碳係數

經濟部能源署每年公告電力排碳係數。以下是查核日（2026 年 8 月）時的最新公告值：

係數類別	數值 (kg CO ₂ e/度)	適用對象	備註
公用售電業	0.467	整體平均	114 年度
產業	0.466	表燈營業、低壓、高壓、特高壓	114 年度起新增分類
民生住宅	0.471	表燈非營業、包燈	同上
全國電力排放	0.456 (初估)	納入民間綠電直轉供	多用於國家統計

（資料來源：經濟部能源署 114 年度電力排碳係數公告，2026 年 6 月 2 日發布。113 年度為 0.474。查核日 2026 年 8 月。）

兩個使用上的注意事項。第一，大學實驗室與研究機構通常屬於「產業」類別（表燈營業或高壓電力），所以應該使用 0.466 而非民生的 0.471。第二，這個係數逐年下降（113 年度 0.474 → 114 年度 0.467，降幅約 1.5%），所以引用時務必註明年度。

21.3.3 PUE：機房的隱形成本

PUE (Power Usage Effectiveness) 是「資料中心總用電」除以「IT 設備用電」。它衡量的是冷卻、供電損耗等額外開銷。

PUE	額外開銷	評價	典型情境
1.1	10%	極佳	先進的專業資料中心
1.3	30%	良好	國網中心台南分部的規劃值
1.5	50%	一般	多數企業機房
2.0	100%	差	未經最佳化的舊機房

（國網中心台南分部的 PUE 規劃值為 1.3，查核日 2026 年 8 月。）

PUE 對實驗室的實務意涵是：如果你的機器放在一間普通的研究室裡吹冷氣，實際的 PUE 可能接近

或超過 2.0。這意味著你的電費與碳排是 IT 用電的兩倍——那個差距在做預算時經常被漏掉。

21.3.4 實際算幾個例子

用產業碳排係數 0.466、單卡功耗 350 W、PUE 1.3 計算：

情境	用電度數	碳排 (kg CO ₂ e)	約等於
單卡跑 24 小時	11 度	5.1	很小
第 17 章的 500 M 模型訓練	533 度	248.5	約 0.25 噸
8 卡跑一個月	2,621 度	1,221.3	約 1.22 噸

(第二列採用第 17.6.4 節的預算範例：1,172 GPU 小時。)

這些數字傳達一個重要的訊息：單次的小規模實驗，環境成本可以忽略。一次 24 小時的訓練只有 5 公斤碳排，相當於很短的一段車程。所以不必為了做一個實驗而有罪惡感。

但第三列開始就有意義了。8 張卡跑一個月是 1.22 噸——如果一個實驗室常態性地這樣運作，一年就是十幾噸。而這正是第 21.5 節要討論的：什麼規模以上該把環境成本納入規劃。

21.3.5 電價

電價由經濟部電價費率審議會決定，原則上每年召開兩次。查核日 (2026 年 8 月) 時的狀態是：2025 年 10 月 1 日的調整中產業電價維持不變；2026 年 3 月 27 日的審議會決議民生用電費率維持不變。

實務上的建議是：不要自己去查電價表算，而是直接向學校的總務或事務單位詢問你們實際適用的費率。學校通常是高壓或特高壓用戶，且可能有時間電價 (尖離峰差異大)，自己按住宅費率估算會嚴重失準。

一個實用的估算捷徑

如果你完全不知道費率，可以先用「每度 3 至 4 元」做粗估，並在報告中明確標示這是估計值。這個範圍對多數學術用戶而言不會離譜太遠，足以支撐「這個實驗要花幾百元還是幾萬元」的量級判斷——而在規劃階段，量級通常就夠了。

21.3.6 把碳排寫進實驗紀錄

第 15 章的 24 小時訓練報告已經要求記錄功耗。這裡把它接成完整的碳排追蹤。

| formosa/planning/carbon.py

```

from dataclasses import dataclass, asdict
from datetime import date

@dataclass
class CarbonRecord:
    """每次訓練都應產生一筆，附在環境指紋旁"""
    run_name: str
    gpu_hours: float
    watts_per_gpu: float
    n_gpus: int
    pue: float
    carbon_factor: float          # kg CO2e/度
    factor_year: str              # 例如 "114 年度"
    factor_as_of: str             # 公告日
    factor_source: str = "經濟部能源局"

    @property
    def kwh(self) -> float:
        return self.gpu_hours * self.watts_per_gpu / 1000 * self.pue

    @property
    def co2e_kg(self) -> float:
        return self.kwh * self.carbon_factor

    def to_dict(self):
        d = asdict(self)
        d.update({"kwh": round(self.kwh, 2),
                  "co2e_kg": round(self.co2e_kg, 2),
                  "co2e_tonnes": round(self.co2e_kg / 1000, 4),
                  "recorded_at": date.today().isoformat()})
        return d

def project_total(records: list[CarbonRecord]):
    """專案累計——這才是有意義的量級"""
    total_kwh = sum(r.kwh for r in records)
    total_co2 = sum(r.co2e_kg for r in records)
    return {
        "n_runs": len(records),
        "total_gpu_hours": round(sum(r.gpu_hours for r in records), 1),
        "total_kwh": round(total_kwh, 1),
        "total_co2e_kg": round(total_co2, 1),
        "total_co2e_tonnes": round(total_co2 / 1000, 3),
        # 揭露用的一句話
        "disclosure": (f"本研究共使用約 {sum(r.gpu_hours for r in records):.0f} "
                       f"GPU 小時，估計用電 {total_kwh:.0f} 度、"
                       f"碳排放約 {total_co2/1000:.2f} 公噸 CO2e"),
    }

```

disclosure 這個欄位直接產生可以貼進論文的一句話。第 21.5.3 節建議學位論文揭露碳排，而這個函式讓那件事變成零額外工作——訓練跑完就有現成的句子。

注意 factor_year 與 factor_as_of 是必填的。理由與第 4.7.3 節的環境指紋相同：一個沒有標明來源年度的碳排數字，三年後無法被驗證。而碳排係數逐年下降，用錯年度會產生系統性的偏差。

21.4 四種方案的總持有成本

21.4.1 比較的架構

第 1.4.4 節與第 17.6.2 節都碰過這個比較，這裡做完整的版本。

方案	前期成本	持續成本	關鍵限制
自建	硬體採購	電費、維運人力、折舊	需要空間、電力、散熱
校級共用	無（或分攤）	可能有計點制度	排隊、時段限制
國家級設施	無	可能有計價	申請週期、使用規範
商業雲端	無	按時計費	長期昂貴、資料外流評估

21.4.2 三年期的總持有成本

做這個比較時必須涵蓋的項目：

1. 硬體採購（自建）或使用費（其他）。
2. 電費： $\text{GPU 小時} \times \text{功耗} \times \text{PUE} \times \text{費率}$ 。
3. 空間與基礎設施：機房改裝、電力擴充、散熱。
4. 維運人力：估算每月需要多少工時，乘上合理的時薪。
5. 折舊與汰換：GPU 的實用壽命通常估三到五年。
6. 閒置成本：自建設備在你不用的時候仍在折舊。
7. 機會成本：等待國家級資源審查期間無法進行的工作。

第四項與第六項是新手最常漏掉的。維運不是零成本——驅動更新、硬碟故障、散熱清理、使用者支援，這些加起來每月可能要好幾小時。而閒置成本則說明了一個常見的誤判：如果你的設備只有 30% 的時間在用，那實際的每 GPU 小時成本是帳面的三倍多。

21.4.3 決策準則

你的情況	建議	理由	章節
用量小且不確定	雲端或校級共用	避免閒置	21.4.2

你的情況	建議	理由	章節
用量穩定且持續超過一年	自建	損益兩平通常在數月	第 1.4.4 節
需要大規模但只是階段性	國家級設施	規模與成本兼顧	21.2
資料極度敏感	自建	不外流	第 8 章
尖峰需求	雲端補足	彈性	21.4.1

第二列的「損益兩平通常在數月」值得說明：以雲端的計價與自建的硬體成本相比，若設備能維持高使用率，通常幾個月就回本。但那個前提是「高使用率」——回到第六項的閒置成本，使用率不足時自建反而較貴。

所以本書的建議是：先用雲端或共用資源跑幾個月，量出你真實的使用率，再決定要不要自建。這個順序能避免一個常見的錯誤——先買了設備，才發現用不到那麼多。

21.4.4 一個完整的比較範例

把 21.4.2 節的七項成本套用在一個具體情境上：某實驗室需要 5,000 GPU 小時，考慮三年的規劃。

項目	自建（2 張卡）	雲端	國家級設施
前期投入	硬體採購	無	無（需申請）
每 GPU 小時	電費約 1.6 元	市場價格	依計畫而定
維運人力	每月數小時	無	無
折舊	三至五年	無	無
閒置成本	有（依使用率）	無	無
取得時間	採購週期	立即	審查週期
資料外流風險	無	需評估	需依規範

（每 GPU 小時的電費依 350 W、PUE 1.3、每度 3.5 元計算： $0.35 \times 1.3 \times 3.5 \approx 1.59$ 元。硬體價格與雲端費率隨時變動，請於決策時查詢。）

這張表刻意留白了幾個數字——那是必須由你當下查詢的。但它的結構本身就有價值：它列出了所有該比較的維度，而多數人的比較只看了第二列。

最後兩列尤其重要。取得時間影響你的專案時程（申請可能要數週到數月），資料外流風險則可能直接排除某個選項（第 8 章的敏感資料）。這兩者都不是金額，但它們常常才是決定性的因素。

21.4.5 混合策略

實務上很少是「只選一個」。一個常見且務實的組合是：

用途	用哪個資源	理由	章節
日常開發與除錯	自有小卡	需要立即可用、頻繁中斷	第 18 章
中等規模實驗	校級共用	不需獨占、成本低	21.7
大規模訓練	國家級設施	規模與成本兼顧	21.2
尖峰或臨時需求	雲端	彈性	21.4.1
敏感資料	自有設備	不外流	第 8 章

這個組合的關鍵是把不同性質的工作放到適合的資源上。日常除錯需要的是「隨時可用」而非「規模大」，用國家級設施反而不方便（排隊、時段限制）；而大規模訓練需要的是規模，用自有的兩張卡跑三個月不切實際。

第五列則是一條硬性的界線：涉及第 8 章的敏感資料時，資源選擇由治理需求決定，而非成本。這是少數不能純用金錢計算的情況。

21.5 把環境成本納入規劃

21.5.1 什麼規模以上該考慮

21.3.4 節的數字給出了一個清楚的分界。

規模	約略碳排	該做什麼	理由
單次小實驗	幾公斤	不必特別考慮	量級太小
一次完整訓練	約 0.25 噸	記錄下來	建立基準

規模	約略碳排	該做什麼	理由
實驗室年度用量	十幾噸	納入規劃與檢視	有實質意義
大型計畫	數十至數百噸	應在計畫書中揭露	影響決策

本書的建議是第二列開始：從第一次完整訓練就開始記錄。這不是為了限制你，而是為了建立基準——當你有了自己的數字，往後的判斷才有依據。第 15 章的 24 小時訓練報告已經要求記錄功耗，這裡只是把它換算成碳排。

21.5.2 減少環境成本的手段

好消息是：多數降低環境成本的做法，同時也降低金錢成本與時間成本。它們不是取捨，而是同一件事的三種說法。

手段	節省幅度	同時省下	章節
提升 MFU	正比於提升幅度	時間、電費	第 18 章
選對模型規模	可能數倍	時間、電費	第 17.2 節
避免無效訓練	視情況	全部	第 16 章的實驗紀律
改用適配而非從零訓練	一至兩個數量級	全部	第六篇
改善 PUE	最多 30% 至 50%	電費	21.3.3
離峰時段執行	電費	(碳排未必降)	21.3.5

第一列的意義值得強調：第 18 章的系統案例要求把 MFU 提升一倍，那同時就是電費減半、碳排減半。一個原本要跑 20 天的訓練變成 10 天，省下的不只是時間。

第三列可能是最大的一項。第 16 章說過，單一種子的結果有 50.6% 的機率是誤導的——一個因為實驗設計不良而必須重做的訓練，它的環境成本是完全浪費的。做對實驗本身就是一種節能。

最後一列有一個需要澄清的地方：離峰時段執行能省電費（若採時間電價），但碳排未必降低——那取決於當時的電力結構。若要真正降低碳排，需要看即時的電力排放資訊，而那超出多數專案的操作範圍。誠實的做法是：省電費就說省電費，不要順便宣稱減碳。

21.5.3 該不該揭露

情境	建議	理由	如何做
課程作業	記錄但不必揭露	建立習慣	寫進實驗紀錄
學位論文	建議揭露	學術誠信與完整性	一段附註
期刊投稿	依領域慣例	部分會議已要求	查該會議的規定
計畫申請	建議主動說明	顯示規劃完整	含在資源需求中
對外服務	視規模	可能有法遵要求	諮詢相關單位

第二列的建議理由是實際的：一份論文若說明了「本研究共使用約 3,500 GPU 小時、估計碳排約 1.6 噸」，讀者能更準確地判斷這個結果的成本與可重現性。那與第 16 章要求報告種子數與信賴區間是同一個精神——讓讀者能自行評估。

21.6 資料時效與查核清單

21.6.1 本章哪些資料會過期

這一節是本章最重要的部分之一。以下所有項目都必須在使用前重新查核。

項目	本書查核日	更新頻率	查核來源
設施規格與名稱	2026 年 8 月	每隔數年	國網中心官方網站
申請資格與流程	2026 年 8 月	不定期	同上
電力排碳係數	114 年度 (2026 年 6 月公告)	每年一次	經濟部能源署
電價費率	2026 年 8 月	每年約兩次	台電官網或校內事務單位
PUE 規劃值	2026 年 8 月	依設施而定	設施方公告
雲端價格	未列出	隨時可能調整	各服務商官網

注意最後一列：本章刻意不列出商業雲端的價格。原因是它們變動太快，且依區域、機型、承諾用量而異——列出來只會製造一個很快就錯誤的參考點。正確的做法是在做決策時當場查詢，並記錄查詢

日期。

21.6.2 查核腳本

scripts/refresh_compute_facts.py (骨架)

```
"""本章的版本敏感資料應每學期重新查核一次。
這支腳本不自動抓取（各單位的網頁結構會變），
而是產生一份待查核清單並記錄結果。"""

import json
from datetime import date
from pathlib import Path

FACTS = [
    {"key": "carbon_factor_industry",
     "desc": "產業電力排碳係數 (kg CO2e/度)",
     "source": "經濟部能源局 電力排碳係數公告",
     "value": 0.466, "as_of": "2026-06-02", "year_label": "114 年度"},
    {"key": "carbon_factor_public",
     "desc": "公用售電業電力排碳係數",
     "source": "同上", "value": 0.467, "as_of": "2026-06-02",
     "year_label": "114 年度"},
    {"key": "nchc_facilities",
     "desc": "國網中心主要系統與規格",
     "source": "國網中心官方網站 超級電腦頁面",
     "value": "見第 21.1.2 節", "as_of": "2026-08"},
    {"key": "electricity_rate",
     "desc": "適用電價費率",
     "source": "台電官網／校內事務單位",
     "value": None, "as_of": None,
     "note": "必須向校內單位查詢實際適用費率"},
]

def staleness_report(max_age_days=180):
    today = date.today()
    rows = []
    for f in FACTS:
        if not f.get("as_of"):
            rows.append(**f, "status": "從未查核"); continue
        a = f["as_of"]
        d = date.fromisoformat(a if len(a) > 7 else a + "-01")
        age = (today - d).days
        rows.append(**f, "age_days": age,
                    "status": "需重新查核" if age > max_age_days else "尚可")
    return rows

def print_report(rows):
    print(f"{'項目':<28}{'狀態':<12}{'距查核日':>10}")
    for r in rows:
        age = r.get("age_days")
        print(f"{r['desc'][:26]:<28}{r['status']:<12}"
              f"{'{(str(age) + ' 天') if age is not None else '-':>10}")
        if r["status"] != "尚可":
            print(f"    來源：{r['source']}")
```

這支腳本的價值不在於自動化，而在於把「該查核什麼」變成一份明確的清單。每學期開課前跑一次，更新其中的數值，你的課程資料就不會悄悄過期。

21.7 實驗室的資源治理

21.7.1 共用制度的三個問題

第 4.9.1 節提過共用 GPU 的三個問題，這裡給出完整的制度設計。

問題	典型情境	制度解法	技術解法
資源被獨占	一人跑滿所有卡數天	時段配額、單人上限	排程系統的優先權
閒置浪費	占著卡但沒在跑	定期檢視使用率	自動偵測閒置並回收
磁碟爆滿	checkpoint 沒清理	每人配額	自動清理舊檔
資源不足時的爭議	兩人同時要用	事先約定的優先順序	排隊機制
環境不一致	各自裝各自的套件	容器或共用環境	第 4.5 節

21.7.2 一個可行的時段制度

第 4.9.1 節提過把 GPU 分成開發與訓練兩個時段。這裡給出更完整的版本。

時段	規則	目的	執行方式
平日白天	單次不超過 30 分鐘	確保所有人都能除錯	排程系統的時限
平日夜間	需登記，單次最長 12 小時	中等規模實驗	共用登記表
週末與假日	長時間訓練，需事先登記	大型訓練	同上
期末衝刺期	依專題進度排優先序	公平分配尖峰需求	由指導老師協調

這個制度不需要複雜的軟體——一張共用試算表加上大家的自律就能運作。重點不在於技術，而在於規則要事先講清楚並公告。多數的資源爭議來自「沒有規則」而非「規則不好」。

21.7.3 使用率的量測

| scripts/gpu_utilization_report.py (節錄)

```

import subprocess, json, time
from collections import defaultdict

def sample_gpu_state():
    """取樣目前各卡的使用狀態"""
    try:
        out = subprocess.check_output([
            "nvidia-smi",
            "--query-gpu=index,utilization.gpu,memory.used,memory.total",
            "--format=csv,noheader,nounits"], text=True)
    except (FileNotFoundError, subprocess.CalledProcessError):
        return None

    rows = []
    for line in out.strip().splitlines():
        idx, util, used, total = [x.strip() for x in line.split(",")]
        rows.append({"gpu": int(idx), "util": int(util),
            "mem_used_MB": int(used),
            "mem_total_MB": int(total),
            "ts": time.time()})

    return rows

def summarize(samples, idle_util=5, occupied_mem_ratio=0.1):
    """區分三種狀態：閒置、佔用但未使用、實際使用"""
    stats = defaultdict(lambda: {"idle": 0, "occupied_idle": 0,
        "active": 0, "n": 0})

    for s in samples:
        g = stats[s["gpu"]]
        g["n"] += 1
        mem_ratio = s["mem_used_MB"] / max(s["mem_total_MB"], 1)
        if s["util"] > idle_util:
            g["active"] += 1
        elif mem_ratio > occupied_mem_ratio:
            g["occupied_idle"] += 1      # 佔著記憶體卻沒在算
        else:
            g["idle"] += 1

    out = {}
    for gpu, g in stats.items():
        n = max(g["n"], 1)
        out[gpu] = {
            "active_ratio": round(g["active"] / n, 3),
            "occupied_idle_ratio": round(g["occupied_idle"] / n, 3),
            "idle_ratio": round(g["idle"] / n, 3),
        }
    return out

```

occupied_idle 這個狀態是本工具最有價值的部分：卡被佔著但沒在計算。這通常是某人開了一個互動式的環境然後去吃飯，或是程式卡在資料載入。在共用環境中，這是最大的浪費來源——因為別人看到卡「有人用」就不會去碰，但實際上它什麼也沒做。

量出這個比例之後，制度就有了依據。如果 occupied_idle 長期超過三成，那就該加入自動回收機制或調整登記規則。

程式實作

本章的三個實作偏向規劃與治理工具，不需要大量算力，但它們的產出會直接用在你的專題申請與資源管理上。

程式 21A 資源盤點與成本計算器

目標：把第 17 章的算力預算計算器擴充成完整的四方案比較工具，含電力與碳排。

formosa/planning/tco.py (節錄)

```
from dataclasses import dataclass, field

@dataclass
class CarbonConfig:
    """版本敏感：使用前請依 21.6 節重新查核"""
    factor_kg_per_kwh: float = 0.466 # 114 年度產業係數
    as_of: str = "2026-06-02"
    label: str = "經濟部能源局 114 年度產業電力排碳係數"

@dataclass
class SiteConfig:
    pue: float = 1.3
    price_per_kwh: float = 3.5 # 應向校內單位查詢實際費率
    price_source: str = "估計值，需查核"

def energy_and_carbon(gpu_hours, watts_per_gpu, site: SiteConfig,
                     carbon: CarbonConfig):
    kwh = gpu_hours * watts_per_gpu / 1000 * site.pue
    return {
        "gpu_hours": round(gpu_hours, 1),
        "kwh": round(kwh, 1),
        "electricity_twd": round(kwh * site.price_per_kwh),
        "co2e_kg": round(kwh * carbon.factor_kg_per_kwh, 1),
        "co2e_tonnes": round(kwh * carbon.factor_kg_per_kwh / 1000, 3),
        "assumptions": {
            "pue": site.pue,
            "price_per_kwh": site.price_per_kwh,
            "price_source": site.price_source,
            "carbon_factor": carbon.factor_kg_per_kwh,
            "carbon_as_of": carbon.as_of,
        },
    }

def compare_options(gpu_hours_needed, options: list[dict]):
    """options 每項需含 name、上手成本、每 GPU 小時成本、限制"""
    rows = []
    for o in options:
        total = o["setup_twd"] + gpu_hours_needed * o["per_gpu_hour_twd"]
        rows.append({**o, "total_twd": round(total),
                      "per_hour_effective": round(total / max(gpu_hours_needed, 1), 2)})
    rows.sort(key=lambda r: r["total_twd"])
    return rows
```

注意 assumptions 這個欄位：所有的估算都必須把假設一起回傳。這與第 17.6.3 節要求預算書附假設清單是同一個原則——一個沒有標明假設的成本數字是無法被檢驗的。

必須產出的四項：

- 你的專題所需 GPU 小時（用第 17 章的公式）。
- 四種方案的三年期總持有成本比較。
- 電力與碳排估算，含所有假設與查核日。
- 一段判斷：你會選哪一個？在什麼條件下會改變決定？

程式 21B 算力申請書產生器

目標：把第 17 章的預算書與本章的申請要點整合成一份可提交的文件。

段落	內容	資料來源	章節
目標與必要性	要做什麼、自有資源的缺口	程式 21A	21.2.3
技術方案	模型與資料規格	第 17 章的預算書	第 17.6 節
資源需求	GPU 小時、儲存、期程	程式 17B	第 17.6.1 節
可行性佐證	小規模成果與 MFU 實測	第 13、18 章	21.2.2
資料合規	語料清冊與授權	第 8 章	第 8 章
降級方案	資源不足時的備案	自行規劃	21.2.3

延伸挑戰：寫一個「自我審查」功能，依 21.2.4 節的六種常見失誤逐項檢查你的申請書。例如：有沒有實測 MFU？有沒有保留餘裕？有沒有小規模成果？在提交之前跑一次，可以避免最常見的退件原因。

程式 21C 實驗室資源監控

目標：實作 21.7.3 節的使用率量測，並產出可供制度決策的報表。

必須完成的五項：

- 連續取樣至少一週，記錄各卡的三種狀態比例。
- 找出 occupied_idle 最高的時段——那通常是制度該介入的地方。

- 計算實際的每 GPU 小時成本（總持有成本除以「實際使用」的小時數，而非日曆小時數）。
- 與帳面成本對照：若使用率只有 40%，實際成本是帳面的 2.5 倍。
- 一頁建議：依你的量測結果，提出一項具體的制度或技術改善。

一個常見的發現

多數實驗室第一次做這個量測時，會發現實際使用率遠低於預期——常見的數字是 30% 到 50%。這不是誰的錯，而是「排程摩擦」的自然結果：等待、除錯、資料準備、以及單純的沒人在用。知道這個數字之後，「該不該再買一張卡」這個問題就有了不同的答案——也許先提升使用率更划算。

系統案例：實驗室三年技術路線規劃

本章的系統案例是第 5 篇的收束，也是第一與第二學期所有量化工具的整合：為一個資源受限的臺灣大學實驗室，規劃一條三年的技術路線。

情境設定

條件	設定	意涵	章節
現有硬體	2 張消費級 GPU	約 60 TFLOPS 峰值	第 18 章
實測 MFU	0.18	有最佳化空間	第 15.6 節
可用繁中語料	約 3 B 詞元（去重後）	限制模型規模	第 9、17.4 節
人力	教師 1 人、研究生 3 人	維運能力有限	21.4.2
經費	每年有限的計畫經費	需精打細算	21.4
目標	做出可用的臺灣本土應用	需明確定義	第 1 章

要規劃的三個年度

年度	該做什麼	為什麼是這個順序	章節
第一年	最佳化現有資源、建立語料與評測、做出小規模成果	這是申請更多資源的前提	第 8、9、13、18 章
第二年	申請國家級資源、做適配而非從零訓練	有了佐證才申請得到	第 21.2 節、第六篇

年度	該做什麼	為什麼是這個順序	章節
第三年	產學合作取得差異化資料、擴大應用	資料才是長期的護城河	第 1.6.5 節、第八篇

這個順序的邏輯是本書一貫的立場：先把手上的做好，再去要更多。第一年的重點不是規模，而是建立三樣東西——可治理的語料、可信的評測、以及能證明你會用資源的成果。沒有這三樣，第二年的申請書寫不出來。

必須回答的六個問題

1. 第一年：把 MFU 從 0.18 提升到多少是合理的？提升後等同於多了幾張卡？
2. 第一年：以 3 B 詞元的語料，由資料決定的最適模型規模是多少？（第 17.4.3 節）
3. 第二年：申請多少 GPU 小時？依據是什麼？降級方案是什麼？
4. 第二年：從零訓練與在開放權重模型上適配，成本差幾倍？你選哪一個？
5. 第三年：什麼樣的資料是別人拿不到的？如何取得？（第 8 章的合作授權）
6. 三年合計：總電費與碳排估計是多少？

第四個問題是這個案例的核心，也是本書從第 1.4 節就埋下的伏筆。以 2 張卡與 3 B 詞元的條件，從零訓練能做出的模型規模非常有限；而在一個既有的開放權重模型上做繁中適配（第 22 章），能用同樣的資源達到高得多的實用性。能算出這一點並接受它，是這個案例最重要的產出。

交付要求

- 三年路線圖：每年的目標、產出、資源需求與檢核點。
- 第一年的最佳化計畫：依第 18 章的自檢清單，列出你會做的項目與預期效益。
- 資料規劃：可用語料的盤點、由資料決定的模型上限、以及取得更多資料的管道。
- 第二年的申請書草案：依 21.2.3 節的八個段落。
- 成本與碳排總表：三年的估計，含所有假設與查核日。
- 路線比較：從零訓練 vs 適配的完整比較，以及你的選擇與理由。
- 風險與轉折點：什麼情況下你會改變這個規劃？

評分重點

- 第一年是否以「最佳化與基礎建設」為主。直接跳到訓練大模型者扣分。
- 模型規模是否由資料反推而非憑感覺。

- 是否誠實比較了從零訓練與適配兩條路線。
- 申請書是否包含降級方案。
- 碳排估算是是否使用官方公告係數並註明年度與查核日。
- 風險與轉折點是否具體。「如果順利就繼續」不算。

章末評量

一、概念題

1. 臺灣的算力來源有哪五層？為什麼說它不是「有或沒有」的二元問題？
2. 為什麼「能證明你已經在小規模上跑通」比「宣稱你會做什麼」更有說服力？
3. 解釋 PUE 的意義。為什麼一般研究室的實際 PUE 可能接近 2.0？
4. 電力排碳係數為什麼要分「產業」與「民生住宅」？大學實驗室該用哪一個？
5. 為什麼說「多數降低環境成本的做法，同時也降低金錢與時間成本」？請舉三例。
6. 為什麼「離峰執行能省電費但未必減碳」？
7. 什麼是 occupied_idle 狀態？為什麼它是共用環境中最大的浪費來源？
8. 為什麼本章刻意不列出商業雲端的價格？
9. 為什麼系統案例的第一年重點是最佳化與基礎建設，而非訓練大模型？

二、計算題

1. 晶創 26 的 Nano4 實測效能 81.55 PFLOPS、功率 2.214 MW。計算其能效 (GFLOPS/W)。
2. 承上題，若該系統全年滿載運轉，一年需要多少度電？(一年以 8,760 小時計)
3. 一次訓練用了 1,500 GPU 小時，單卡功耗 400 W、PUE 1.4。計算用電度數，以及用產業碳排係數 0.466 計算的碳排放。
4. 承上題，若電價為每度 3.5 元，電費是多少？
5. 一個實驗室有 4 張卡，量測到實際使用率為 35%。若帳面的每 GPU 小時成本是 8 元，實際的每「有效」GPU 小時成本是多少？
6. 你的 MFU 從 0.18 提升到 0.36。原本要跑 30 天的訓練現在要幾天？省下的電費若每度 3.5 元、單卡 350 W、PUE 1.3、共 2 張卡，是多少？

三、除錯題

1. 同學的算力申請書被退件，理由是「用量估算依據不足」。請列出四個他可能漏掉的東西。
2. 同學用民生電力排碳係數計算實驗室的碳排。請說明問題與正確做法。
3. 同學的成本估算只算了電費，沒算 PUE。若實際 PUE 是 1.8，他低估了多少？
4. 實驗室新買了兩張卡，但三個月後大家仍覺得算力不足。請列出三個該檢查的地方。
5. 同學宣稱「我們的訓練排在離峰時段，所以是低碳的」。請說明這個說法的問題。
6. 同學引用本書的碳排係數 0.466 寫進 2028 年的論文。請說明風險。

四、系統設計題

1. 你的系所要建置一座共用的 GPU 叢集供全系使用。請設計完整的治理方案：資源分配規則、使用率監控、爭議處理、以及如何避免 occupied_idle 的浪費。並說明哪些部分需要制度而非技術。
2. 你要說服系上主管投入經費建置算力。請設計一份三頁的簡報大綱：需要哪些資料、如何呈現總持有成本、以及如何回應「為什麼不直接用雲端」的質疑。

五、臺灣情境與倫理題

1. 臺灣的電力供應是實質限制，而 AI 訓練的用電需求持續成長。請討論：學術界在這個議題上有什麼責任？是自律、揭露、還是應該有制度性的規範？
2. 國家級算力設施的資源有限，而申請者眾。請討論：什麼樣的分配原則是公平的？大型計畫與小型研究、資深與新進研究者之間該如何權衡？

評量提示（給自學者）

- 計算題第 1 題： $81.55 \times 10^{15} \div (2.214 \times 10^6) \approx 3.68 \times 10^{10} \text{ FLOPS/W} = 36.8 \text{ GFLOPS/W}$ 。
- 計算題第 2 題： $2.214 \text{ MW} \times 8,760 \text{ 小時} \approx 19,394 \text{ MWh}$ ，約 1,939 萬度。
- 計算題第 3 題： $1,500 \times 0.4 \times 1.4 = 840 \text{ 度}$ ； $840 \times 0.466 \approx 391.4 \text{ 公斤 CO}_2\text{e}$ 。
- 計算題第 4 題： $840 \times 3.5 = 2,940 \text{ 元}$ 。
- 計算題第 5 題： $8 \div 0.35 \approx 22.9 \text{ 元}$ 。
- 計算題第 6 題： $30 \times (0.18/0.36) = 15 \text{ 天}$ ，省 15 天；電費 = $15 \times 24 \times 2 \times 0.35 \times 1.3 \times 3.5 \approx 1,147 \text{ 元}$ 。
- 除錯題第 3 題：低估了 80%——他算的是 IT 用電，實際總用電是它的 1.8 倍。

研究延伸與版本注意事項

- 國網中心官方網站：設施規格、申請流程與服務項目的權威來源，每學期應重新查核。
- 經濟部能源署的電力排碳係數公告：每年更新，是碳排計算的官方依據。
- 台灣電力公司的電價表：每年約調整兩次；但實務上請優先向校內事務單位查詢實際適用費率。
- Green500 排行榜：能效的國際比較基準，可用來評估設施的節能表現。
- AI 訓練的碳足跡研究：國際上有多篇量化研究，可作為與本地數字對照的參考。
- 本章是全書資料時效性最強的一章。所有數字都標註了查核日，使用前務必依 21.6.1 節的清單重新確認。

常見問題

問：我只是學生，需要關心算力生態嗎？答：需要。你的專題規模、指導老師的計畫申請、甚至你畢業後的工作，都會受這些條件影響。而更實際的是：本章的成本計算會直接告訴你「你的專題該做多」——那比事後發現跑不完要好得多。

問：申請國家級資源會不會很難？答：技術上不難，但需要時間與準備。多數人失敗的原因不是能力不足，而是準備不夠——沒有小規模成果、沒有實測資料、資料來源說不清楚。而那些準備工作，本書的前二十章已經帶你做完了。

問：碳排的數字有必要算嗎？答：對小規模實驗確實可以忽略（21.3.4 節的第一列）。但養成記錄的習慣有兩個價值：一是當你的規模變大時你已經有基準，二是它會讓你更認真看待「不要浪費一次訓練」——而那本來就是第 16 章的實驗紀律。

問：本章的數字什麼時候會過期？答：電力排碳係數每年更新一次（約在年中公告前一年度）、電價每年約調整兩次、設施規格每隔數年更新。本書標註的查核日是 2026 年 8 月。如果你現在讀到這裡而距離那個日期已超過半年，請先跑一次 21.6.2 節的查核清單。

教師使用建議

本章排在第 21 週，是第 5 篇的最後一章，建議 2 小時講授加 1 小時實驗。本章的資料時效性強，教師應在每學期開課前更新一次。

時段	內容	互動設計	注意事項
前 25 分鐘	21.1 至 21.2 節，算力地圖與申請	展示一份真實的申請書（去識別）	審查重點要具體
中間 25 分鐘	21.3 節，電力與碳排	現場算學生自己專題的碳排	小規模可忽略要說清楚
後 25 分鐘	21.4 至 21.5 節，總持有成本與環境	算一次自建 vs 雲端的損益兩平	閒置成本要強調
最後 35 分鐘	21.7 節與系統案例	各組討論實驗室的資源制度	制度問題與技術問題要分開
實驗課 1 小時	程式 21A 與 21C	實際量測實驗室的 GPU 使用率	結果通常令人意外

一個效果極佳的活動：在課程開始前一週就啟動程式 21C 的監控，上課時公布實驗室實際的 GPU 使

用率。多數情況下這個數字會遠低於大家的預期（30% 到 50% 是常見的），而那個落差會立刻讓「我們需要更多卡」這個訴求變成一個需要重新思考的問題。

關於資料更新：建議把本章的所有版本敏感數字集中在一份課程講義或設定檔中，每學期開課前依 21.6.1 節的清單更新一次。這既是教學需要，也是示範第 4 章的版本管理原則——讓學生看到你自己也在做這件事，比講十遍都有效。

高中授課的調整：21.1 用「臺灣有哪些超級電腦」引起興趣（規格數字對高中生很有吸引力）；21.2 略過；21.3 完整教（電費與碳排的計算對高中生完全可行，且與環境議題相通）；21.4 用「買車 vs 租車」的比喻；21.5 完整教（這是很好的科技與社會議題）；21.6、21.7 略過。程式做 21A 的填表版。

附：本章速查表

把本章的公式與版本敏感數值整理成一張表。使用前請依 21.6.1 節重新查核。

要算什麼	公式或數值	來源	查核日
用電度數	GPU 小時 × 功耗(kW) × PUE	—	—
碳排放	用電度數 × 排碳係數	—	—
產業排碳係數	0.466 kg CO ₂ e/度 (114 年度)	經濟部能源署	2026-06
公用售電業係數	0.467 kg CO ₂ e/度 (114 年度)	同上	2026-06
民生住宅係數	0.471 kg CO ₂ e/度 (114 年度)	同上	2026-06
電價	需向校內事務單位查詢	台電 / 校內	每年約兩次
PUE 參考值	專業機房 1.1–1.3、一般研究室可能近 2.0	設施方公告	2026-08
台灣杉二號	2,016 × V100、9 PFLOPS	國網中心	2026-08
晶創 26 Nano4	Rpeak 117.92 / Rmax 81.55 PFLOPS、2.214 MW	同上	2026-08
Nano4 能效	約 36.8 GFLOPS/W	由上式計算	2026-08
有效 GPU 小時成本	總持有成本 ÷ 實際使用小時	自行量測	—

使用這張表的三個提醒。第一，排碳係數要選對類別——大學實驗室用產業係數而非民生。第二，電價不要自己查表算，直接問校內單位。第三，所有引用都要附查核日，因為這些數字每年都在變。

本章小結

1. 臺灣的算力是一個五層的光譜：個人設備、實驗室自建、校級共用、國家級設施、商業雲端。
2. 國網中心是學研界取得大規模算力的主要管道；台灣杉二號為 2,016 顆 V100、9 PFLOPS，晶創 26 的 Nano4 峰值達 117.92 PFLOPS (查核日 2026 年 8 月)。
3. Nano4 的實測效能 81.55 PFLOPS、功率 2.214 MW，能效約 36.8 GFLOPS/W。
4. 申請審查最看重「必要性、用量依據、可行性佐證」；能證明小規模跑通比宣稱能力更有說服力。
5. 常見的申請失誤包括用規格書峰值估算、沒有餘裕、沒盤點自有資源、以及缺少小規模成果。
6. 碳排計算為「GPU 小時 × 功耗 × PUE × 排碳係數」；114 年度產業係數為 0.466 kg CO₂e/度。
7. 大學實驗室通常適用「產業」而非「民生住宅」的排碳係數。
8. PUE 衡量冷卻與供電損耗；國網中心台南分部規劃值為 1.3，而一般研究室可能接近 2.0。
9. 單次小實驗的碳排僅數公斤可忽略；但 8 卡跑一個月約 1.22 噸，實驗室年度用量則有實質意義。
10. 多數降低環境成本的手段同時降低金錢與時間成本；提升 MFU 一倍即等同電費與碳排減半。
11. 避免無效訓練可能是最大的一項節能——做對實驗本身就是節能。
12. 離峰執行能省電費但未必減碳，兩者不應混為一談。
13. 總持有成本必須涵蓋維運人力、折舊與閒置成本；使用率不足時自建反而較貴。
14. occupied_idle (佔著卡但沒在算) 是共用環境中最大的浪費來源，應量測後再設計制度。
15. 本章的設施規格、電價與碳排係數都會過期，使用前必須依 21.6.1 節的清單重新查核。
16. 資源受限時的正確順序是：先把手上的做好，再去要更多——這也是申請能否成功的關鍵。

成果檢核

完成本章後你必須繳交：

- 資源盤點與成本計算 (程式 21A)：四方案比較、電力與碳排估算、完整的假設清單。
- 算力申請書草案 (程式 21B)：八個段落齊備，含降級方案，並通過自我審查。
- GPU 使用率報告 (程式 21C)：至少一週的量測、三種狀態的比例、以及實際的每有效 GPU 小時成本。
- 三年技術路線規劃：六個問題的完整回答，含從零訓練與適配的成本比較。

- 碳排紀錄：你在本書至今所有訓練的估計碳排總和，含使用的係數與查核日。
- 資料查核紀錄：依 21.6.1 節的清單，記錄你實際查核的日期與結果。

第 5 篇到此結束。這四章把「硬體」從一個模糊的限制變成一組可診斷、可規劃、可治理的系統：第 18 章讓你能找出單卡的瓶頸，第 19 章讓你判斷該不該多卡，第 20 章讓你評估架構的取捨，本章讓你把這一切換算成臺灣的實際資源與成本。

從第 6 篇開始，我們會回到模型本身——但這次是在別人的模型上工作。而本章算出來的那些數字，將決定你在那裡能做多少。

索引

依詞條首字排序，數字為出現的章次。英文詞條排在中文詞條之前。索引依本冊內容編製；跨篇的完整索引見全書合訂本。

英文詞條

3D 平行 19
activation 18、19
activation checkpointing 18
AdamW 18
barrier 19
BF16 18、19、20
checkpoint 18、19、21
DataLoader 18、19
DDP 19
DistributedSampler 19
dropout 18、19
FlashAttention 18、20
FLOPs 18、20、21
FP16 18
FP32 18、19
FP8 18
FSDP 19
fused AdamW 18
GFLOPS/W 21
GPT 21
GPU 使用率 18、21
GradScaler 18
KV cache 18、20
LoRA 18、19
loss scaling 18
MFU 18、19、21
MoE 19、20
no_sync 19
NTK 縮放 20
occupied_idle 21
OOM 18、19
PUE 21
PyTorch 18、19
RAG 18、20
ring all-reduce 19
RMSNorm 18
RNN 20
RoPE 20
softmax 18
SwiGLU 18
Tensor Core 18
TF32 18
torch.compile 18
Transformer 19、20
YaRN 20
ZeRO 19

中文詞條

三年路線 21
上下文長度 20
大海撈針 20
干擾針 20
中段遺失 20
專家平行 19
張量 18、19
張量平行 19
梯度 18、19、20
梯度重疊 19

分片	18、19	梯度累積	18、19
分散式訓練	18、19	混合注意力	20
反向傳播	18、19	混合架構	20
可重現	19、21	混合精度	18、19
台灣杉	21	嵌入	19
正規化	18	晶創	21
交叉熵	18	稀疏注意力	20
全域詞元	20	評測	18、19、20、21
共享記憶體	18	詞元	19、20、21
因果遮罩	18	開放權重	20、21
死結	19	閒置成本	21
串流處理	20	集體操作	19
位置內插	20	微調	20
位置編碼	18、20	滑動視窗	20
困惑度	20	資料平行	19
批次大小	18、19	資源治理	21
注意力	18、19、20	路由	19、20
注意力沉沒	20	運算強度	18
狀態空間模型	20	電力排碳係數	21
長上下文	18、20	電價	21
前饋網路	19、20	圖最佳化	18
屋頂線	18	碳排放	21
活躍參數量	20	算力	18、19、20、21
負載平衡	19、20	算力申請	21
降級方案	21	管線平行	19
個資	20、21	審查重點	21
容量上限	20	融合核心	18
氣泡	19	選擇性	20
能效	21	隨機種子	19
脊點	18	環境指紋	18、19、21
記憶體剖析	18	總持有成本	21
記憶體階層	18	總參數量	20
記憶體碎片	18	繁體中文	20、21
參數量	19、20、21	擴展效率	19
國網中心	19、21		

大模型導論 Introduction to Large Language Models

第 5 篇 算力、系統與現代架構 (第 18–21 章)

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

課程與勘誤 : @prof (<https://page.line.me/prof>)

臺灣自編大學教科書系列 | 2026 年 8 月 第一版